SkillShift-Bench: Benchmarking Skill Learning under Environment Evolution for Web Agents

Bo-Yu Chen[1,2] **Zhi Zhou**[1,2*] **Yu-Feng Li**[1,2*]
[1]School of Artificial Intelligence, Nanjing University
[2]National Key Laboratory for Novel Software Technology, Nanjing University
{chenby, zhouz, liyf}@lamda.nju.edu.cn

Abstract

Skill learning enables web agents to reuse prior experience for long-horizon interactive tasks. However, learned skills are typically evaluated in closed environments, overlooking the fact that real-world web deployment involves continuous shifts in invocation context, perceptual grounding, and execution feasibility. To this end, we introduce SkillShift-Bench, the first benchmark for evaluating skill learning with controlled open-environment shifts. SkillShift-Bench simulates Contextual Shift, Perceptual Shift, and Execution Shift across three self-hosted web systems, and evaluates agents with diagnostics covering task success, interaction cost, skill use, and skill failure. Comprehensive experiments reveal systematic robustness degradation under controlled shifts, characterized by lower success rates, higher interaction costs, and more frequent skill failures, with Execution Shift exerting the strongest pressure. We further formulate shifted skill execution as a nonstationary local executability problem and provide both a dynamic-regret analysis and a practical test-time adaptation method, demonstrating the feasibility of improving learned-skill execution without relearning the skill library. Code, tasks, environments, documentation, and the leaderboard are available at https://skillshift-bench.github.io/.

1 Introduction

Web agents increasingly complete long-horizon tasks by reusing learned skills rather than planning each step from scratch (Wang et al., 2024, 2025b,a). This reuse improves efficiency, but it assumes local compatibility between the current web environment and the context in which the skill was learned. When interface structure, grounding evidence, or runtime conditions change (Cheng et al., 2024; Yang et al., 2025; Kara et al., 2026), a skill may remain semantically relevant while its next operation is no longer locally executable. We study this gap as learned skill executability under same-site environment evolution.

Existing evaluations provide important but incomplete views of this problem. Offline web datasets and simulated web tasks offer broad coverage, but do not test executable skill transfer in interactive browsers (Yao et al., 2022; Deng et al., 2023). Self-hosted benchmarks support reproducible execution, but usually instantiate tasks in fixed website snapshots (Zhou et al., 2024; Koh et al., 2024; Le Sellier de Chezelles et al., 2025). Remote and open web benchmarks expose agents to realistic change (Drouin et al., 2024; He et al., 2024; Pan et al., 2024; Yoran et al., 2024), while task content, interface state, and runtime conditions often vary together, limiting fine-grained attribution (Xue et al., 2025). Recent robustness and evolution benchmarks further show the importance of non-static evaluation (Ishmam and Marino, 2026; Bai et al., 2026; Li et al., 2026; Kara et al., 2026). However, they mainly target general agent robustness rather than whether previously learned reusable skills remain executable when task semantics are preserved.

We introduce SkillShift-Bench, a controlled benchmark for diagnosing learned skill executability under environment evolution. It defines three mismatch modes aligned with the stages of reusable skill execution: Contextual Shift targets skill invocation, Perceptual Shift targets grounding, and Execution Shift targets runtime feasibility. Across these modes, task intent, task-critical entities, required functionality, and grading criteria are preserved. This design turns website evolution into controlled interventions on learned skill reuse rather than task redefinition.

Using SkillShift-Bench, we evaluate representative learned skill methods (Wang et al., 2025b,a; Zheng et al., 2025; Prabhu et al., 2026) across clean, evolved, and shifted environments. The re-

*Corresponding authors.

sults show consistent transfer degradation under controlled same-site evolution, with the strongest pressure under Execution Shift. This pattern suggests that many errors arise from local executability mismatch: the learned library can still contain task-aligned skills, while the executor must select an operation that is feasible in the current-page state.

Motivated by this finding, we formulate shifted skill execution as a nonstationary local executability problem, separating skill availability from current action feasibility. We analyze this setting with SWOP (Sliding Window Optimistic Prior-Guided Planning), an idealized prior-guided adaptation model with a variation-dependent dynamic regret guarantee, following the nonstationary online decision making perspective (Besbes et al., 2014; Cheung et al., 2020; Mao et al., 2021). We then instantiate the same principle as MPCR (Masked Prior-Guided Candidate Reranking), a fixed-library executor-side method that reranks candidate actions using current-page feasibility evidence (Sun et al., 2023; Qin et al., 2024; Wu et al., 2025). Since MPCR keeps the learned skill library fixed, its gains reflect test-time action selection rather than additional skill acquisition.

In summary, we make three contributions:

- We introduce SkillShift-Bench, a controlled benchmark for evaluating learned skill executability under Contextual, Perceptual, and Execution Shifts.
- We formulate shifted skill execution as nonstationary local executability and analyze SWOP with a variation-dependent dynamic regret guarantee.
- We propose MPCR, a test-time, fixed-library executor-side reranker that improves transfer through feasibility-aware action selection.

2 Related Work

2.1 Skill learning and reusable web procedures

Web agents increasingly reuse prior experience, workflows, tools, and executable procedures to support long-horizon interaction (Shinn et al., 2023; Wang et al., 2024). Recent work studies such reuse through workflow memory (Wang et al., 2025b), programmatic skill induction (Wang et al., 2025a), autonomous browser skill discovery (Zheng et al., 2025), learned tools (Prabhu et al., 2026), API-based interaction (Song et al., 2025), routing over large skill libraries (Zheng et al., 2026), and neuro-symbolic induction of logic-grounded skill programs from interaction traces (Shao et al., 2026a). These methods show that reusable procedures can reduce repeated planning and improve web agent efficiency (Wang et al., 2025b,a; Zheng et al., 2025; Prabhu et al., 2026). Our work studies a complementary question: whether a previously acquired procedure remains usable when the same website changes the context, interface evidence, or runtime conditions needed for execution.

2.2 Web agent benchmarks and environment realism

Web agent benchmarks vary in coverage, realism, reproducibility, and attribution. Offline or non-browser settings such as WebShop, Mind2Web, and ChinaTravel provide broad task coverage and, in constraint-rich planning domains, evaluation against explicitly specified requirements rather than only final outcomes (Yao et al., 2022; Deng et al., 2023; Shao et al., 2026b). Self-hosted environments such as World of Bits, WebArena, VisualWebArena, and BrowserGym support reproducible execution in controlled website snapshots (Shi et al., 2017; Zhou et al., 2024; Koh et al., 2024; Le Sellier de Chezelles et al., 2025). Remote hosted benchmarks such as WorkArena and WorkArena++ evaluate agents in realistic workplace environments (Drouin et al., 2024; Boisvert et al., 2024). Live or online web benchmarks such as WebVoyager, WebLinx, WebCanvas, Online Mind2Web, and AssistantBench improve ecological validity by evaluating agents in more realistic interaction settings (He et al., 2024; Lu et al., 2024; Pan et al., 2024; Xue et al., 2025; Yoran et al., 2024). Together, these benchmarks have advanced general web agent evaluation. We focus on an aligned transfer setting for reusable procedures, where task semantics are preserved while local execution conditions vary.

2.3 Environment variation and diagnostic robustness

Recent work further studies how web agents behave under environment variation. REAL evaluates agents in deterministic simulations of real websites, and WebCanvas emphasizes realistic workflows with controllable benchmark construction (Garg et al., 2025; Pan et al., 2024). TimeWarp studies temporal interface change, StressWeb introduces structured interaction

variability, ProEvolve supports programmable benchmark evolution, and WAREX injects website and network failures into existing benchmarks (Ishmam and Marino, 2026; Bai et al., 2026; Li et al., 2026; Kara et al., 2026). These studies show that static evaluation can obscure important robustness behavior (Kara et al., 2026; Bai et al., 2026), a concern that also holds beyond the browser, where statically trained tool-use agents degrade once tool interfaces and specifications depart from their training distribution (Lv et al., 2026), in line with the open-environment view that training-time assumptions need not hold at deployment (Guo et al., 2025). We build on this motivation, but use environment variation as an intervention on learned skill transfer. The focus is not only whether an agent remains successful, but whether a previously useful procedure remains invokable, groundable, and executable under aligned task semantics. Appendix A provides a criterion-level comparison with existing web-agent benchmarks.

2.4 Breakage and repair of recorded web procedures

Software engineering studies an analogous problem as web-test maintenance: recorded end-to-end scripts break when an application evolves, mostly through changed element locators and reorganized page structure rather than removed functionality (Imtiaz et al., 2019; Hammoudi et al., 2016). Remedies either harden locators against structural change, as in robust XPath generation (Leotta et al., 2016), or self-heal a broken script against the evolved interface using structural (Choudhary et al., 2011) or visual (Stocco et al., 2018) evidence of the intended target, while a separate line stabilizes asynchronous waits (Pei et al., 2025). Perceptual Shift instantiates these breakage causes at the level of a learned procedure, perturbing the same locator and DOM-structure signals that these methods defend (Leotta et al., 2016; Stocco et al., 2018). Two differences shape the remedy: a skill is selected at run time rather than executed unconditionally, so a still-valid procedure may be invoked where it no longer applies; and Execution Shift breaks execution through runtime state rather than interface change (Kara et al., 2026), which no repair of a stored script can anticipate. We therefore adapt the executor instead of repairing the library.

2.5 Test-time executor-side adaptation for shifted skill execution

Shifted skill execution is related to nonstationary decision making, GUI grounding, candidate reranking, and reliability analysis. Nonstationary bandits and reinforcement learning study decision making under changing rewards or transitions (Besbes et al., 2014; Cheung et al., 2020; Mao et al., 2021; Feng et al., 2023), and test-time adaptation adjusts a deployed model from unlabeled test-time evidence, including under open-world data shift (Zhou et al., 2023), at the prompt level for zero-shot models (Zhang et al., 2024), and with recent analyses of when such adaptation is learnable (Zhou et al., 2026). GUI grounding methods improve target identification under visual and structural variation (Zheng et al., 2024; Cheng et al., 2024; Gou et al., 2025). Reranking methods support selection among candidate outputs (Sun et al., 2023; Qin et al., 2024; Wu et al., 2025), and reliability studies show that aggregate success may hide unstable execution behavior (Kara et al., 2026). These directions provide useful context for test-time executor adaptation. Our setting connects them through fixed-library skill reuse, where the executor selects among prior-guided skill and primitive candidates using current-page feasibility evidence.

3 SkillShift-Bench: Benchmark Design

SkillShift-Bench is designed to diagnose learned-skill executability under controlled same-site environment evolution. It preserves task semantics across aligned variants and varies only the local conditions that affect skill reuse. The following sections describe the transfer setting, shift modes, benchmark construction, and diagnostic metrics.

3.1 Controlled Transfer Setting

SkillShift-Bench is organized around aligned task families. Each family contains a source context, where a skill is learned or selected, and an evolved target context, where the same task is attempted under changed website conditions. To support fine-grained attribution, we enforce four invariants. Goal invariance keeps the instruction tied to the same task intent. Entity invariance aligns task-critical objects such as projects, issues, products, posts, pages, and user accounts. Functionality invariance ensures that the operations required for completion remain available. Outcome invariance keeps the automatic success criteria equivalent

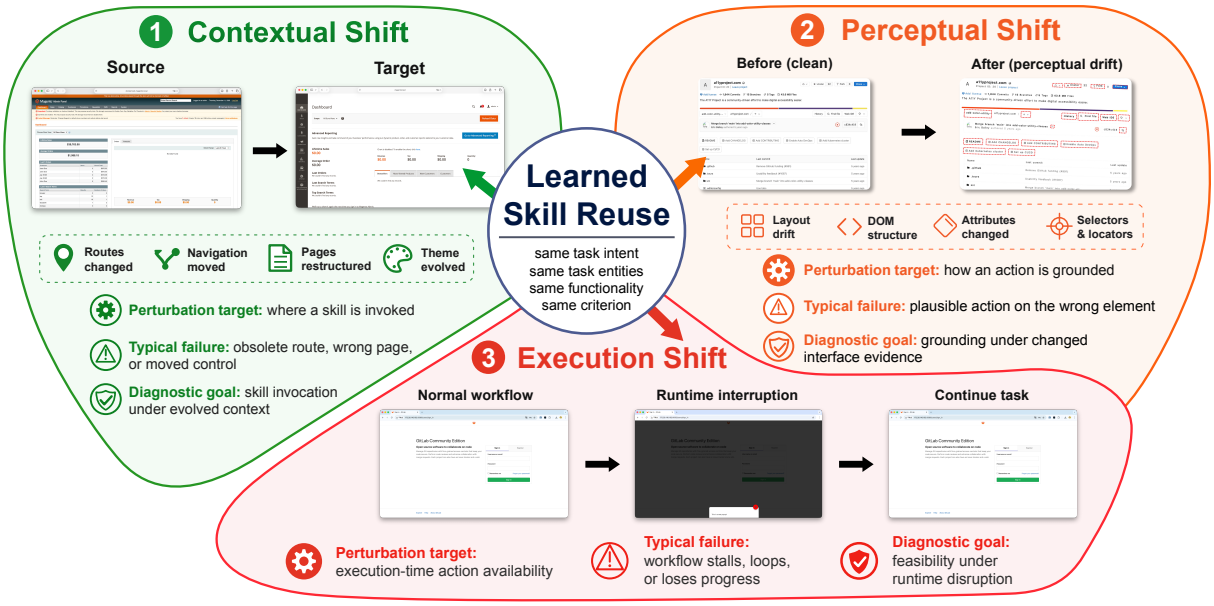


Figure 1: Three controlled shift modes in SkillShift-Bench. Aligned variants preserve task intent, task-critical entities, required functionality, and success criteria, while Contextual, Perceptual, and Execution Shifts respectively target skill invocation, grounding evidence, and runtime feasibility.

across variants.

This design keeps the task fixed while varying the website conditions under which learned skills are reused. SkillShift-Bench therefore evaluates skill transfer under controlled environment evolution rather than under task redefinition.

3.2 Skill Environment Mismatch Modes

A reusable skill must be invoked in a compatible context, grounded to the intended interface evidence, and executed under feasible runtime conditions. SkillShift-Bench defines three mismatch modes that correspond to these stages, as shown in Figure 1.

Contextual Shift. Contextual Shift evaluates skill invocation after the deployment context evolves. We instantiate it with paired versions or themes of the same website, where routes, navigation, page structure, workflow layout, or control placement may change while the task remains aligned.

Perceptual Shift. Perceptual Shift evaluates grounding under changed interface evidence. It applies task-preserving perturbations to layout, DOM structure, element attributes, and locator signals, while keeping the task-critical objects and required functionality intact (Imtiaz et al., 2019; Leotta et al., 2016).

Execution Shift. Execution Shift evaluates runtime feasibility during task execution. It injects reproducible modal blockers that intercept interactions, testing whether the agent can recover from execution-time interruptions and continue the original task flow (Kara et al., 2026).

Implementation details for the perceptual and execution perturbation modules are given in Appendix B.2, together with the configurable injection schedules for Execution Shift in Appendix B.2.3. Appendix B.4 shows that recorded failure trajectories exhibit stage-distinct signatures matching this decomposition.

3.3 Benchmark Instantiation

We instantiate SkillShift-Bench on three controlled web systems: GitLab, Magento, and WordPress. They cover developer collaboration, e-commerce administration, and content management, and each supports reproducible deployment, controlled modification, and automatic state inspection. The selected version and theme pairs follow official release, migration, and theme documentation, with environment-selection criteria and pairing details provided in Appendix B.1.

Task construction. Task construction follows a validity-first pipeline. For each website, we instantiate task-relevant data and align the entities and states needed across source and target contexts. We then write natural language instructions around

these aligned entities and implement task-specific automatic scorers for final state verification. Each task family is checked against the four invariants in Section 3.1 before shifted variants are generated. Only validated families enter the shift generation stage, where local execution conditions are varied while the task remains fixed. Validation is manual and covers instruction validity, scorer correctness, functional availability in both variants, and the existence of an executable solution under each of the six conditions; Appendix B.3 gives the full protocol and its distinction from a human performance baseline. This process yields 355 natural language tasks across three web systems.

Evaluation conditions. Each task family is instantiated under six conditions. `Src` is the clean source context. `Src-Perc` and `Src-Exec` apply Perceptual Shift and Execution Shift to the source context. `Tgt` is the clean target context and measures transfer under Contextual Shift. `Tgt-Perc` and `Tgt-Exec` apply Perceptual Shift and Execution Shift to the target context. Together, these conditions allow controlled comparisons across clean execution, same-site transfer, and additional perceptual or execution shifts under matched tasks.

3.4 Diagnostic Metrics

We characterize each run with four complementary diagnostics. **Success Rate (SR)** measures the fraction of tasks that meet the specified success criterion. **Average Steps (AS)** measures the mean number of actions before success, failure, or timeout, capturing interaction cost and recovery burden. **Average Skill Calls (ASC)** measures the mean number of learned skill invocations per task, capturing reliance on the skill library. **Skill Failure Rate (SFR)** measures the proportion of invoked skills that fail during attempted execution, with runtime errors counted as skill failures. These diagnostics capture final correctness, trajectory cost, skill reuse behavior, and local executability under controlled environment evolution. SFR aggregates retrieval-level and executability-level failures, and is unstable when skills are rarely invoked, since the ratio is then taken over a very small denominator; we therefore report it as a supplementary diagnostic and characterize low-invocation regimes through ASC. Appendix B.4 decomposes the two causes.

4 Prior-Guided Test-Time Executor Adaptation

SkillShift-Bench separates task semantics from local execution conditions by preserving the former while varying the latter across aligned environments. This separation motivates a prior-guided view of skill transfer: a learned library may remain task-relevant, but the executor must still choose actions that are executable in the current page state. This section first formalizes shifted skill execution as nonstationary local executability and then introduces an executor-side reranking method that keeps the learned skill library fixed while adapting test-time action selection.

4.1 SWOP as Prior-Guided Adaptation

We formalize this view with SWOP (Sliding-Window Optimistic Prior-Guided Planning), an idealized finite-horizon model for adapting a skill-guided executor under nonstationary execution conditions. SWOP keeps the executor anchored to a reference prior while using recent evidence to update which actions appear reliable in the current environment.

Following nonstationary episodic MDP formulations (Cheung et al., 2020; Mao et al., 2021; Feng et al., 2023), we model environment evolution as a finite-horizon information-state MDP sequence:

$$\mathcal{M}^{(k)} = \big(\Xi, \{\Omega(\xi)\}_{\xi\in\Xi}, \{r_h^{(k)}\}_{h=1}^{H}, \{P_h^{(k)}\}_{h=1}^{H}, H\big), \quad k = 1, \dots, K. \tag{1}$$

Here Ξ is a finite information-state space, $\Omega(\xi)$ is the state-dependent action domain, and evolution is captured by changes in the rewards $r_h^{(k)}$ and transitions $P_h^{(k)}$.

At each episode, SWOP estimates rewards and transitions from a recent sliding window, adds optimism for uncertainty, and computes a KL-regularized policy around a reference prior π_{ref}, combining sliding-window optimism and regularized MDP ideas (Cheung et al., 2020; Mao et al., 2021; Geist et al., 2019):

$$\widehat{\pi}_h^k(\omega \mid \xi) \propto \pi_{\text{ref}}(\omega \mid \xi) \exp(\widehat{Q}_h^k(\xi, \omega)/\alpha). \tag{2}$$

The prior represents the preference induced by the fixed skill library and base executor. The sliding-window estimate allows the policy to respond to recent execution evidence.

Following variation-budget dynamic-regret analyses in nonstationary bandits and MDPs (Besbes

et al., 2014; Cheung et al., 2020; Mao et al., 2021), we measure nonstationarity by the adjacent reward and transition variation:

$$\Delta_r^{(k)} := \max_{h,\xi,\omega} \left| r_h^{(k+1)}(\xi,\omega) - r_h^{(k)}(\xi,\omega) \right|, \tag{3}$$

$$\Delta_p^{(k)} := \max_{h,\xi,\omega} \left\| P_h^{(k+1)}(\cdot \mid \xi,\omega) - P_h^{(k)}(\cdot \mid \xi,\omega) \right\|_1, \tag{4}$$

and aggregate it as:

$$V_K := \sum_{k=1}^{K-1} \Delta_r^{(k)} + H \sum_{k=1}^{K-1} \Delta_p^{(k)}. \tag{5}$$

SWOP is evaluated by dynamic regret against the episode-wise optimal KL-regularized policy:

$$\mathrm{DynReg}(K) := \sum_{k=1}^{K} \left(V_1^{\star,(k)}(\xi_1^k) - V_1^{\hat{\pi}^k,(k)}(\xi_1^k) \right). \tag{6}$$

Theorem 4.1 (Informal dynamic-regret guarantee)**.** *Consider the finite-state SWOP abstraction with bounded rewards, fixed episode-invariant action domains, a full-support reference prior, and the sampling and concentration conditions stated in Appendix C. Let $\hat{\pi}^k$ be the policy produced by SWOP at episode k. With an oracle-tuned fixed window chosen before the run as a deterministic function of the variation budget V_K, SWOP satisfies, with probability at least $1-\delta$,*

$$\mathrm{DynReg}(K) \le \widetilde{O}\left(K^{2/3} (V_K + 1)^{1/3} \right), \tag{7}$$

where fixed problem-dependent factors are suppressed. Consequently, when H, $|\Xi|$, and $\max_{\xi \in \Xi} |\Omega(\xi)|$ are fixed and $V_K = o(K)$, SWOP obtains sublinear dynamic regret.

This guarantee suggests that, when environment variation grows sublinearly, prior-guided adaptation can track the episode-wise comparator by combining learned priors with recent feasibility evidence. Appendix C provides the formal SWOP statement and proof.

4.2 From SWOP to Practical Executor Design

SWOP is an analysis model, while deployed web agents operate over language-conditioned observations, browser states, page elements, and candidate actions rather than explicit MDP models. We therefore use SWOP as a design guide for test-time action selection. It suggests adapting with recent execution evidence, retaining uncertainty-aware alternatives when prior-preferred actions become unreliable, and regularizing selection toward learned skill preferences. These principles translate into a practical executor objective: selecting among primitive and skill-action candidates at test time based on current-page feasibility evidence. Appendix D.1 states this correspondence componentwise and reports the empirical evidence supporting each of its components.

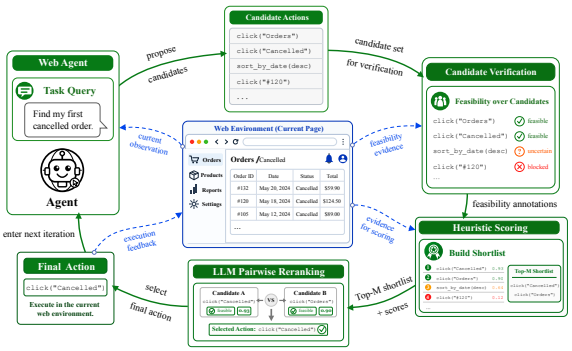


Figure 2: Execution flow of MPCR within the web-agent loop.

4.3 MPCR as Feasibility-Aware Candidate Reranking

MPCR (Masked Prior-Guided Candidate Reranking) instantiates the above principles as an executor-side adaptation method. Figure 2 shows the execution flow.

Given the current observation, task context, fixed skill library, and base executor, MPCR proposes primitive and skill-action candidates, verifies them with current-page feasibility evidence, scores them according to executability, and ranks the shortlist through LLM-based comparison or reranking (Sun et al., 2023; Qin et al., 2024; Wu et al., 2025).

MPCR selects only from proposed candidates and introduces no new skills. Execution feedback is carried into the next decision step. This realizes the SWOP principle in practical web execution while preserving the fixed learned skill library. Appendix D gives the full MPCR algorithm and implementation details.

5 Experiments and Results

We evaluate SkillShift-Bench and MPCR through three research questions: **RQ1** studies how learned-skill methods fail under controlled environment evolution; **RQ2** examines whether the observed adaptation dynamics are consistent with SWOP;

and **RQ3** tests whether test-time executor-side adaptation improves fixed-library transfer.

5.1 Experimental Setup

Benchmark and metrics. We evaluate all methods under the six SkillShift-Bench conditions defined in Section 3.3: `Src`, `Src-Perc`, `Src-Exec`, `Tgt`, `Tgt-Perc`, and `Tgt-Exec`. For compact analysis, we also report three pooled views: *Base* combines `Src` and `Tgt`, *Perceptual* combines `Src-Perc` and `Tgt-Perc`, and *Execution* combines `Src-Exec` and `Tgt-Exec`. We use the four diagnostics defined in Section 3.4: SR, AS, ASC, and SFR.

Compared methods. We compare four skill learning methods: AWM, ASI, SkillWeaver, and WALT (Wang et al., 2025b,a; Zheng et al., 2025; Prabhu et al., 2026). Each method learns skills in `Src` using its native protocol and is evaluated under the aligned transfer conditions. Method cards and library sizes are provided in Appendix E.1 and Appendix E.2.

Fixed-library protocol. For RQ2 and RQ3, ASI first learns a skill library on `Src`. The library is then frozen and shared by ASI and MPCR. ASI uses its original selector, while MPCR changes only test-time selection over the same skills and primitive actions. This protocol isolates executor-side adaptation from additional skill learning.

Models and execution. The benchmark-level analysis uses DeepSeek-V3.2, GPT-5 mini, and Claude Haiku 4.5 (DeepSeek-AI, 2025; OpenAI, 2025; Anthropic, 2025). The fixed-library ASI and MPCR comparison uses GPT-5 mini. MPCR considers at most 2 candidate actions per step and keeps a top-2 shortlist, evaluating it as a lightweight test-time reranking method rather than a broad-search procedure. Appendix D.8.2 sweeps this candidate budget and reports the empirical basis for the default. Appendix E.3 further reports backbone-level diagnostic results.

5.2 RQ1: How Do Existing Learned-Skill Methods Fail under Controlled Environment Evolution?

Figure 3 and Table 1 show a consistent decline from clean source execution to evolved and shifted environments across all evaluated methods. For example, ASI decreases from 53.65% on `Src` to 41.84% on `Tgt`, and to 15.79% on `Tgt-Exec`. Similar trends appear for AWM, SkillWeaver, and WALT.

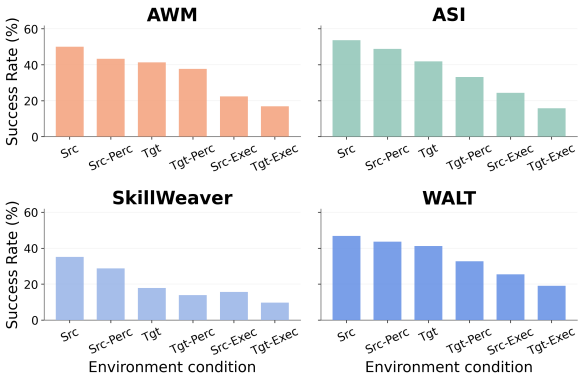


Figure 3: Robustness profiles under controlled environment evolution.

Under the aligned transfer setting, these drops indicate that skill reuse is sensitive to local execution conditions rather than only to task-level relevance.

The pooled views separate the pressure introduced by different shifts. Success decreases from 40.97% in the *Base* view to 35.23% in *Perceptual* and 18.69% in *Execution*. Trajectory cost also increases across the same views, with average steps rising from 8.49 to 9.38 and 11.99, while skill failure rate rises from 9.83% to 16.70% and 27.60%. This pattern shows that environment evolution affects both task completion and the execution process, with runtime feasibility shifts creating the strongest pressure.

The diagnostics also reveal complementary failure channels. AWM is evaluated through workflow reuse, so explicit skill-call diagnostics are not applicable. ASI and WALT expose execution pressure through skill failure rate, while SkillWeaver often reflects it through longer trajectories and more skill calls. These patterns motivate reporting process-level diagnostics in addition to final success.

5.3 RQ2: Are the Observed Adaptation Dynamics Consistent with the SWOP Theoretical Account?

SWOP models shifted skill execution as nonstationary local executability, where an executor stays close to learned priors while using recent feasibility evidence. Since the theoretical per-step loss is not directly observable in web interaction, we use final failure as a task-level proxy.

For method m and task i, let $Y_{m,i} \in \{0, 1\}$ be the automatic success label and $L_{m,i} = 1 - Y_{m,i}$ the proxy failure loss. We define the cumulative advantage of MPCR over ASI as $A(K) = \sum_{i=1}^{K}(L_{\mathrm{ASI},i} - L_{\mathrm{MPCR},i})$, where positive values

Metric	Method	Website			Environment						O/A
		Git-Lab	Ma-gento	Word-Press	Src	Src-Perc	Src-Exec	Tgt	Tgt-Perc	Tgt-Exec	
SR (%)	AWM	17.71	25.49	**62.66**	50.07	43.35	22.45	41.28	**37.65**	16.90	35.29
	ASI	20.81	27.78	60.27	**53.65**	**48.80**	24.43	**41.84**	33.19	15.79	**36.29**
	SkillWeaver	13.78	14.37	32.29	35.13	28.75	15.63	17.80	13.85	9.71	20.15
	WALT	**21.07**	**27.95**	55.38	46.82	43.53	**25.46**	41.17	32.74	**19.09**	34.80
AS	AWM	8.03	8.22	4.76	6.04	6.32	8.31	6.34	**6.59**	8.44	7.01
	ASI	**7.87**	**8.06**	**4.73**	**5.90**	**5.95**	**8.23**	**6.29**	6.70	**8.24**	**6.89**
	SkillWeaver	14.79	17.96	12.63	11.34	13.48	19.82	12.59	14.74	18.80	15.13
	WALT	12.39	11.46	8.53	8.77	9.92	12.09	10.66	11.36	11.96	10.79
ASC	AWM	–	–	–	–	–	–	–	–	–	–
	ASI	0.12	0.19	0.17	0.18	0.26	0.29	0.10	0.10	0.05	0.16
	SkillWeaver	0.78	0.63	2.64	1.13	1.16	1.14	1.45	1.50	1.73	1.35
	WALT	0.07	0.19	0.00	0.05	0.05	0.21	0.08	0.03	0.11	0.09
SFR (%)	AWM	–	–	–	–	–	–	–	–	–	–
	ASI	19.55	48.27	16.26	9.83	24.85	55.20	13.53	28.77	35.98	28.03
	SkillWeaver	**4.61**	**1.11**	0.21	**0.00**	**0.41**	**0.00**	**2.03**	**4.24**	**5.17**	**1.97**
	WALT	48.52	23.88	**0.00**	10.30	23.29	31.50	23.28	18.64	37.76	24.13

Table 1: Transfer diagnostics across websites and environment settings. O/A denotes the overall average across the corresponding settings. Bold values mark the highest SR and the lowest AS / SFR entries. ASC is reported as a non-directional diagnostic. "–" indicates metrics not applicable to AWM because it uses prompted textual workflows rather than executable skill invocations (Wang et al., 2025b).

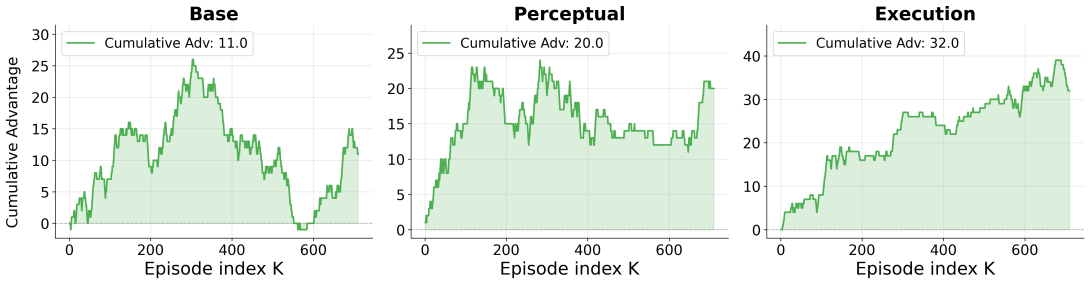


Figure 4: Cumulative task-failure advantage of MPCR over ASI.

indicate fewer accumulated failures under the same fixed skill library.

Figure 4 shows positive cumulative advantage in all condition groups. The final advantage is 11.0 under *Base*, 20.0 under *Perceptual*, and 32.0 under *Execution*. This ordering follows the executability pressure observed in RQ1. When the learned prior remains reliable, the room for correction is smaller. When grounding evidence changes or runtime blockers appear, current feasibility evidence becomes more useful for action selection.

These dynamics are consistent with the SWOP mechanism. In practical web execution, MPCR follows the same prior-guided adaptation principle by conditioning test-time action selection on current execution evidence. The stronger advantage under shifted conditions suggests that part of the transfer loss reflects current executability mismatch, which can be partly mitigated through executor adaptation.

5.4 RQ3: Can Test-Time Executor-Side Adaptation Improve Transfer under a Fixed Skill Library?

Table 2 shows that MPCR improves ASI in all six environments. Average success increases from 37.90% to 41.26%, corresponding to an 8.87% relative improvement.

The gains are clearest when local execution becomes more fragile. MPCR improves `Src-Exec` from 26.68% to 33.23% and `Tgt-Exec` from 17.75% to 19.99%, with relative gains of 24.55% and 12.62%. It also maintains a positive gain in clean target transfer, where the learned prior is already more aligned with the current environment. This pressure-dependent pattern supports the local

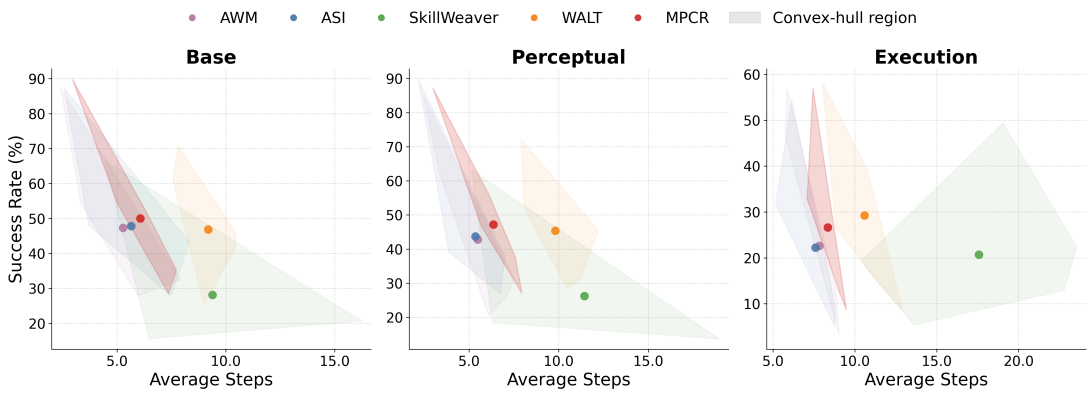


Figure 5: Success and trajectory cost for fixed-library executor adaptation. Points denote method centers across environment condition groups, and shaded hulls show configuration-level variation.

Environment	MPCR		ASI	
	SR (%)	ASC	SR (%)	ASC
Src	56.67 (+7.35%)	0.22	52.79	0.05
Src-Perc	57.09 (+7.68%)	0.20	53.02	0.13
Src-Exec	33.23 (+24.55%)	0.19	26.68	0.07
Tgt	43.32 (+1.14%)	0.12	42.83	0.04
Tgt-Perc	37.24 (+8.51%)	0.14	34.32	0.05
Tgt-Exec	19.99 (+12.62%)	0.03	17.75	0.02
Average	**41.26** (+8.87%)	**0.15**	**37.90**	**0.06**

Table 2: MPCR versus ASI across transfer conditions. Relative SR gains are computed against ASI.

executability framing: executor adaptation is most useful when the learned skill remains relevant but direct reuse becomes less reliable.

Because ASI and MPCR share the same learned library, these improvements are attributable to test-time selection rather than additional skill acquisition. MPCR increases average skill calls from 0.06 to 0.15, while remaining well below one skill call per task. This indicates evidence-conditioned activation of existing skills rather than broad reuse.

Figure 5 further shows that the improvement is not simply obtained by longer trajectories. Across *Base*, *Perceptual*, and *Execution*, MPCR improves success over ASI while staying in a comparable trajectory-cost regime. This suggests that feasibility-aware selection improves fixed-library transfer behavior by recovering part of the loss caused by local executability mismatch.

Supplementary analyses support this attribution. Appendix D.8.1 ablates the executor stages and shows, through a randomized control that preserves the pipeline while emitting no reranking call, that the gain follows from the content of the feasibility-conditioned decision rather than from an extra model call per step. Appendix E.4 reproduces the same pressure-dependent pattern on a second learned library, Appendix E.5 reports cross-seed dispersion indicating that the gain is stable, and Appendix E.7 contrasts this test-time route with relearning the library in the target environment.

6 Conclusion

We introduced SkillShift-Bench, a controlled benchmark for evaluating learned-skill executability under same-site environment evolution. By preserving task semantics while varying contextual, perceptual, and execution conditions, it exposes cases where task-relevant skills become difficult to invoke, ground, or execute. We further formalized this skill-environment mismatch with SWOP and instantiated it as MPCR, which improves fixed-library transfer through feasibility-aware reranking. These results suggest that robust web agents should complement reusable skills with test-time adaptation to current feasibility evidence under evolving environments.

Limitations

This work is designed as a controlled diagnostic study. SkillShift-Bench focuses on self-hosted same-site evolution over representative web systems, which supports reproducible execution and reliable failure attribution but leaves broader website coverage to future work. Our experiments mainly use relatively capable LLM-based web agents to reduce confounding from basic instruction-following or tool-use failures, and future studies can further examine smaller or more resource-constrained models. The current benchmark also focuses on English desktop web interac-

tions, with multilingual, mobile, and accessibility-oriented settings as natural extensions.

Ethics Statement

SkillShift-Bench is evaluated in controlled, self-hosted, and authorized web environments using scripted tasks, synthetic or benchmark-specific accounts, seeded task data, and automatic grading. It does not interact with third-party live services, real user data, or private user accounts. Before release, we check task instructions, seeded entities, account names, and grader-visible content to avoid personally identifying information and offensive content, and replace any accidental cases with synthetic placeholders. The perturbations are designed to study environment evolution and execution robustness, not to bypass access controls, evade platform policies, or automate unauthorized activity. We use third-party systems, baseline methods, and model APIs under their released licenses, documentation, or provider terms. Released artifacts will include only benchmark-specific code, task definitions, perturbation scripts, and metadata that we are permitted to distribute, and are intended for reproducible robustness analysis and research evaluation under controlled-use settings.

Acknowledgment

This research was supported by the Jiangsu Science Foundation (BK20243012, BG2024036), Natural Science Foundation of China (624B2068, 62576162), and the Fundamental Research Funds for the Central Universities (022114380023).

References

Adobe. 2023. Adobe Commerce 2.4.6 release notes. https://experienceleague.adobe.com/en/docs/commerce-operations/release/notes/adobe-commerce/2-4-6. Release announcement; Published: 2023-03-14; Accessed: 2026-05-05.

Adobe. 2024. Install the Data Migration Tool. https://experienceleague.adobe.com/en/docs/commerce-operations/tools/data-migration/basics/install. Documentation; Accessed: 2026-05-05.

Anthropic. 2025. Introducing Claude Haiku 4.5. https://www.anthropic.com/news/claude-haiku-4-5. Release announcement; Published: 2025-10-15; Accessed: 2026-05-05.

Haoyue Bai, Dong Wang, Long Chen, Bingguang Hao, Pengyang Shao, Yonghui Yang, Yicheng He, and Chenyi Zhuang. 2026. StressWeb: A diagnostic benchmark for web agent robustness under realistic interaction variability. *Preprint*, arXiv:2604.16385.

Omar Besbes, Yonatan Gur, and Assaf Zeevi. 2014. Stochastic multi-armed-bandit problem with non-stationary rewards. In *Advances in Neural Information Processing Systems*, volume 27, pages 199–207, Montreal, Quebec, Canada.

Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier de Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. 2024. WorkArena++: Towards compositional planning and reasoning-based common knowledge work tasks. In *Advances in Neural Information Processing Systems*, volume 37, pages 5996–6051, Vancouver, Canada. Neural Information Processing Systems Foundation, Inc.

Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, Bangkok, Thailand. Association for Computational Linguistics.

Wang Chi Cheung, David Simchi-Levi, and Ruihao Zhu. 2020. Reinforcement learning for non-stationary Markov decision processes: The blessing of (More) optimism. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 1843–1854. PMLR.

Shauvik Roy Choudhary, Dan Zhao, Husayn Versee, and Alessandro Orso. 2011. WATER: Web application TEst repair. In *Proceedings of the First International Workshop on End-to-End Test Script Engineering*, ETSE 2011, pages 24–29. ACM.

DeepSeek-AI. 2025. DeepSeek-V3.2 release. https://api-docs.deepseek.com/news/news251201. Release announcement; Published: 2025-12-01; Accessed: 2026-05-05.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, volume 36, pages 28091–28114, New Orleans, Louisiana, USA. Neural Information Processing Systems Foundation, Inc.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. 2024. WorkArena: How capable are web agents at solving common knowledge work tasks? In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11642–11662. PMLR.

Songtao Feng, Ming Yin, Ruiquan Huang, Yu-Xiang Wang, Jing Yang, and Yingbin Liang. 2023. Non-stationary reinforcement learning under general function approximation. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 9976–10007. PMLR.

Div Garg, Diego Caples, Andis Draguns, Nikil Ravi, Pranav Putta, Naman Garg, Prannay Hebbar, Youngchul Joo, Jindong Gu, Charles London, Christian Schroeder de Witt, and Sumeet Motwani. 2025. REAL: Benchmarking autonomous agents on deterministic simulations of real websites. In *Advances in Neural Information Processing Systems*, volume 38, pages 150683–150705. Neural Information Processing Systems Foundation, Inc.

Matthieu Geist, Bruno Scherrer, and Olivier Pietquin. 2019. A theory of regularized Markov decision processes. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2160–2169. PMLR.

GitLab. 2019. GitLab 12.0 released with visual reviews, project dependency list, access limiting by IP address. https://gitlab.com/gitlab-com/www-gitlab-com/-/blob/marketing-ops-20220425-20220508/sites/uncategorized/source/releases/posts/2019-06-22-gitlab-12-0-released.html.md. Release announcement; Published: 2019-06-22; Accessed: 2026-05-14.

GitLab. 2023a. GitLab 16.0 released with value streams dashboards and generative AI code suggestions. https://about.gitlab.com/releases/2023/05/22/gitlab-16-0-released/. Release announcement; Published: 2023-05-22; Accessed: 2026-05-05.

GitLab. 2023b. GitLab Security Release: 16.1.1, 16.0.6, and 15.11.10. https://docs.gitlab.com/releases/patches/patch-release-gitlab-16-1-1-released/. Release announcement; Published: 2023-06-29; Accessed: 2026-05-14.

GitLab. 2026. Plan Your Upgrade Path. https://docs.gitlab.com/update/upgrade_paths/. Documentation; Accessed: 2026-05-25.

Boyu Gou, Demi Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. 2025. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In *International Conference on Learning Representations*.

Lan-Zhe Guo, Lin-Han Jia, Jie-Jing Shao, and Yu-Feng Li. 2025. Robust semi-supervised learning in open environments. *Frontiers of Computer Science*, 19(8):198345.

Mouna Hammoudi, Gregg Rothermel, and Paolo Tonella. 2016. Why do record/replay tests of web applications break? In *2016 IEEE International Conference on Software Testing, Verification and Validation*, ICST 2016, pages 180–190. IEEE.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6864–6890, Bangkok, Thailand. Association for Computational Linguistics.

Javaria Imtiaz, Salman Sherin, Muhammad Uzair Khan, and Muhammad Zohaib Iqbal. 2019. A systematic literature review of test breakage prevention and repair techniques. *Information and Software Technology*, 113:1–19.

Md Farhan Ishmam and Kenneth Marino. 2026. TimeWarp: Evaluating web agents by revisiting the past. *Preprint*, arXiv:2603.04949.

Su Kara, Fazle Elahi Faisal, and Suman Nath. 2026. WAREX: Web agent reliability evaluation on existing benchmarks. *Transactions on Machine Learning Research*, 2026.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. 2024. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905, Bangkok, Thailand. Association for Computational Linguistics.

Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Graham Neubig, Quentin Cappart, Russ Salakhutdinov, and Nicolas Chapados. 2025. The BrowserGym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025.

Maurizio Leotta, Andrea Stocco, Filippo Ricca, and Paolo Tonella. 2016. ROBULA+: An algorithm for generating robust XPath locators for web testing. *Journal of Software: Evolution and Process*, 28(3):177–204.

Sergey Levine. 2018. Reinforcement learning and control as probabilistic inference: Tutorial and review. *Preprint*, arXiv:1805.00909.

Guangrui Li, Yaochen Xie, Yi Liu, Ziwei Dong, Xingyuan Pan, Tianqi Zheng, Jason Choi, Michael J. Morais, Binit Jha, Shaunak Mishra, Bingrou Zhou, Chen Luo, Monica Xiao Cheng, and Dawn Song. 2026. The World Won't Stay Still:

Programmable evolution for agent benchmarks. *Preprint*, arXiv:2603.05910.

Xing Han Lu, Zdeněk Kasner, and Siva Reddy. 2024. WebLINX: Real-world website navigation with multi-turn dialogue. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 33007–33056. PMLR.

Song-Lin Lv, Weiming Wu, Rui Zhu, Zi-Jian Cheng, and Lan-Zhe Guo. 2026. Can agents generalize to the open world? Unveiling the fragility of static training in tool use. In *Proceedings of the 43rd International Conference on Machine Learning*.

Magento. 2017. Magento Open Source release notes (1.9 and later). `https://devdocs-openmage.org/guides/m1x/ce19-ee114/ce1.9_release-notes.html`. Release announcement; Archived Magento 1.x release notes; Accessed: 2026-05-25.

Weichao Mao, Kaiqing Zhang, Ruihao Zhu, David Simchi-Levi, and Tamer Basar. 2021. Near-optimal model-free reinforcement learning in non-stationary episodic MDPs. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 7447–7458. PMLR.

OpenAI. 2025. GPT-5 mini. `https://platform.openai.com/docs/models/gpt-5-mini`. Documentation; Accessed: 2026-05-05.

Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, and Zhengyang Wu. 2024. WebCanvas: Benchmarking web agents in online environments. In *Agentic Markets @ ICML 2024*.

Yu Pei, Jeongju Sohn, Sarra Habchi, and Mike Papadakis. 2025. Non-flaky and nearly optimal time-based treatment of asynchronous wait web tests. *ACM Transactions on Software Engineering and Methodology*, 34(2):45.

Viraj Prabhu, Yutong Dai, Matthew Fernandez, Krithika Ramakrishnan, Jing Gu, Yanqi Luo, Silvio Savarese, Caiming Xiong, Junnan Li, Zeyuan Chen, and Ran Xu. 2026. WALT: Web agents that learn tools. In *International Conference on Learning Representations*.

Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. 2024. Large language models are effective text rankers with pairwise ranking prompting. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 1504–1518, Mexico City, Mexico. Association for Computational Linguistics.

Jie-Jing Shao, Haiyan Yin, Yueming Lyu, Xingrui Yu, Lan-Zhe Guo, Ivor W. Tsang, James T. Kwok, and Yu-Feng Li. 2026a. Lifting traces to logic: Programmatic skill induction with neuro-symbolic learning for long-horizon agentic tasks. In *Proceedings of the 43rd International Conference on Machine Learning*.

Jie-Jing Shao, Bo-Wen Zhang, Xiao-Wen Yang, Baizhi Chen, Siyu Han, Jinghao Pang, Wen-Da Wei, Guohao Cai, Zhenhua Dong, Lan-Zhe Guo, and Yu-Feng Li. 2026b. ChinaTravel: An open-ended travel planning benchmark with compositional constraint validation for language agents. In *International Conference on Learning Representations*.

Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. 2017. World of bits: An open-domain platform for web-based agents. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3135–3144. PMLR.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, New Orleans, Louisiana, USA. Neural Information Processing Systems Foundation, Inc.

Yueqi Song, Frank F. Xu, Shuyan Zhou, and Graham Neubig. 2025. Beyond browsing: API-based web agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 11066–11085, Vienna, Austria. Association for Computational Linguistics.

Andrea Stocco, Rahulkrishna Yandrapally, and Ali Mesbah. 2018. Visual web test repair. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2018, pages 503–514. ACM.

Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14918–14937, Singapore. Association for Computational Linguistics.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.

Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. 2025a. Inducing programmatic skills for agentic tasks. In *Second Conference on Language Modeling*.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025b. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR.

WordPress.org. 2017. News Portal WordPress theme. https://wordpress.org/themes/news-portal/. Theme page; Initial release: 2017-07-11; Accessed: 2026-05-25.

WordPress.org. 2023. Twenty Twenty-Four. https://wordpress.org/themes/twentytwentyfour/. Theme page; Accessed: 2026-05-05.

Jingyu Wu, Aditya Shrivastava, Jing Zhu, Alfy Samuel, Anoop Kumar, and Daben Liu. 2025. LLM optimization unlocks real-time pairwise reranking. *Preprint*, arXiv:2511.07555.

Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An illusion of progress? assessing the current state of web agents. In *Second Conference on Language Modeling*.

Jingqi Yang, Zhilong Song, Jiawei Chen, Mingli Song, Sheng Zhou, Linjun Sun, Xiaogang Ouyang, Chun Chen, and Can Wang. 2025. GUI-Robust: A comprehensive dataset for testing GUI agent robustness in real-world anomalies. *Preprint*, arXiv:2506.14477.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*, volume 35, pages 20744–20757, New Orleans, Louisiana, USA. Neural Information Processing Systems Foundation, Inc.

Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. 2024. AssistantBench: Can web agents solve realistic and time-consuming tasks? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8938–8968, Miami, Florida, USA. Association for Computational Linguistics.

Ding-Chu Zhang, Zhi Zhou, and Yu-Feng Li. 2024. Robust test-time adaptation for zero-shot prompt tuning. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence*, volume 38, pages 16714–16722.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. 2025. SkillWeaver: Web agents can self-improve by discovering and honing skills. *Preprint*, arXiv:2504.07079.

Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) is a generalist web agent, if grounded. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 61349–61385. PMLR.

YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. 2026. SkillRouter: Skill routing for LLM agents at scale. *Preprint*, arXiv:2603.22455.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*.

Zhi Zhou, Lan-Zhe Guo, Lin-Han Jia, Ding-Chu Zhang, and Yu-Feng Li. 2023. ODS: Test-time adaptation in the presence of open-world data shift. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 42574–42588.

Zhi Zhou, Ming Yang, Shi-Yu Tian, Kun-Yang Yu, Lan-Zhe Guo, and Yu-Feng Li. 2026. On the learnability of test-time adaptation: A recovery complexity perspective. In *Proceedings of the 43rd International Conference on Machine Learning*.

A Comparison with Existing Web Agent Benchmarks

To situate SkillShift-Bench within the broader design space of web-agent evaluation, Table 3 summarizes key properties of representative benchmarks and evaluation frameworks.

Evaluation criteria. **Variation Diversity** measures whether a benchmark covers multiple qualitatively distinct types of environment variation. **Variation Controllability** measures whether the benchmark supports controlled and reproducible environment perturbations. **Evaluation Reproducibility** measures whether tasks, states, perturbations, and grading results can be stably reproduced. **Website Fidelity** measures whether the benchmark preserves realistic web interfaces, interaction structures, page complexity, and long-horizon workflows. **Diagnostic Granularity** measures whether the benchmark supports evaluation through complementary metrics across multiple dimensions. **Experience Transferability** measures whether the benchmark evaluates how robustly reusable experience or procedures acquired by an agent remain effective under environment changes. **Failure Attributability** measures whether the benchmark supports attributing agent failures to specific components of the evaluation process.

Static and live web benchmarks. Mind2Web, WebArena, VisualWebArena, WorkArena, WorkArena++, Online-Mind2Web, WebCanvas, and REAL provide important foundations for web-agent evaluation (Deng et al., 2023; Zhou et al., 2024; Koh et al., 2024; Drouin et al., 2024; Boisvert et al., 2024; Xue et al., 2025; Pan et al., 2024; Garg et al., 2025). They cover offline action prediction, executable self-hosted tasks, enterprise workflows, live web evaluation, and deterministic website replicas. These benchmarks are valuable for measuring general web-agent capability across realistic or executable web settings. However, they are not primarily designed to compare aligned variants under controlled environment evolution. As a result, they usually measure whether an agent can solve a task in a given environment, rather than whether experience acquired in one environment remains valid after the same task is transferred to a changed environment.

Robustness and evolution benchmarks. WAREX, TimeWarp, ProEvolve, and StressWeb are closer to SkillShift-Bench because they explicitly study environment change, perturbation, or reliability. WAREX injects website and network failures into existing benchmarks, making runtime instability visible (Kara et al., 2026). TimeWarp evaluates agents across temporally different interface versions (Ishmam and Marino, 2026). ProEvolve studies programmable evolution of agent environments (Li et al., 2026). StressWeb introduces controlled interaction variability for diagnostic robustness evaluation (Bai et al., 2026). These benchmarks show that environment variation can substantially affect web-agent performance. However, their main evaluation object is still general agent robustness under shifted, perturbed, or harder conditions. In other words, environment variation is primarily used as a test condition for end-to-end agent or multimodal-model capability.

Skill transfer as the evaluation object. SkillShift-Bench studies a different unit of evaluation. In our setting, the source environment is not merely a baseline condition; it is the context in which reusable skills are acquired, selected, or induced. The target and shifted environments then test whether the same learned skills remain executable under preserved task intent, target entities, functional support, and grading semantics. Therefore, the measured gap is not simply an end-to-end performance drop under a harder environment. It is a skill-transfer gap: a previously useful skill may remain semantically relevant while becoming locally non-executable because its invocation context, perceptual grounding evidence, or runtime preconditions no longer match the deployment environment. This setting is especially important for skill-based agents because skill reuse carries implicit assumptions about when the skill should be invoked, what interface evidence grounds its next action, and which local conditions must hold for execution (Wang et al., 2025b,a; Zheng et al., 2025; Prabhu et al., 2026).

B Benchmark Construction Details

This section describes how SkillShift-Bench is constructed to support controlled evaluation under environment evolution. We detail the selection of environments and source–target pairs, the validation of aligned task families, and the implementation of shifted variants.

Benchmark	Variation Diversity	Variation Controllability	Evaluation Reproducibility	Website Fidelity	Diagnostic Granularity	Experience Transferability	Failure Attributability
Mind2Web	✗	✗	✓	△	△	✗	✗
WebArena	✗	✗	✓	✓	△	✗	✗
Visual WebArena	✗	✗	✓	✓	△	✗	✗
WorkArena / WorkArena++	✗	✗	△	✓	△	✗	✗
WebCanvas	△	✗	△	✓	✓	✗	△
Online-Mind2Web	✗	✗	△	✓	△	✗	✗
REAL	✗	✗	✓	✓	△	✗	✗
WAREX	✓	✓	✓	△	✓	✗	✓
TimeWarp	✗	✓	✓	✓	✗	△	△
ProEvolve	✓	✓	✓	✗	△	△	△
StressWeb	✓	✓	✓	△	✓	✗	✓
SkillShift-Bench (Ours)	✓	✓	✓	✓	✓	✓	✓

Table 3: Comparison of benchmarks across evaluation criteria. ✓ = explicitly supported as a central benchmark objective under the criterion definitions above, △ = partially supported or indirectly covered, and ✗ = not a primary target of the original benchmark design or not publicly verifiable.

B.1 Environment Selection and Version Pairing

SkillShift-Bench selects environments and version pairs under two levels of constraints. At the website level, candidate systems must be mature open-source projects that support complete local self-hosted deployment. This requirement decouples evaluation from the public Internet and removes uncontrolled factors such as live service updates, network variability, third-party availability, and account-side changes. In addition, the targeted notion of environment evolution must primarily reflect structural changes in the front-end interaction layer rather than ordinary data-content updates. We therefore exclude websites whose evolution is mainly expressed as changing data content, such as map or knowledge-base growth, because such changes would entangle UI robustness with dynamic information retrieval.

At the version-pair level, the source and target variants must satisfy two further criteria. First, the version gap must contain substantial interface evolution, preferably a discontinuity in design language, navigation organization, workflow structure, or interaction pattern. Minor cosmetic changes are insufficient, because they do not create a meaningful test of learned-skill executability. Second, cross-version data migration must be feasible. For version-driven systems, we follow the official upgrade or migration path, including required intermediate upgrade points when applicable, so that the agent faces the same task-level semantic data state across versions (GitLab, 2026; Adobe, 2024). This means that task-critical entities, attributes, relations, and grader-visible outcomes are preserved even when the underlying database schema changes.

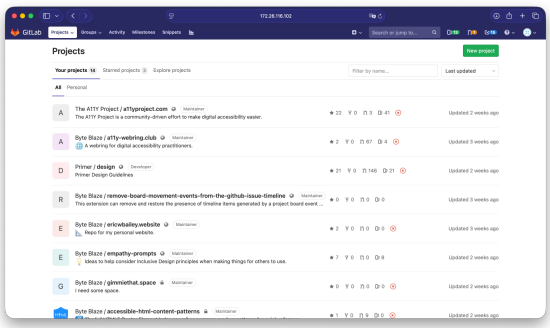
Applying these criteria, we instantiate SkillShift-Bench with three complementary environments: GitLab, Magento, and WordPress. They cover developer collaboration, e-commerce administration, and

content management, while exposing different forms of interface evolution under reproducible self-hosted deployments.

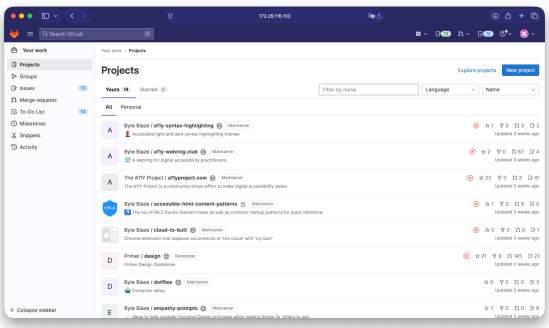
GitLab. GitLab provides the developer-platform setting in SkillShift-Bench. The selected tasks cover repositories, issues, merge requests, labels, milestones, project settings, and user or group administration. These workflows require agents to operate inside nested project contexts and to rely on navigation paths, sidebars, management panels, and local action controls. As a mature open-source platform with self-hosted deployment support, GitLab satisfies our deployment criterion and can be reset in isolated containers. More importantly, its version evolution affects the interaction layer rather than only the underlying project data: project navigation, management pages, issue views, merge-request views, and settings workflows all undergo interface-level changes.

We use GitLab **v12.0.0** (GitLab, 2019) as the source variant and GitLab **v16.0.6** (GitLab, 2023b) as the target variant. This pair creates a realistic contextual-shift setting. Core developer operations remain available, but the routes, page contexts, menu hierarchy, and action locations needed to execute those operations may differ across versions (GitLab, 2023a). To preserve task-level comparability, the deployment is migrated through a controlled upgrade procedure, so that projects, issues, labels, milestones, users, permissions, and other benchmark entities remain semantically aligned. Under this setup, a transfer failure is not explained by missing GitLab functionality or changed task data; it is instead evidence that a previously learned skill no longer matches the evolved execution context.

Figure 6 shows the GitLab source-target pair on task-relevant project-management pages. The screenshots are included as construction evidence rather than as grading targets, illustrating the interface changes under which the same developer-platform operations must still be executed.



(a) GitLab v12.0.0 source variant.



(b) GitLab v16.0.6 target variant.

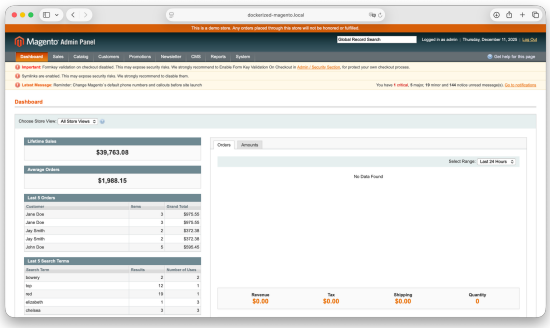
Figure 6: GitLab source and target variants.

Magento. Magento serves as the e-commerce administration environment. Its task space includes products, categories, customers, orders, inventory, attributes, and multi-field administrative forms. Compared with GitLab, the main pressure here comes less from nested project contexts and more from table- and form-heavy record manipulation: agents must search, filter, inspect, edit, and save structured business objects. Magento is suitable for our website-level criteria because it is a mature open-source e-commerce system that supports private deployment and deterministic reset. Its evolution is also interaction-centric. The product catalog is not treated as the evolving object; instead, the benchmark focuses on how the administrative interface for operating on the same catalog entities changes across major versions.

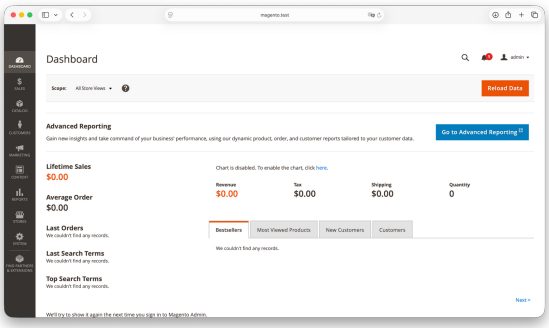
For the source variant, we use **v1.9.3** (Magento, 2017); for the target variant, we use **v2.4.6** (Adobe, 2023). This pair captures a major break in the Magento administration experience. Magento 2 reorganizes menus, grids, form layouts, product-editing workflows, and save flows relative to Magento 1, producing a strong test of version-driven skill transfer (Adobe, 2024, 2023). To keep the comparison focused on interface evolution, both deployments are initialized from a shared canonical benchmark store state and materialized through version-specific setup procedures (Adobe, 2024). This preserves task-critical products, categories, customers, orders, inventory records, and attributes at the semantic level. Thus, the

e-commerce operation required by the instruction remains the same, while the interaction surface through which the operation is carried out changes substantially.

Figure 7 presents the Magento source-target pair on task-relevant administration pages. The visual comparison highlights the administrative UI migration that agents must handle while operating over aligned catalog-management data.



(a) Magento v1.9.3 source variant.



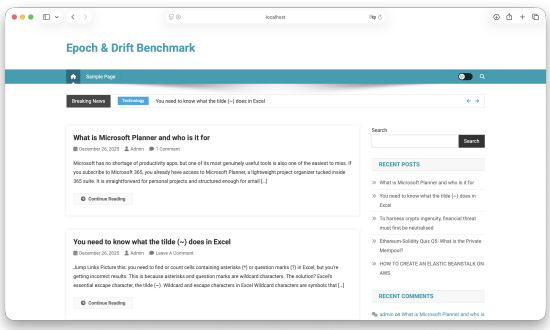
(b) Magento v2.4.6 target variant.

Figure 7: Magento source and target variants.

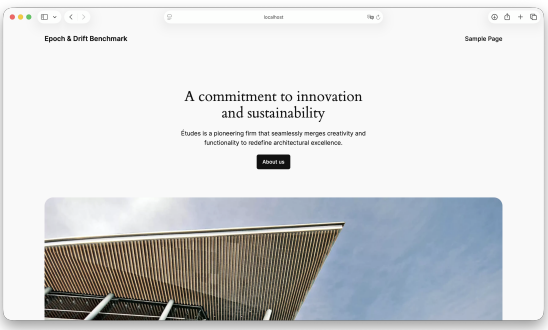
WordPress. WordPress adds a complementary content-management setting. The tasks involve posts, pages, menus, media, comments, and public-facing content verification. Unlike GitLab and Magento, WordPress is not used primarily as a back-end version migration case. Instead, it isolates presentation-layer evolution: the same content-management state is evaluated under different themes. This makes WordPress useful for separating UI and data evolution. The website itself satisfies our deployment criterion as a mature open-source system that can be locally hosted and deterministically reset, while the chosen theme pair changes the observable layout, DOM structure, styling, and front-end navigation without changing the underlying seeded content.

We use the **News Portal** (WordPress.org, 2017) and **Twenty Twenty-Four** (WordPress.org, 2023) themes under the same WordPress-based content-management deployment. The contrast between these themes provides a substantial front-end shift: page layout, navigation presentation, DOM organization, and styling assumptions differ markedly. No cross-version database migration is required in this case. The same WordPress instance and seeded database can be evaluated under both themes, which gives exact task-level data alignment. Posts, pages, menus, comments, and grading-relevant content remain fixed; only the presentation layer changes. WordPress therefore tests whether learned skills remain executable when front-end structure evolves while the back-end data state is held constant.

Figure 8 shows the WordPress theme pair. In contrast to the GitLab and Magento examples, the figure illustrates theme-level presentation drift rather than a software-version migration.



(a) WordPress News Portal source variant.



(b) WordPress Twenty Twenty-Four target variant.

Figure 8: WordPress source and target variants.

B.2 Shift Perturbation Implementation

Shifted variants are generated through controlled interventions that preserve the underlying task semantics. Contextual Shift is realized by the version or theme pairings in Section B.1, while Perceptual Shift and Execution Shift are implemented as runtime modules that alter grounding evidence and action feasibility, respectively.

B.2.1 Perceptual Shift Implementation

Perceptual Shift is implemented as a deterministic observable-surface transformation applied after page load. The goal is not to change the task, the underlying data, or the available functionality, but to alter the interface evidence through which an agent grounds its next operation. This design targets agents whose learned skills rely on stable visual hierarchy, DOM structure, attributes, or locator cues.

Concretely, the perturbation module applies two classes of transformations. The first class modifies non-semantic visual presentation, such as typography, spacing, contrast, button styling, and small geometric distortions. These changes alter the rendered appearance of the page while preserving the functional role and textual content of task-relevant elements. The second class modifies DOM-surface signals, including locator attributes, class names, and lightweight tag-level structure. For example, stable locator cues may be removed or perturbed, synthetic classes may be added, and simple inline tags may be replaced by semantically similar alternatives. These transformations are designed to change the agent-facing grounding evidence rather than the website state itself.

To ensure reproducibility, all stochastic choices are generated from a fixed seed. Thus, the same task instance produces the same shifted interface across repeated runs. The perturbation is also re-applied to dynamically inserted elements, so the perceptual pressure persists throughout the episode rather than only at initial page load. This allows Perceptual Shift to serve as a controlled stress test for grounding robustness while preserving task intent, task-critical entities, functional roles, navigable paths, and grader semantics.

B.2.2 Execution Shift Implementation

Execution Shift is implemented as a controlled runtime interruption mechanism. Unlike Perceptual Shift, which changes the observable surface used for grounding, Execution Shift changes whether an otherwise valid action can be immediately realized in the browser.

The main execution-shift mechanism injects a modal blocker during the task episode. After the relevant page is loaded, a full-screen overlay is inserted into the interface and intercepts user interactions with the underlying page. As a result, task-relevant controls may still be present and visually accessible, but actions directed at them cannot take effect until the interruption is resolved. The agent must therefore detect the blocker, dismiss it, and then resume the original task flow.

The interruption controller is activated only within the execution window of a benchmark task. Start and end signals delimit when perturbations are allowed to affect the environment, preventing interference with task initialization, grading, or unrelated browser traffic. Once the modal blocker is dismissed, the controller records the recovery event and stops re-injecting the same blocker for that episode. This makes the perturbation reproducible while still requiring the agent to perform recovery before continuing the task.

B.2.3 Injection Regimes for Execution Shift

The interruption controller exposes the *injection schedule* as a configurable dimension of the benchmark rather than as a fixed property of Execution Shift. SkillShift-Bench implements the three regimes summarized in Table 4. Under `once`, a server-side episode flag and a client-side browser-storage flag jointly guarantee that a dismissed blocker is not reinstated. Under `multiple`, both persistent guards are removed, so the injection predicate evaluates to true on every document load and the blocker reappears after each navigation. Under `random`, injection is decided by a Bernoulli draw at each document load and the overlay is revealed after a randomized in-page delay, so the agent cannot anticipate when the interruption will occur.

Regime	Behavior	Mechanism
`once`	Injected once per episode; not reinstated after dismissal	Server-side episode flag plus client-side storage guard
`multiple`	Reinjected on every page navigation	Persistent guards removed; injection predicate always true
`random`	Injected with fixed probability at a randomized in-page delay	Per-load Bernoulli draw plus client-side delay timer

Table 4: Injection regimes available for Execution Shift.

All Execution Shift conditions reported in the main paper use `once`. This is the most conservative of the three schedules: a single successful recovery restores an unobstructed page for the remainder of the episode, so the measured degradation is a lower bound on the pressure that runtime interruption can exert. Appendix E.6 reports how both the fixed-library baseline and MPCR behave under the two more aggressive schedules.

B.3 Task Validation Protocol

Every task family in SkillShift-Bench is manually verified before it enters the shift-generation stage. The validation covers four properties. *Validity* checks that the instruction describes a well-formed operation on the seeded website state. *Correctness* checks that the automatic scorer accepts exactly the intended final state and rejects near-miss states, such as a partially filled form or an unsaved edit. *Functional availability* checks that every capability required by the instruction is present in both the source and the target variant, so that a failure cannot be attributed to a removed feature. *Solvability* checks that at least one executable solution path exists under each of the six evaluation conditions, including the perceptual and execution perturbations; task families for which a perturbation removes the last remaining solution path are revised or discarded.

We emphasize that this protocol establishes task solvability, not a human performance reference. SkillShift-Bench does not report a human baseline, and the numbers in Section 5 should not be read against an implicit human ceiling. The protocol instead supports the benchmark's attribution claim: because a solution exists in every condition, a drop in success between aligned conditions reflects the agent's inability to invoke, ground, or execute a learned skill rather than the absence of a feasible path.

B.4 Shift-Induced Failure Modes

This section examines how a previously working skill breaks under each shift. To isolate the effect of a single shift, we analyze *regression* sets: tasks that the fixed-library ASI agent with GPT-5 mini solves in the reference condition but fails under the shifted condition, with the reference condition being `Src` for Perceptual and Execution Shift and the `Src`–`Tgt` pair for Contextual Shift. Both the quantitative sets and the illustrative cases below are drawn from Magento. All cited trajectories, step counts, and runtime errors are taken from the recorded execution logs.

Regression set	n	Steps	Trunc.
Contextual	13	$5.4 \rightarrow 6.2$	6/13
Perceptual	10	$5.9 \rightarrow 9.7$	9/10
Execution	29	$5.4 \rightarrow 9.5$	26/29

Table 5: Regression sets used for failure-mode analysis, on Magento with GPT-5 mini. "Steps" reports mean trajectory length in the reference condition and in the shifted condition; "Trunc." counts shifted runs that exhaust the step budget.

Table 5 already separates two behavioral regimes. Under Perceptual and Execution Shift, failing trajectories roughly double in length and almost always exhaust the step budget, indicating unrecoverable

looping. Under Contextual Shift, trajectories grow only slightly and truncate in fewer than half of the cases, indicating premature termination on a wrong route or a stale answer.

Contextual Shift breaks skill invocation. When routes, control placement, and workflow layout evolve, the learned skill is invoked against a target that has moved. In the task "total payment of the last five completed orders," the source trajectory navigates the order grid and returns the correct total in five steps. Under Contextual Shift the agent instead fires the reporting step immediately, in a single action, and returns a stale figure without ever re-grounding on the relocated order page. The skill remains task-relevant, but it is invoked against a page state that the learned procedure no longer locates, and the run ends early on a wrong answer rather than looping. This is the signature that Table 5 records as short trajectories with comparatively low truncation.

Perceptual Shift breaks skill grounding. The perturbation preserves task-critical objects and functionality while altering DOM structure, attributes, and locator signals, which appear in the logs as injected synthetic class markers. The learned action remains plausible but can no longer be grounded to the intended element. In the task "order identifier of the newest pending order," the clean trajectory invokes two of ASI's named learned skills, for order navigation and status filtering. Under Perceptual Shift the status filter that was previously rendered as a selection element is rendered as a table header, so the selection action fails repeatedly with a type error and the run loops until the step budget is exhausted. What the shift violates is the grounding assumption encoded in the skill itself, namely that a particular control type sits at a particular node.

Execution Shift breaks execution feasibility. A reproducible modal blocker intercepts interactions, so semantically correct and correctly targeted skill steps become unexecutable in the current page state. In the task "reviews mentioning *decent*," which the clean condition solves in three to four steps, the review-link click times out repeatedly and an explicit navigation call times out as well, because the overlay keeps intercepting pointer events. The agent registers that something is wrong, stalling to ask whether it may proceed, but never resolves the interruption; once the overlay reshuffles the document, the previously valid identifier for the search box can no longer be found, producing a ten-step loop with no answer. Awareness of the obstruction is therefore not sufficient: the failure lies in restoring feasibility, not in identifying what blocks it.

Implications. The three shifts do not lower success uniformly. They induce *stage-distinct failure signatures* that match the diagnostic intent of each mode: wrong-route or early-termination behavior for invocation, un-actionable-locator loops for grounding, and interception-induced stalls for execution feasibility. This is the structure that a feasibility-aware, current-page-conditioned reranker is designed to exploit at transfer time, because in all three cases the shifted skill remains task-relevant while its default action is no longer the feasible one in the current page state.

This analysis also qualifies the Skill Failure Rate diagnostic defined in Section 3.4. SFR aggregates two distinct causes: a skill that is retrieved and executed but returns an incorrect result, and a skill whose steps are correct but cannot be realized in the current page state. The qualitative decomposition above separates them, and SFR should be read as a supplementary diagnostic rather than as a primary metric. SFR is additionally uninformative when skills are almost never invoked, since a near-zero denominator makes the ratio unstable; we therefore report skill-avoidance behavior through ASC in such regimes and refrain from interpreting SFR values computed over a handful of invocations.

C Dynamic-Regret Analysis of SWOP

This appendix is organized as a theorem-first proof appendix, while the proof itself is ordered so that each proof step consumes previously stated mathematical interfaces. Subsection C.1 defines the SWOP model, objective, sliding-window estimator, and algorithm. Subsection C.2 states the fixed-window dynamic-regret theorem and the oracle-tuned corollary used by the informal statement in the main text. Subsections C.3 and C.4 collect the KL-Bellman, boundedness, concentration, filtration, and auxiliary martingale facts used by the proof. Subsection C.5 proves the fixed-window theorem from near-optimism,

certificate decomposition, summed-bonus control, martingale control, and drift counting. Subsection C.6 proves the oracle-window balancing statement.

C.1 Model, Objective, and Algorithm

C.1.1 Nonstationary Information-State MDP Sequence

We consider a finite-horizon information state MDP sequence that varies with episode. There are in total K episodes, and the horizon length of each episode is fixed as H.

The episode-k environment is

$$\mathcal{M}^{(k)} = \Big(\Xi, \{\Omega(\xi)\}_{\xi\in\Xi}, \{r_h^{(k)}\}_{h=1}^H, \{P_h^{(k)}\}_{h=1}^H, H\Big), \qquad k = 1, \dots, K, \tag{8}$$

where

- Ξ is a finite information state space, $|\Xi| = S_\xi$;
- for each information state $\xi \in \Xi$, $\Omega(\xi)$ is the fixed feasible action set, and it does not vary with episode;
- define
$$A := \max_{\xi\in\Xi} |\Omega(\xi)|;$$
- for each stage $h \in [H] := \{1, \dots, H\}$, episode k, state ξ and action $\omega \in \Omega(\xi)$, we have
$$P_h^{(k)}(\cdot \mid \xi, \omega) \in \Delta(\Xi), \qquad r_h^{(k)}(\xi, \omega) \in [0, 1].$$

For the theoretical analysis, we use a fixed episode-invariant candidate universe $\Omega(\xi)$. This convention keeps the KL-regularized Bellman operators and comparators defined on a common support across episodes. Changes in local executability are represented through the non-stationary rewards and transitions rather than through a changing action support.

This fixed-domain abstraction is representation-agnostic. Executable programs, API functions, or tools can be represented as actions in $\Omega(\xi)$ when they are exposed to the executor as callable candidates. In contrast, prompt-level workflow memories, such as AWM workflows (Wang et al., 2025b), are represented through the behavioral prior or contextual state information rather than as additional executable actions.

C.1.2 KL-Regularized Objective

Following regularized MDP and control-as-inference formulations (Geist et al., 2019; Levine, 2018), we take a reference prior on the fixed action domain

$$\pi_{\mathrm{ref}}(\cdot \mid \xi) \in \Delta(\Omega(\xi)), \qquad \forall \xi \in \Xi,$$

and assume that there exists $\pi_{\min} > 0$ such that

$$\pi_{\mathrm{ref}}(\omega \mid \xi) \geq \pi_{\min}, \qquad \forall \xi \in \Xi,\ \forall \omega \in \Omega(\xi).$$

Fix the regularization parameter $\alpha > 0$. For any local randomized policy $\pi_h(\cdot \mid \xi) \in \Delta(\Omega(\xi))$, define its one-step KL-regularized return at stage h, episode k as

$$\tilde{r}_h^{(k)}(\xi, \pi_h) := \langle \pi_h(\cdot \mid \xi), r_h^{(k)}(\xi, \cdot)\rangle - \alpha\, \mathrm{KL}(\pi_h(\cdot \mid \xi) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid \xi)). \tag{9}$$

Equivalently, if the concrete action $\omega_h \sim \pi_h(\cdot \mid \xi_h)$ is taken, then the within-episode KL-regularized return is

$$\sum_{h=1}^H \left(r_h^{(k)}(\xi_h, \omega_h) - \alpha \log \frac{\pi_h(\omega_h \mid \xi_h)}{\pi_{\mathrm{ref}}(\omega_h \mid \xi_h)} \right).$$

Under this fixed-support analytical convention, masking is absorbed into the construction of the candidate universe and the non-stationary executability model. The KL penalty is therefore evaluated on the common support $\Omega(\xi)$.

Interpretation. π_{ref} can be understood as a behavioral prior provided by a large language model (LLM), human knowledge, or prompt-level learned workflows. This includes workflow-memory methods such as AWM, where learned procedures guide action selection through contextual prompting rather than by adding new executable actions. The agent optimizes a trade-off between environment reward and the cost of deviating from the prior.

C.1.3 Policy Class and Episode Value

Let Π be the class of all stage-dependent Markov randomized policies satisfying the support constraint, namely

$$\pi = \{\pi_h(\cdot \mid \xi)\}_{h=1}^{H}, \qquad \mathrm{supp}(\pi_h(\cdot \mid \xi)) \subseteq \Omega(\xi),\ \forall h, \xi.$$

For any policy $\pi \in \Pi$, define its KL-regularized value in episode k by

$$V_1^{\pi,(k)}(\xi) := \mathbb{E}^{\pi}\left[\sum_{h=1}^{H}\left(r_h^{(k)}(\xi_h, \omega_h) - \alpha \log \frac{\pi_h(\omega_h \mid \xi_h)}{\pi_{\mathrm{ref}}(\omega_h \mid \xi_h)}\right) \,\middle|\, \xi_1 = \xi\right]. \tag{10}$$

The stage-wise policy value functions used below are defined by the following backward recursion:

$$V_{H+1}^{\pi,(k)}(\xi) := 0, \qquad \forall \xi \in \Xi,$$

$$Q_h^{\pi,(k)}(\xi, \omega) := r_h^{(k)}(\xi, \omega) + P_h^{(k)}(\cdot \mid \xi, \omega)^{\top} V_{h+1}^{\pi,(k)},$$

and

$$V_h^{\pi,(k)}(\xi) = \sum_{\omega \in \Omega(\xi)} \pi_h(\omega \mid \xi)\left[Q_h^{\pi,(k)}(\xi, \omega) - \alpha \log \frac{\pi_h(\omega \mid \xi)}{\pi_{\mathrm{ref}}(\omega \mid \xi)}\right], \qquad h = H, \ldots, 1.$$

The definition in (10) is the $h = 1$ special case of this recursion.

Let the initial distribution be $\mu_1 \in \Delta(\Xi)$. In the main regret theorem below we use the pathwise initial states ξ_1^k directly, rather than introducing separate μ_1-averaged shorthand values.

C.1.4 Variation Budgets

Following variation-budget analyses of non-stationary bandits and episodic MDPs (Besbes et al., 2014; Cheung et al., 2020; Mao et al., 2021), we allow the environment to change only between episodes. Define the reward drift budget and the transition drift budget as

$$B_r := \sum_{k=1}^{K-1} \max_{h \in [H],\, \xi \in \Xi,\, \omega \in \Omega(\xi)} \left| r_h^{(k+1)}(\xi, \omega) - r_h^{(k)}(\xi, \omega) \right|, \tag{11}$$

$$B_p := \sum_{k=1}^{K-1} \max_{h \in [H],\, \xi \in \Xi,\, \omega \in \Omega(\xi)} \left\| P_h^{(k+1)}(\cdot \mid \xi, \omega) - P_h^{(k)}(\cdot \mid \xi, \omega) \right\|_1. \tag{12}$$

Sometimes the two can also be combined into a total variation quantity as

$$V_K := B_r + H B_p, \tag{13}$$

because in finite-horizon Bellman recursion, transition perturbations are usually amplified through downstream value functions by an $O(H)$ factor. Subsequent theorem statements can be written either separately as functions of B_r, B_p or uniformly as functions of V_K.

Primitive adjacent drifts. For later Bellman-stability and sliding-window arguments, define

$$\Delta_r^{(k)} := \max_{h\in[H],\,\xi\in\Xi,\,\omega\in\Omega(\xi)} \left| r_h^{(k+1)}(\xi,\omega) - r_h^{(k)}(\xi,\omega) \right|, \qquad k = 1,\dots,K-1,$$

and

$$\Delta_p^{(k)} := \max_{h\in[H],\,\xi\in\Xi,\,\omega\in\Omega(\xi)} \left\| P_h^{(k+1)}(\cdot \mid \xi,\omega) - P_h^{(k)}(\cdot \mid \xi,\omega) \right\|_1, \qquad k = 1,\dots,K-1.$$

Then

$$B_r = \sum_{k=1}^{K-1} \Delta_r^{(k)}, \qquad B_p = \sum_{k=1}^{K-1} \Delta_p^{(k)}.$$

Logarithmic notation. Throughout the appendix, $\widetilde{O}(\cdot)$ hides only absolute constants and polylogarithmic factors in

$$S_\xi,\ A,\ H,\ K,\ \delta^{-1},$$

and does not hide polynomial dependence on S_ξ, A, H, K or on the variation budgets B_r, B_p, V_K.

C.1.5 Dynamic Comparator and Dynamic Regret

As in dynamic-regret analyses for non-stationary decision processes (Besbes et al., 2014; Cheung et al., 2020; Mao et al., 2021), in the episodic setting, the dynamic benchmark compares each episode with that episode's own optimal KL-regularized policy (Geist et al., 2019). The KL-regularized episodic dynamic regret used in the main theorem is

$$\mathrm{DynReg}(K) := \sum_{k=1}^{K} \left(V_1^{\star,(k)}(\xi_1^k) - V_1^{\hat{\pi}^k,(k)}(\xi_1^k) \right). \tag{14}$$

This is the standard value-based episodic dynamic regret, evaluated along the realized initial states and the algorithmic history. No separate cumulative-value shorthand or expected-regret object is used in the proof of Theorem C.1.

The theorem is stated in the standard variation-dependent form used for non-stationary decision processes. This form is more informative than an unconditional rate, because it separates statistical estimation error from the intrinsic cost of environmental variation. Sublinear regret follows in the low-variation regimes stated in Corollary C.2. In the common fixed-complexity regime where H, S_ξ, A are treated as problem constants, the oracle-tuned window gives the concise condition $V_K = o(K)$. The corresponding growing-dimension condition is stated explicitly in Corollary C.2.

C.1.6 Sliding-Window Empirical Model and Bonus

Following sliding-window optimism for non-stationary reinforcement learning (Cheung et al., 2020; Mao et al., 2021), to perform provable learning in a nonstationary environment, we use a sliding window of length W and fix throughout

$$1 \le W \le K.$$

For episode k, define the window

$$\mathcal{W}_k := \{\max\{1, k-W\}, \dots, k-1\}. \tag{15}$$

At the beginning of episode k the algorithm uses only the data from the most recent episodes in the window $\mathcal{W}_k$ to construct the window empirical model. The basic idea is:

1. Use only the most recent window data in order to track environment changes;
2. Estimate rewards and transitions within the window;
3. Construct an upper-confidence bonus;

4. Perform optimistic KL-regularized backward planning on the "empirical model + bonus";

5. Output the corresponding softmax policy and execute it.

In this way, the environment inside the window can be regarded as "approximately stationary", while old data outside the window are actively discarded to reduce the bias caused by drift.

For each (h, ξ, ω), define the number of visits inside the window by

$$N_h^k(\xi, \omega) := \sum_{k' \in \mathcal{W}_k} \mathbf{1}\{(\xi_h^{k'}, \omega_h^{k'}) = (\xi, \omega)\}. \tag{16}$$

The empirical reward mean uses the unique zero-count default value:

$$\hat{r}_h^k(\xi, \omega) := \begin{cases} \dfrac{1}{N_h^k(\xi, \omega)} \displaystyle\sum_{k' \in \mathcal{W}_k} r_h^{k'} \mathbf{1}\{(\xi_h^{k'}, \omega_h^{k'}) = (\xi, \omega)\}, & N_h^k(\xi, \omega) \geq 1, \\ 0, & N_h^k(\xi, \omega) = 0. \end{cases}$$

The empirical transition distribution uses a pre-fixed default distribution

$$P_h^\circ(\cdot \mid \xi, \omega) \in \Delta(\Xi)$$

as the unique cold-start convention, and is defined by

$$\hat{P}_h^k(\xi' \mid \xi, \omega) := \begin{cases} \dfrac{1}{N_h^k(\xi, \omega)} \displaystyle\sum_{k' \in \mathcal{W}_k} \mathbf{1}\{(\xi_h^{k'}, \omega_h^{k'}, \xi_{h+1}^{k'}) = (\xi, \omega, \xi')\}, & N_h^k(\xi, \omega) \geq 1, \\ P_h^\circ(\xi' \mid \xi, \omega), & N_h^k(\xi, \omega) = 0. \end{cases}$$

Here $r_h^{k'}$ denotes the immediate reward observed at stage h of episode k'.

The only UCB bonus used throughout is defined as

$$b_h^k(\xi, \omega) = c_b H \sqrt{\frac{S_\xi \log(C_{\log} S_\xi AHK/\delta)}{(N_h^k(\xi, \omega) + 1) \wedge W}}, \tag{17}$$

where $C_{\log} \geq e^2$ is an absolute constant and $c_b > 0$ is chosen according to the parameter convention in Subsection C.5.5. Throughout, assume $\delta \in (0, 1)$, $1 \leq W \leq K$, and $S_\xi, A, H, K \geq 1$. We choose $C_{\log}$ sufficiently large for the reward and transition concentration union bounds, and choose c_b so that

$$c_b \sqrt{S_\xi \log(C_{\log} S_\xi AHK/\delta)} \geq 2.$$

This displayed condition is used to uniformly absorb the maximal bonus in cold-start cases; the explicit concentration-absorption lower bound on c_b is fixed in Subsection C.5.5. Drift does not enter the bonus; it is handled only through D_k from Subsection C.5.2 and the final regret decomposition.

C.1.7 Optimistic Soft Planning

Define the optimistic Q-backup by

$$\widetilde{Q}_h^k(\xi, \omega) := \min\left\{H - h + 1,\ \hat{r}_h^k(\xi, \omega) + b_h^k(\xi, \omega) + \hat{P}_h^k(\cdot \mid \xi, \omega)^\top \widetilde{V}_{h+1}^k\right\}, \tag{18}$$

with terminal condition

$$\widetilde{V}_{H+1}^k(\xi) = 0.$$

Then define the optimistic value by

$$\widetilde{V}_h^k(\xi) := \alpha \log \sum_{\omega \in \Omega(\xi)} \pi_{\mathrm{ref}}(\omega \mid \xi) \exp\left(\frac{\widetilde{Q}_h^k(\xi, \omega)}{\alpha}\right). \tag{19}$$

Algorithm 1 SWOP: Sliding Window Optimistic Prior-Guided Planning

Require: Window size $W \in \{1, \dots, K\}$, regularization parameter α, confidence level δ, reference prior π_{ref}, and default transition kernels $P_h^\circ(\cdot \mid \xi, \omega)$.

for $k = 1, 2, \dots, K$ **do**
  Set $\mathcal{W}_k = \{\max\{1, k - W\}, \dots, k - 1\}$.
  for each (h, ξ, ω) **do**
    Compute $N_h^k(\xi, \omega)$, $\hat{r}_h^k(\xi, \omega)$, $\hat{P}_h^k(\cdot \mid \xi, \omega)$, and $b_h^k(\xi, \omega)$ as in Subsubsection C.1.6.
  end for
  Set $\widetilde{V}_{H+1}^k(\xi) = 0$ for all $\xi \in \Xi$.
  for $h = H, H - 1, \dots, 1$ **do**
    for each $\xi \in \Xi$ and $\omega \in \Omega(\xi)$ **do**
      Compute

$$\widetilde{Q}_h^k(\xi, \omega) = \min\left\{H - h + 1, \hat{r}_h^k(\xi, \omega) + b_h^k(\xi, \omega) + \hat{P}_h^k(\cdot \mid \xi, \omega)^\top \widetilde{V}_{h+1}^k\right\}.$$

    end for
    for each $\xi \in \Xi$ **do**
      Compute $\widetilde{V}_h^k(\xi)$ by the optimistic soft Bellman backup and set $\hat{\pi}_h^k(\cdot \mid \xi)$ to the induced softmax policy.
    end for
  end for
  Execute $\hat{\pi}^k = \{\hat{\pi}_h^k\}_{h=1}^H$ in episode k and store the trajectory.
end for

The corresponding output policy is defined by

$$\hat{\pi}_h^k(\omega \mid \xi) = \frac{\pi_{\mathrm{ref}}(\omega \mid \xi) \exp(\widetilde{Q}_h^k(\xi, \omega)/\alpha)}{\sum_{\omega' \in \Omega(\xi)} \pi_{\mathrm{ref}}(\omega' \mid \xi) \exp(\widetilde{Q}_h^k(\xi, \omega')/\alpha)}. \tag{20}$$

This is the natural form of sliding-window UCB-style optimism under KL regularization (Cheung et al., 2020; Geist et al., 2019). Algorithm 1 summarizes the resulting SWOP procedure across episodes, combining sliding-window estimation, optimistic soft planning, policy execution, and trajectory update.

C.2 Main Theoretical Guarantees

This subsection states the two main formal outputs of Appendix C. The first result is the fixed-window high-probability dynamic-regret bound. The second result is the oracle-tuned fixed-window upper envelope corresponding to the informal statement in the main text. The proofs are given in Subsections C.5 and C.6.

C.2.1 Dynamic-Regret Bound for a Fixed Window

The following result builds on the non-stationary MDP sliding-window optimism perspective and the KL-regularized MDP formulation cited above. The theorem itself is the paper's SWOP-specific analysis under the fixed-domain abstraction and proof setup developed in this appendix.

Theorem C.1 (SWOP Dynamic Regret under Sliding-Window KL-Regularized Optimistic Planning)**.** *Fix $\alpha > 0$, confidence level $\delta \in (0, 1)$, and a window length $W \in \{1, \dots, K\}$ chosen before the algorithm is run. Suppose the following standing assumptions hold.*

1. *The state space Ξ is finite with $|\Xi| = S_\xi$, and the action set $\Omega(\xi)$ is fixed over episodes for each $\xi \in \Xi$. Let $A := \max_{\xi \in \Xi} |\Omega(\xi)|$. Throughout, $K, H, S_\xi, A \geq 1$, $\mu_1 \in \Delta(\Xi)$, and the initial state of each episode satisfies $\xi_1^k \sim \mu_1$ independently of the completed past history.*

2. *For every $k \in [K]$, $h \in [H]$, $\xi \in \Xi$, and $\omega \in \Omega(\xi)$,*

$$r_h^{(k)}(\xi, \omega) \in [0, 1], \qquad P_h^{(k)}(\cdot \mid \xi, \omega) \in \Delta(\Xi).$$

The adjacent drifts $\Delta_r^{(k)}$, $\Delta_p^{(k)}$, the budgets B_r, B_p, and $V_K = B_r + HB_p$ are defined in Subsection C.1.4.

3. *The reference prior has full support on the fixed action domain:*

$$\pi_{\mathrm{ref}}(\omega \mid \xi) \geq \pi_{\min} > 0, \qquad \forall \xi \in \Xi,\ \omega \in \Omega(\xi).$$

All KL-regularized values, policies, and Bellman operators use the same α and π_{ref}.

4. *The sampling and filtration conventions in Subsection C.4.1 hold, including the environment predictability convention. In particular, at the beginning of episode k, the episode-k model objects are fixed or predictable, all algorithmic planning objects are measurable with respect to the completed history of episodes $1, \ldots, k-1$, and $\xi_1^k \sim \mu_1$ is independent of that past.*

5. *Algorithm 1 uses fixed default transition kernels*

$$P_h^\circ(\cdot \mid \xi, \omega) \in \Delta(\Xi), \qquad h \in [H],\ \xi \in \Xi,\ \omega \in \Omega(\xi),$$

and the unified bonus

$$b_h^k(\xi, \omega) = c_b H \sqrt{\frac{S_\xi \log(C_{\log} S_\xi AHK/\delta)}{(N_h^k(\xi, \omega) + 1) \wedge W}},$$

where $C_{\log} \geq e^2$ is chosen sufficiently large for Lemmas C.13–C.15, and c_b satisfies the concentration-absorption and cold-start conditions in Subsection C.5.5.

The regret is the value-based dynamic regret defined in (14). Then there exist absolute constants $C_1, C_2, C_3 > 0$ such that, with probability at least $1 - \delta$, the pathwise random variable $\mathrm{DynReg}(K)$ satisfies

$$\mathrm{DynReg}(K) \leq C_1 H^2 K S_\xi \sqrt{\frac{A \log(C_{\log} S_\xi AHK/\delta)}{W}} + C_2 WH(B_r + HB_p) + C_3 H^2 \sqrt{KH \log(C_{\log}/\delta)}. \tag{21}$$

Equivalently, up to logarithmic factors,

$$\mathrm{DynReg}(K) = \widetilde{O}\left(H^2 K S_\xi \sqrt{\frac{A}{W}} + WH(B_r + HB_p) + H^2\sqrt{KH} \right).$$

C.2.2 Oracle-Tuned Window Corollary

Corollary C.2 (Oracle-Tuned Fixed-Window Balancing)**.** *Recall that*

$$V_K := B_r + HB_p.$$

Assume that the window length is selected before deployment as a deterministic function of the variation budget V_K. More precisely, for any admissible $W_{\mathrm{orc}} \in \{1, \ldots, K\}$ fixed before the algorithm is run, Theorem C.1 gives, with probability at least $1 - \delta$,

$$\mathrm{DynReg}(K) \leq \widetilde{O}\left(H^2 K S_\xi \sqrt{\frac{A}{W_{\mathrm{orc}}}} + W_{\mathrm{orc}} H V_K + H^2\sqrt{KH} \right).$$

Equivalently, an oracle with knowledge of V_K may choose the deterministic upper-envelope minimizer

$$W_{\mathrm{orc}} \in \arg\min_{1 \leq W \leq K} \left\{ H^2 K S_\xi \sqrt{\frac{A}{W}} + WHV_K \right\}$$

before the run, yielding

$$\mathrm{DynReg}(K) \le \widetilde{O}\left(\min_{1\le W\le K} \left\{ H^2 K S_\xi \sqrt{\frac{A}{W}} + WHV_K \right\} + H^2\sqrt{KH} \right).$$

For a uniform display over all variation regimes, use the smoothed quantity $V_K + 1$ and set

$$W_\star := \left(\frac{H^2 S_\xi^2 A K^2}{(V_K+1)^2} \right)^{1/3}, \tag{22}$$

with the admissible deterministic window

$$W_{\mathrm{orc}} := \max\{1,\ \min\{K,\ \lceil W_\star \rceil\}\}.$$

In the untruncated case $1 \le W_\star \le K$, running the algorithm with this fixed window gives

$$\mathrm{DynReg}(K) \le \widetilde{O}\left(H^{5/3} S_\xi^{2/3} A^{1/3} K^{2/3} (V_K+1)^{1/3} + H^2\sqrt{KH} \right), \tag{23}$$

with probability at least $1-\delta$.

In the common fixed-complexity regime where H, S_ξ, A are treated as problem constants, the tuned-window bound gives

$$\mathrm{DynReg}(K) = \widetilde{O}\left(K^{2/3}(V_K+1)^{1/3} \right) + o(K),$$

and hence sublinear dynamic regret whenever $V_K = o(K)$. If these problem parameters are allowed to grow with K, the corresponding sufficient sublinearity conditions are

$$H^5 S_\xi^2 A(V_K+1) = o(K), \qquad H^2\sqrt{KH} = o(K).$$

C.2.3 Proof Roadmap and Dependency Graph

The proof of Theorem C.1 is organized around five mathematical interfaces.

1. **Near-optimism under window drift.** Proposition C.20 shows that the clipped optimistic value upper-bounds the episode-wise optimal value up to the local window-drift term:

$$V_h^{\star,(k)}(\xi) - \widetilde{V}_h^k(\xi) \le C_{\mathrm{opt}}(H-h+1)D_k.$$

2. **Certificate decomposition.** Lemma C.22 controls the episode-level gap between the optimistic value and the value of the executed policy by the realized bonuses, a martingale remainder, and the same drift quantity. The fixed-window theorem consumes the following pathwise interface:

$$\widetilde{V}_1^k(\xi_1^k) - V_1^{\hat{\pi}^k,(k)}(\xi_1^k) \le 2\sum_{h=1}^{H} b_h^k(\xi_h^k, \omega_h^k) + M_k + C_{\mathrm{cert}} H D_k.$$

3. **Summed-bonus control.** Proposition C.23 controls the cumulative realized uncertainty bonus under overlapping sliding-window counts.

4. **Martingale control.** Proposition C.24 controls the unified pathwise remainder $\sum_k M_k$ with high probability.

5. **Window drift counting.** Equation (28) converts episode-wise window drift into the variation-budget contribution $W(B_r + HB_p)$.

Combining these five interfaces yields the three terms in the fixed-window theorem: statistical estimation, non-stationary drift, and martingale noise.

C.3 Supporting KL-Regularized Bellman Facts

This subsection collects the KL-regularized Bellman algebra, log-sum-exp monotonicity, and boundedness facts used by the main proof. It also includes additional Bellman stability facts for context; those stability facts are not invoked in the proof of Theorem C.1, whose drift control uses Lemma C.18 and (28).

C.3.1 Soft Bellman Recursion

This subsection uses the standard KL-regularized Bellman and softmax form for regularized MDPs (Geist et al., 2019; Levine, 2018), specialized to the fixed-domain episode-k model.

For each episode k, define the stage-h optimal regularized value function $V_h^{\star,(k)} : \Xi \to \mathbb{R}$, with terminal condition

$$V_{H+1}^{\star,(k)}(\xi) = 0, \qquad \forall \xi \in \Xi.$$

The corresponding soft Q-function is defined by

$$Q_h^{\star,(k)}(\xi,\omega) := r_h^{(k)}(\xi,\omega) + P_h^{(k)}(\cdot \mid \xi,\omega)^\top V_{h+1}^{\star,(k)}. \tag{24}$$

Then the KL-regularized Bellman recursion is

$$V_h^{\star,(k)}(\xi) = \alpha \log \sum_{\omega \in \Omega(\xi)} \pi_{\mathrm{ref}}(\omega \mid \xi) \exp\left(\frac{Q_h^{\star,(k)}(\xi,\omega)}{\alpha}\right), \qquad h = H, \dots, 1. \tag{25}$$

For any episode k, stage h, and bounded function $f : \Xi \to \mathbb{R}$, define

$$(\mathcal{T}_h^{(k)} f)(\xi) := \alpha \log \sum_{\omega \in \Omega(\xi)} \pi_{\mathrm{ref}}(\omega \mid \xi) \exp\left(\frac{r_h^{(k)}(\xi,\omega) + P_h^{(k)}(\cdot \mid \xi,\omega)^\top f}{\alpha}\right).$$

With this notation,

$$V_{H+1}^{\star,(k)} \equiv 0, \qquad V_h^{\star,(k)} = \mathcal{T}_h^{(k)} V_{h+1}^{\star,(k)}.$$

The corresponding stage-wise greedy regularized policy has the following closed-form softmax form:

$$\pi_h^{\star,(k)}(\omega \mid \xi) = \frac{\pi_{\mathrm{ref}}(\omega \mid \xi) \exp\Big(Q_h^{\star,(k)}(\xi,\omega)/\alpha\Big)}{\sum_{\omega' \in \Omega(\xi)} \pi_{\mathrm{ref}}(\omega' \mid \xi) \exp\Big(Q_h^{\star,(k)}(\xi,\omega')/\alpha\Big)}, \qquad \omega \in \Omega(\xi). \tag{26}$$

This closed-form expression is a key advantage of KL regularization: the optimal policy is naturally absolutely continuous with respect to π_{ref} as a soft policy, thereby avoiding the discontinuity and tie-breaking technical issues caused by hard argmax selection.

C.3.2 KL-Bellman Optimality

Proposition C.3 (KL-Bellman optimality and softmax form)**.** *For any fixed episode k, stage h, and state ξ, let $q = (q_\omega)_{\omega \in \Omega(\xi)} \in \mathbb{R}^{|\Omega(\xi)|}$. Then*

$$\max_{\pi \in \Delta(\Omega(\xi))} \left\{ \sum_\omega \pi(\omega) q_\omega - \alpha \,\mathrm{KL}\big(\pi \| \pi_{\mathrm{ref}}(\cdot \mid \xi)\big) \right\} = \alpha \log \sum_{\omega \in \Omega(\xi)} \pi_{\mathrm{ref}}(\omega \mid \xi) \exp(q_\omega/\alpha),$$

and the unique maximizer is

$$\pi^q(\omega) = \frac{\pi_{\mathrm{ref}}(\omega \mid \xi) \exp(q_\omega/\alpha)}{\sum_{\omega'} \pi_{\mathrm{ref}}(\omega' \mid \xi) \exp(q_{\omega'}/\alpha)}.$$

Therefore, by taking

$$q_\omega = r_h^{(k)}(\xi,\omega) + P_h^{(k)}(\cdot \mid \xi,\omega)^\top V_{h+1}^{\star,(k)}$$

in the above equality and applying backward induction from $h = H$ we obtain the KL-regularized Bellman recursion and the optimal soft policy in this section.

Proof. Let $p_\omega = \pi_{\mathrm{ref}}(\omega \mid \xi)$. By assumption $p_\omega > 0$. Let

$$Z(q) := \sum_\omega p_\omega \exp(q_\omega/\alpha), \qquad \pi^q(\omega) := \frac{p_\omega \exp(q_\omega/\alpha)}{Z(q)}.$$

For any $\pi \in \Delta(\Omega(\xi))$, direct expansion of the KL gives

$$\alpha \log Z(q) - \{\langle \pi, q\rangle - \alpha \mathrm{KL}(\pi \| p)\} = \alpha \mathrm{KL}(\pi \| \pi^q) \geq 0.$$

Therefore the maximum value is $\alpha \log Z(q)$, and since $\mathrm{KL}(\pi \| \pi^q) = 0$ if and only if $\pi = \pi^q$, the maximizer is unique.

The terminal layer $V_{H+1}^{\star,(k)} \equiv 0$ has been fixed. If $V_{h+1}^{\star,(k)}$ has been defined, then the optimal local choice at stage h is exactly the maximization problem above with $q_\omega = Q_h^{\star,(k)}(\xi, \omega)$ substituted in; its maximum value gives $V_h^{\star,(k)}(\xi)$, and its unique maximizing policy gives $\pi_h^{\star,(k)}(\cdot \mid \xi)$. The backward induction is complete. This proves the claim. □

C.3.3 Log-Sum-Exp and Bellman Non-Expansiveness

Lemma C.4 (Log-Sum-Exp Backup Is ℓ_∞-Non-Expansive in Q)**.** *Fix any state ξ and define the map*

$$\Phi_\xi(q) := \alpha \log \sum_{\omega \in \Omega(\xi)} \pi_{\mathrm{ref}}(\omega \mid \xi) \exp(q_\omega/\alpha), \qquad q \in \mathbb{R}^{|\Omega(\xi)|}.$$

Then for any $q, q' \in \mathbb{R}^{|\Omega(\xi)|}$,

$$|\Phi_\xi(q) - \Phi_\xi(q')| \leq \|q - q'\|_\infty.$$

Proof. Let

$$\delta := \|q - q'\|_\infty.$$

Then for each action ω,

$$q_\omega \leq q'_\omega + \delta.$$

Hence

$$\exp(q_\omega/\alpha) \leq e^{\delta/\alpha} \exp(q'_\omega/\alpha).$$

Multiplying both sides by $\pi_{\mathrm{ref}}(\omega \mid \xi)$ and summing gives

$$\sum_\omega \pi_{\mathrm{ref}}(\omega \mid \xi) \exp(q_\omega/\alpha) \leq e^{\delta/\alpha} \sum_\omega \pi_{\mathrm{ref}}(\omega \mid \xi) \exp(q'_\omega/\alpha).$$

Taking $\alpha \log$ gives

$$\Phi_\xi(q) \leq \Phi_\xi(q') + \delta.$$

Swapping q, q' gives similarly

$$\Phi_\xi(q') \leq \Phi_\xi(q) + \delta.$$

Thus

$$|\Phi_\xi(q) - \Phi_\xi(q')| \leq \delta = \|q - q'\|_\infty.$$

This proves the claim.

□

Lemma C.5 (Log-Sum-Exp Monotonicity and Translation)**.** *Fix any state ξ and let*

$$\Phi_\xi(q) := \alpha \log \sum_{\omega \in \Omega(\xi)} \pi_{\mathrm{ref}}(\omega \mid \xi) \exp(q_\omega/\alpha).$$

Then the following two properties hold.

1. *If $q_\omega \geq q'_\omega$ for all $\omega \in \Omega(\xi)$, then*
$$\Phi_\xi(q) \geq \Phi_\xi(q').$$

2. *For any scalar $c \in \mathbb{R}$,*
$$\Phi_\xi(q + c\mathbf{1}) = \Phi_\xi(q) + c.$$
Consequently, if $q_\omega \geq q'_\omega - \varepsilon$ for all $\omega \in \Omega(\xi)$, then
$$\Phi_\xi(q) \geq \Phi_\xi(q') - \varepsilon.$$

Proof. The first claim follows because the exponential function is increasing and all weights $\pi_{\mathrm{ref}}(\omega \mid \xi)$ are nonnegative. For the second claim,

$$\Phi_\xi(q+c\mathbf{1}) = \alpha \log \sum_\omega \pi_{\mathrm{ref}}(\omega \mid \xi) \exp((q_\omega + c)/\alpha) = \alpha \log \left(e^{c/\alpha} \sum_\omega \pi_{\mathrm{ref}}(\omega \mid \xi) e^{q_\omega/\alpha} \right) = \Phi_\xi(q) + c.$$

If $q_\omega \geq q'_\omega - \varepsilon$ for all ω, then by monotonicity and translation,

$$\Phi_\xi(q) \geq \Phi_\xi(q' - \varepsilon\mathbf{1}) = \Phi_\xi(q') - \varepsilon.$$

This proves the claim. □

Lemma C.6 (Soft Bellman Operator Is Non-Expansive in Future Values)**.** *For any fixed k, h, and any two functions $f, g : \Xi \to \mathbb{R}$,*
$$\|\mathcal{T}_h^{(k)} f - \mathcal{T}_h^{(k)} g\|_\infty \leq \|f - g\|_\infty.$$

Proof. Fix $\xi \in \Xi$. Define the action-value vectors

$$q_\omega := r_h^{(k)}(\xi, \omega) + P_h^{(k)}(\cdot \mid \xi, \omega)^\top f, \qquad q'_\omega := r_h^{(k)}(\xi, \omega) + P_h^{(k)}(\cdot \mid \xi, \omega)^\top g.$$

By Lemma C.4,
$$|(\mathcal{T}_h^{(k)} f)(\xi) - (\mathcal{T}_h^{(k)} g)(\xi)| \leq \max_\omega \left| P_h^{(k)}(\cdot \mid \xi, \omega)^\top (f - g) \right|.$$

Because $P_h^{(k)}(\cdot \mid \xi, \omega)$ is a probability distribution,

$$\left| P_h^{(k)}(\cdot \mid \xi, \omega)^\top (f - g) \right| \leq \|f - g\|_\infty.$$

Taking the supremum over all ξ gives

$$\|\mathcal{T}_h^{(k)} f - \mathcal{T}_h^{(k)} g\|_\infty \leq \|f - g\|_\infty.$$

This proves the claim. □

C.3.4 Boundedness of Soft Value Functions

Lemma C.7 (Finite-Horizon Boundedness of Soft Value Functions)**.** *For any episode k, stage h, state ξ,*

$$0 \leq V_h^{\star,(k)}(\xi) \leq H - h + 1.$$

Therefore

$$\|V_h^{\star,(k)}\|_\infty \leq H - h + 1.$$

Proof. The terminal condition $V_{H+1}^{\star,(k)} \equiv 0$ clearly holds. We now use backward induction.

Assume that $h+1$ has already been established

$$0 \le V_{h+1}^{\star,(k)}(\xi) \le H-h, \qquad \forall \xi.$$

Then for any (ξ, ω),

$$0 \le r_h^{(k)}(\xi,\omega) + P_h^{(k)}(\cdot \mid \xi,\omega)^\top V_{h+1}^{\star,(k)} \le 1 + (H-h) = H-h+1.$$

Since $\alpha > 0$ and π_{ref} has full support, Proposition C.3 gives

$$V_h^{\star,(k)}(\xi) = \max_{\pi_h(\cdot|\xi)} \left\{ \langle \pi_h, Q_h^{\star,(k)}(\xi,\cdot)\rangle - \alpha \mathrm{KL}(\pi_h \| \pi_{\mathrm{ref}}(\cdot \mid \xi)) \right\}.$$

Because $\mathrm{KL} \ge 0$, the above expression does not exceed $\max_\omega Q_h^{\star,(k)}(\xi,\omega) \le H-h+1$.

On the other hand, take $\pi_h = \pi_{\mathrm{ref}}(\cdot \mid \xi)$, then the KL term is 0. Thus

$$V_h^{\star,(k)}(\xi) \ge \sum_\omega \pi_{\mathrm{ref}}(\omega \mid \xi) Q_h^{\star,(k)}(\xi,\omega) \ge 0.$$

The induction is complete. This proves the claim.

□

Remark C.1 (Boundedness convention for optimal and executed values). Lemma C.7 applies to the optimal KL-regularized value $V_h^{\star,(k)}$. The same nonnegative $[0, H-h+1]$ envelope should not be used for an arbitrary policy value $V_h^{\pi,(k)}$, because the KL penalty may make the one-step regularized reward negative. The main proof separately uses the result obtained from optimistic clipping,

$$0 \le \widetilde{Q}_h^k(\xi,\omega) \le H-h+1, \qquad 0 \le \widetilde{V}_h^k(\xi) \le H-h+1.$$

In the martingale-control proof, we only use a conservative envelope for the particular executed policy $\hat{\pi}^k$, derived from the clipped optimistic softmax identity:

$$\alpha \log \frac{\hat{\pi}_h^k(\omega \mid \xi)}{\pi_{\mathrm{ref}}(\omega \mid \xi)} = \widetilde{Q}_h^k(\xi,\omega) - \widetilde{V}_h^k(\xi).$$

Since both clipped optimistic quantities lie in $[0, H]$, this implies

$$\left| r_h^{(k)}(\xi,\omega) - \alpha \log \frac{\hat{\pi}_h^k(\omega \mid \xi)}{\pi_{\mathrm{ref}}(\omega \mid \xi)} \right| \le 2H.$$

Consequently,

$$|V_h^{\hat{\pi}^k,(k)}(\xi)| \le 2H(H-h+1) \le 2H^2.$$

C.3.5 Bellman Stability under Model Perturbation

The results in this subsection are included to connect the notation with standard non-stationary MDP variation arguments. They are not invoked in the proof of Theorem C.1; the fixed-window regret proof uses the conditional-mean drift bound in Lemma C.18 together with the window counting bound (28).

Lemma C.8 (Bellman Backup Stability under One-Step Model Perturbation)**.** *For any $h \in [H]$, arbitrary function $f : \Xi \to \mathbb{R}$,*

$$\|\mathcal{T}_h^{(k+1)} f - \mathcal{T}_h^{(k)} f\|_\infty \le \Delta_r^{(k)} + \Delta_p^{(k)} \|f\|_\infty.$$

Proof. Fix $\xi \in \Xi$. Define the action-values

$$q_\omega := r_h^{(k+1)}(\xi,\omega) + P_h^{(k+1)}(\cdot \mid \xi,\omega)^\top f,$$
$$q'_\omega := r_h^{(k)}(\xi,\omega) + P_h^{(k)}(\cdot \mid \xi,\omega)^\top f.$$

By Lemma C.4,

$$|(\mathcal{T}_h^{(k+1)} f)(\xi) - (\mathcal{T}_h^{(k)} f)(\xi)| \le \max_\omega |q_\omega - q'_\omega|.$$

For each ω,

$$|q_\omega - q'_\omega| \le |r_h^{(k+1)}(\xi,\omega) - r_h^{(k)}(\xi,\omega)| + \left|\left(P_h^{(k+1)} - P_h^{(k)}\right)(\cdot \mid \xi,\omega)^\top f\right|.$$

The first term is at most $\Delta_r^{(k)}$. The second term follows from Hölder's inequality

$$\left|\left(P_h^{(k+1)} - P_h^{(k)}\right)^\top f\right| \le \|P_h^{(k+1)}(\cdot \mid \xi,\omega) - P_h^{(k)}(\cdot \mid \xi,\omega)\|_1 \, \|f\|_\infty \le \Delta_p^{(k)} \|f\|_\infty.$$

Thus, for each ξ,

$$|(\mathcal{T}_h^{(k+1)} f)(\xi) - (\mathcal{T}_h^{(k)} f)(\xi)| \le \Delta_r^{(k)} + \Delta_p^{(k)} \|f\|_\infty.$$

Taking the supremum over ξ gives the conclusion. This proves the claim. □

Theorem C.9 (Soft Bellman Stability across Episodes)**.** *For any adjacent episodes k and $k+1$, and any stage $h \in [H]$,*

$$\|V_h^{\star,(k+1)} - V_h^{\star,(k)}\|_\infty \le (H-h+1)\Delta_r^{(k)} + \frac{(H-h)(H-h+1)}{2}\Delta_p^{(k)}.$$

In particular, a coarse form is

$$\|V_h^{\star,(k+1)} - V_h^{\star,(k)}\|_\infty \le (H-h+1)\left(\Delta_r^{(k)} + H\Delta_p^{(k)}\right).$$

Proof. Define

$$\delta_h^{(k)} := \|V_h^{\star,(k+1)} - V_h^{\star,(k)}\|_\infty.$$

From the terminal condition we know

$$\delta_{H+1}^{(k)} = 0.$$

For any $h \le H$,

$$V_h^{\star,(k+1)} = \mathcal{T}_h^{(k+1)} V_{h+1}^{\star,(k+1)}, \qquad V_h^{\star,(k)} = \mathcal{T}_h^{(k)} V_{h+1}^{\star,(k)}.$$

Therefore

$$\delta_h^{(k)} \le \underbrace{\|\mathcal{T}_h^{(k+1)} V_{h+1}^{\star,(k+1)} - \mathcal{T}_h^{(k+1)} V_{h+1}^{\star,(k)}\|_\infty}_{\text{(I)}} + \underbrace{\|\mathcal{T}_h^{(k+1)} V_{h+1}^{\star,(k)} - \mathcal{T}_h^{(k)} V_{h+1}^{\star,(k)}\|_\infty}_{\text{(II)}}.$$

For term (I), By Lemma C.6,

$$\text{(I)} \le \delta_{h+1}^{(k)}.$$

For term (II), By Lemmas C.8 and C.7,

$$\text{(II)} \le \Delta_r^{(k)} + \Delta_p^{(k)} \|V_{h+1}^{\star,(k)}\|_\infty \le \Delta_r^{(k)} + (H-h)\Delta_p^{(k)}.$$

Thus we obtain the recursion

$$\delta_h^{(k)} \le \delta_{h+1}^{(k)} + \Delta_r^{(k)} + (H-h)\Delta_p^{(k)}.$$

Iterating from h to H, and using $\delta_{H+1}^{(k)} = 0$, gives

$$\delta_h^{(k)} \le \sum_{j=h}^{H} \Delta_r^{(k)} + \sum_{j=h}^{H} (H-j)\Delta_p^{(k)}.$$

The first term equals

$$(H-h+1)\Delta_r^{(k)}.$$

The second term equals

$$\sum_{m=0}^{H-h} m\,\Delta_p^{(k)} = \frac{(H-h)(H-h+1)}{2}\Delta_p^{(k)}.$$

Combining gives

$$\delta_h^{(k)} \le (H-h+1)\Delta_r^{(k)} + \frac{(H-h)(H-h+1)}{2}\Delta_p^{(k)}.$$

This proves the claim.

□

Corollary C.10 (Cumulative Value-Function Drift Bound)**.** *Summing at the initial stage $h = 1$ gives*

$$\sum_{k=1}^{K-1} \|V_1^{\star,(k+1)} - V_1^{\star,(k)}\|_\infty \le HB_r + \frac{H(H-1)}{2}B_p.$$

Therefore in big O notation,

$$\sum_{k=1}^{K-1} \|V_1^{\star,(k+1)} - V_1^{\star,(k)}\|_\infty = O\big(H(B_r + HB_p)\big).$$

This corollary is included as explanatory context; the proof of Theorem C.1 uses the conditional-mean drift bound below rather than this cumulative value-drift comparison directly.

Proof. Taking $h = 1$ in Theorem C.9 gives, for each $k = 1, \dots, K-1$,

$$\|V_1^{\star,(k+1)} - V_1^{\star,(k)}\|_\infty \le H\Delta_r^{(k)} + \frac{H(H-1)}{2}\Delta_p^{(k)}.$$

Summing over $k = 1, \dots, K-1$ and using $B_r = \sum_{k=1}^{K-1} \Delta_r^{(k)}$ and $B_p = \sum_{k=1}^{K-1} \Delta_p^{(k)}$ gives

$$\sum_{k=1}^{K-1} \|V_1^{\star,(k+1)} - V_1^{\star,(k)}\|_\infty \le HB_r + \frac{H(H-1)}{2}B_p.$$

The displayed big O form follows directly. This proves the claim.

□

C.4 Concentration, Filtration, and Window-Model Details

This subsection collects the sampling, filtration, concentration, and auxiliary martingale results used by the proof. The concentration statements are formulated for the sliding-window empirical model and its conditional-mean counterpart.

C.4.1 Sampling Model and Filtration

The following interactive sampling model is used throughout. Let

$$\mathcal{F}_{k,h}^{-}$$

denote, at episode k, stage h, when the current state ξ_h^k and current action ω_h^k have already been determined, but the immediate reward r_h^k and next state ξ_{h+1}^k have not yet been revealed, the history σ–algebra. If

$$(\xi_h^k, \omega_h^k) = (\xi, \omega),$$

then the reward observation satisfies

$$\mathbb{E}\Big[r_h^k \mid \mathcal{F}_{k,h}^{-}\Big] = r_h^{(k)}(\xi, \omega), \qquad r_h^k \in [0,1],$$

and the next state satisfies

$$\mathbb{P}\Big(\xi_{h+1}^k = \xi' \mid \mathcal{F}_{k,h}^{-}\Big) = P_h^{(k)}(\xi' \mid \xi, \omega), \qquad \forall \xi' \in \Xi.$$

Here conditional independence between reward noise and transition sampling is not required; the subsequent proofs use only the above conditional mean and conditional law. The window counts, empirical model, bonus, optimistic value, and executed policy constructed by the algorithm at the beginning of episode k depend only on the data from episodes $1, \dots, k-1$.

Reward notation convention. Throughout the appendix, $r_h^{(k)}(\xi, \omega)$ denotes the conditional mean reward function of the episode-k environment, whereas r_h^k denotes the realized reward observation collected at stage h of episode k. Thus the superscript (k) indexes the environment model, while the superscript k without parentheses indexes the realized trajectory.

Global filtration convention. Throughout, a global increasing filtration is assumed to exist, revealing randomness in the natural time order of episodes and stages. At the beginning of each episode k the initial state $\xi_1^k \sim \mu_1$ under the given fixed μ_1 is independent of the past history. At the beginning of episode k, the algorithm constructs $\hat{\pi}^k$ and all planning objects using only the completed history of episodes $1, \dots, k-1$. At stage h, given the current state ξ_h^k, the action is sampled according to

$$\omega_h^k \sim \hat{\pi}_h^k(\cdot \mid \xi_h^k)$$

and is already determined before the reward and next state are revealed; the resulting pre-outcome σ–algebra is exactly the above $\mathcal{F}_{k,h}^{-}$. Conditional on $\mathcal{F}_{k,h}^{-}$, the reward is only required to satisfy the above conditional mean, and the next state is only required to satisfy the above conditional law. The pair $(\mathcal{F}_t, \mathcal{G}_t)$ used in Proposition C.24 is the linearized version of this global filtration under the global time index $t = (k-1)H + h$.

Environment predictability convention. Throughout the appendix, the nonstationary MDP sequence

$$\{\mathcal{M}^{(k)}\}_{k=1}^{K}$$

is either fixed before the interaction begins, or more generally is predictable at the beginning of each episode. Namely, for every episode k, the collection

$$\{r_h^{(k)}(\xi, \omega), P_h^{(k)}(\cdot \mid \xi, \omega) : h \in [H], \xi \in \Xi, \omega \in \Omega(\xi)\}$$

is measurable with respect to the beginning-of-episode k σ-algebra and is not allowed to depend on the current within-episode reward or transition randomness. Conditional on the pre-outcome σ-algebra $\mathcal{F}_{k,h}^{-}$, the only randomness used in the sampling model is the current reward observation and next-state draw satisfying the displayed conditional mean and conditional law.

C.4.2 Martingale Auxiliary Inequality

Lemma C.11 (Azuma–Hoeffding Inequality)**.** *Let $(Y_t, \mathcal{F}_t)_{t=1}^n$ be a martingale difference sequence such that $|Y_t| \le B$ almost surely for every t. Then, for every $u > 0$,*

$$\mathbb{P}\left(\left|\sum_{t=1}^n Y_t\right| \ge u\right) \le 2\exp\left(-\frac{u^2}{2B^2 n}\right).$$

Proof. For each $\lambda \in \mathbb{R}$, Hoeffding's lemma applied conditionally on $\mathcal{F}_{t-1}$ gives

$$\mathbb{E}[\exp(\lambda Y_t) \mid \mathcal{F}_{t-1}] \le \exp\left(\frac{\lambda^2 B^2}{2}\right),$$

because $\mathbb{E}[Y_t \mid \mathcal{F}_{t-1}] = 0$ and $|Y_t| \le B$ almost surely. Iterating this inequality over $t = 1, \dots, n$ yields

$$\mathbb{E}\exp\left(\lambda \sum_{t=1}^n Y_t\right) \le \exp\left(\frac{n\lambda^2 B^2}{2}\right).$$

By Markov's inequality, for any $u > 0$ and $\lambda > 0$,

$$\mathbb{P}\left(\sum_{t=1}^n Y_t \ge u\right) \le \exp\left(-\lambda u + \frac{n\lambda^2 B^2}{2}\right).$$

Choosing $\lambda = u/(nB^2)$ gives

$$\mathbb{P}\left(\sum_{t=1}^n Y_t \ge u\right) \le \exp\left(-\frac{u^2}{2nB^2}\right).$$

Applying the same argument to $-Y_t$ and taking a union bound proves the claim. □

C.4.3 Predictable-Skipping Concentration

This subsection handles the concentration problem caused by random hit locations. The core requirement is: the hit indicator must be measurable before the current reward / next-state outcome is revealed; the proof must not condition on the complete hit-location vector.

Lemma C.12 (Predictable-Skipping Concentration with Stopped Filtration)**.** *Let $M \ge 1$. Suppose there exists a two-level filtration*

$$\mathcal{H}_0 \subseteq \mathcal{G}_1 \subseteq \mathcal{H}_1 \subseteq \mathcal{G}_2 \subseteq \mathcal{H}_2 \subseteq \cdots \subseteq \mathcal{G}_M \subseteq \mathcal{H}_M,$$

where $\mathcal{G}_t$ denotes the pre-outcome information for the t-th candidate sample and $\mathcal{H}_t$ denotes the corresponding post-outcome information. In particular, the displayed nesting implies that

$$\mathcal{H}_s \subseteq \mathcal{G}_t, \qquad 0 \le s < t \le M,$$

with the convention that $\mathcal{H}_0 \subseteq \mathcal{G}_t$ for all $t \ge 1$. This is the temporal measurability relation used below.

$$I_t \in \{0, 1\}, \qquad I_t \in \mathcal{G}_t,$$

and suppose the random variable X_t satisfies

$$X_t \in \mathcal{H}_t, \qquad X_t = I_t X_t, \qquad |X_t| \le B, \qquad \mathbb{E}[X_t \mid \mathcal{G}_t] = 0.$$

Define the n-th selected location

$$\tau_n := \inf\left\{t \in [M] : \sum_{s=1}^t I_s \ge n\right\},$$

If the set is empty, set $\tau_n = \infty$. Let

$$Z_n := \begin{cases} X_{\tau_n}, & \tau_n < \infty, \\ 0, & \tau_n = \infty. \end{cases}$$

Thus the undefined quantity X_∞ is never invoked. Then there exists a stopped filtration $(\mathcal{K}_n)_{n=0}^M$, such that $(Z_n, \mathcal{K}_n)$ is B-bounded martingale difference sequence. Furthermore, for any $\delta' \in (0,1)$, with probability at least $1-\delta'$, simultaneously for all $1 \le n \le M$

$$\left| \sum_{m=1}^{n} Z_m \right| \le 2B\sqrt{n \log(2M/\delta')}.$$

Therefore, if $N := \sum_{t=1}^M I_t \ge 1$, then

$$\left| \frac{1}{N} \sum_{t=1}^{M} I_t X_t \right| = \left| \frac{1}{N} \sum_{m=1}^{N} Z_m \right| \le 2B\sqrt{\frac{\log(2M/\delta')}{N}}.$$

Proof. Let Ω denote the underlying probability sample space in this proof, to avoid conflict with the action-set notation $\Omega(\xi)$.

First, because $I_t \in \mathcal{G}_t$, the event

$$\{\tau_n \le t\} = \left\{ \sum_{s=1}^{t} I_s \ge n \right\}$$

belongs to $\mathcal{G}_t$. Therefore τ_n is a stopping time with respect to the pre-outcome filtration $(\mathcal{G}_t)_{t=1}^M$, and $\{\tau_n = t\} \in \mathcal{G}_t$.

Define the stopped σ-algebra

$$\mathcal{K}_n := \{A : \ A \cap \{\tau_n = t\} \in \mathcal{H}_t \ \forall t \in [M], \quad A \cap \{\tau_n = \infty\} \in \mathcal{H}_M\},$$

and set $\mathcal{K}_0 := \mathcal{H}_0$. Next verify $(\mathcal{K}_n)_{n=0}^M$ is a filtration.

First, $\mathcal{K}_n$ is a σ-algebra. Indeed, $\Omega \in \mathcal{K}_n$ and $\varnothing \in \mathcal{K}_n$ clearly hold, because $\{\tau_n = t\} \in \mathcal{G}_t \subseteq \mathcal{H}_t$ and $\{\tau_n = \infty\} \in \mathcal{G}_M \subseteq \mathcal{H}_M$. If $A \in \mathcal{K}_n$, then for each $t \in [M]$, $A^c \cap \{\tau_n = t\} = \{\tau_n = t\} \setminus (A \cap \{\tau_n = t\}) \in \mathcal{H}_t$, and $A^c \cap \{\tau_n = \infty\} \in \mathcal{H}_M$; if $A_i \in \mathcal{K}_n$, then $(\bigcup_i A_i) \cap \{\tau_n = t\} = \bigcup_i (A_i \cap \{\tau_n = t\}) \in \mathcal{H}_t$, and $(\bigcup_i A_i) \cap \{\tau_n = \infty\} \in \mathcal{H}_M$. Therefore $\mathcal{K}_n$ is a σ-algebra.

Next prove monotonicity. If $A \in \mathcal{K}_{n-1}$, then for any $t \in [M]$:

- When $n = 1$, $A \in \mathcal{H}_0 \subseteq \mathcal{H}_t$, and $\{\tau_1 = t\} \in \mathcal{G}_t \subseteq \mathcal{H}_t$, so $A \cap \{\tau_1 = t\} \in \mathcal{H}_t$; at the same time $A \cap \{\tau_1 = \infty\} \in \mathcal{H}_M$.
- When $n \ge 2$,

  $$A \cap \{\tau_n = t\} = \bigcup_{s=1}^{t-1} \big(A \cap \{\tau_{n-1} = s\} \cap \{\tau_n = t\}\big).$$

  Since $A \in \mathcal{K}_{n-1}$, $A \cap \{\tau_{n-1} = s\} \in \mathcal{H}_s \subseteq \mathcal{H}_t$; at the same time $\{\tau_n = t\} \in \mathcal{G}_t \subseteq \mathcal{H}_t$. Thus each union term belongs to $\mathcal{H}_t$.

On the other hand,

$$A \cap \{\tau_n = \infty\} = \big(A \cap \{\tau_{n-1} = \infty\}\big) \cup \bigcup_{s=1}^{M} \big(A \cap \{\tau_{n-1} = s\} \cap \{\tau_n = \infty\}\big) \in \mathcal{H}_M,$$

because $A \cap \{\tau_{n-1} = \infty\} \in \mathcal{H}_M$, $A \cap \{\tau_{n-1} = s\} \in \mathcal{H}_s \subseteq \mathcal{H}_M$, and $\{\tau_n = \infty\} \in \mathcal{G}_M \subseteq \mathcal{H}_M$.

Therefore $A \in \mathcal{K}_n$, and hence $\mathcal{K}_{n-1} \subseteq \mathcal{K}_n$. Thus $(\mathcal{K}_n)_{n=0}^M$ is a filtration.

Next verify Z_n measurability. For any Borel set $E \subseteq \mathbb{R}$, if $0 \notin E$, then

$$\{Z_n \in E\} \cap \{\tau_n = \infty\} = \varnothing \in \mathcal{H}_M,$$

If $0 \in E$, then

$$\{Z_n \in E\} \cap \{\tau_n = \infty\} = \{\tau_n = \infty\} \in \mathcal{H}_M.$$

At the same time, for each $t \in [M]$,

$$\{Z_n \in E\} \cap \{\tau_n = t\} = \{X_t \in E\} \cap \{\tau_n = t\} \in \mathcal{H}_t,$$

because $X_t \in \mathcal{H}_t$ and $\{\tau_n = t\} \in \mathcal{G}_t \subseteq \mathcal{H}_t$. Thus $\{Z_n \in E\} \in \mathcal{K}_n$, Z_n is measurable with respect to $\mathcal{K}_n$. Also, by $|X_t| \le B$ and $Z_n = 0$ on $\{\tau_n = \infty\}$, $|Z_n| \le B$.

Next prove zero mean. Fix $A \in \mathcal{K}_{n-1}$. We first prove that for each t,

$$A \cap \{\tau_n = t\} \in \mathcal{G}_t.$$

If $n = 1$, then $A \in \mathcal{K}_0 = \mathcal{H}_0 \subseteq \mathcal{G}_t$, and $\{\tau_1 = t\} \in \mathcal{G}_t$. Thus the claim holds. If $n \ge 2$, then on $\{\tau_n = t\}$ the $n-1$ hits must occur at some $s < t$, therefore

$$A \cap \{\tau_n = t\} = \bigcup_{s=1}^{t-1} \left(A \cap \{\tau_{n-1} = s\} \cap \{\tau_n = t\}\right).$$

Since $A \in \mathcal{K}_{n-1}$,

$$A \cap \{\tau_{n-1} = s\} \in \mathcal{H}_s.$$

Also, because $s < t$, we have $\mathcal{H}_s \subseteq \mathcal{G}_t$, and because $\{\tau_n = t\} \in \mathcal{G}_t$, each term in the finite union belongs to $\mathcal{G}_t$. Hence the above measurability holds.

Therefore

$$\mathbb{E}[\mathbf{1}_A Z_n] = \sum_{t=1}^{M} \mathbb{E}\big[\mathbf{1}_{A \cap \{\tau_n = t\}} X_t\big] = \sum_{t=1}^{M} \mathbb{E}\big[\mathbf{1}_{A \cap \{\tau_n = t\}} \mathbb{E}[X_t \mid \mathcal{G}_t]\big] = 0.$$

Thus $\mathbb{E}[Z_n \mid \mathcal{K}_{n-1}] = 0$, and $(Z_n, \mathcal{K}_n)$ is a B-bounded martingale difference sequence. The branch $\tau_n = \infty$ contributes zero and therefore does not produce X_∞ or any nonmeasurable term.

For fixed n, Lemma C.11 gives

$$\mathbb{P}\left(\left|\sum_{m=1}^{n} Z_m\right| \ge u\right) \le 2\exp\left(-\frac{u^2}{2B^2 n}\right).$$

Taking

$$u = 2B\sqrt{n \log(2M/\delta')}$$

makes the right-hand side at most δ'/M. A union bound over $n = 1, \ldots, M$ gives simultaneous control of all partial sums. Finally, if $N \ge 1$, taking $n = N$ gives the empirical mean bound. If $N = 0$, no empirical average is formed; the subsequent $N = 0$ cases are covered by the cold-start bonus clipping mechanism. This proves the claim. □

C.4.4 Reward and Transition Concentration

Before stating the concentration events, define the conditional-mean model around which the empirical window estimates concentrate. For any (k, h, ξ, ω), let

$$N_h^k(\xi, \omega) = \sum_{j \in \mathcal{W}_k} \mathbf{1}\{(\xi_h^j, \omega_h^j) = (\xi, \omega)\}.$$

If $N_h^k(\xi,\omega) \geq 1$, define

$$\breve{r}_h^k(\xi,\omega) := \frac{1}{N_h^k(\xi,\omega)} \sum_{j \in \mathcal{W}_k} r_h^{(j)}(\xi,\omega) \mathbf{1}\{(\xi_h^j, \omega_h^j) = (\xi,\omega)\},$$

and

$$\breve{P}_h^k(\cdot \mid \xi,\omega) := \frac{1}{N_h^k(\xi,\omega)} \sum_{j \in \mathcal{W}_k} P_h^{(j)}(\cdot \mid \xi,\omega) \mathbf{1}\{(\xi_h^j, \omega_h^j) = (\xi,\omega)\}.$$

If $N_h^k(\xi,\omega) = 0$, set

$$\breve{r}_h^k(\xi,\omega) := 0, \qquad \breve{P}_h^k(\cdot \mid \xi,\omega) := P_h^\circ(\cdot \mid \xi,\omega),$$

where $P_h^\circ(\cdot \mid \xi,\omega) \in \Delta(\Xi)$ is fixed before learning begins. These default values are not used in the concentration statements, which are only asserted for $N_h^k(\xi,\omega) \geq 1$, but they keep the notation globally single-valued.

Lemma C.13 (Reward Concentration around the Conditional Mean)**.** *There exists an absolute constant $c_r > 0$ such that, with probability at least $1 - \delta/4$, uniformly over all*

$$k \in [K], \quad h \in [H], \quad \xi \in \Xi, \quad \omega \in \Omega(\xi)$$

quadruples satisfying $N_h^k(\xi,\omega) \geq 1$, we have

$$\left|\hat{r}_h^k(\xi,\omega) - \breve{r}_h^k(\xi,\omega)\right| \leq c_r \sqrt{\frac{\log(C_{\log} S_\xi A H K/\delta)}{(N_h^k(\xi,\omega)+1) \wedge W}}.$$

The concentration event is stated for positive empirical counts. The zero-count regime is covered separately by the clipped cold-start bonus, which is the mechanism used in the near-optimism and certificate arguments.

Proof. Fix k, h, ξ, ω, and denote the episodes in the window as

$$u_1 < u_2 < \cdots < u_{m_k}, \qquad \{u_1, \ldots, u_{m_k}\} = \mathcal{W}_k, \qquad m_k = |\mathcal{W}_k| \leq W.$$

If $m_k = 0$, then there are no quadruples with $N_h^k(\xi,\omega) \geq 1$, so the assertion is empty. We therefore assume $m_k \geq 1$.

Let $\mathcal{G}_s$ be the pre-outcome σ-algebra at stage h of episode u_s, after the state and action have been determined but before the reward and next state have been revealed. Let

$$\mathcal{H}_s := \mathcal{G}_s \vee \sigma(r_h^{u_s}, \xi_{h+1}^{u_s}).$$

By the global filtration convention, $\mathcal{H}_{s-1} \subseteq \mathcal{G}_s$. Define

$$I_s := \mathbf{1}\{(\xi_h^{u_s}, \omega_h^{u_s}) = (\xi,\omega)\}, \qquad X_s := I_s\big(r_h^{u_s} - r_h^{(u_s)}(\xi,\omega)\big).$$

Then $I_s \in \mathcal{G}_s$, $X_s \in \mathcal{H}_s$, $X_s = I_s X_s$, $|X_s| \leq 1$, and the sampling model gives

$$\mathbb{E}[X_s \mid \mathcal{G}_s] = I_s\, \mathbb{E}[r_h^{u_s} - r_h^{(u_s)}(\xi,\omega) \mid \mathcal{G}_s] = 0.$$

Let $N := N_h^k(\xi,\omega)$. If $N \geq 1$, then

$$\hat{r}_h^k(\xi,\omega) - \breve{r}_h^k(\xi,\omega) = \frac{1}{N} \sum_{s=1}^{m_k} I_s\big(r_h^{u_s} - r_h^{(u_s)}(\xi,\omega)\big).$$

The number of possible quadruples is at most

$$K \cdot H \cdot \sum_{\xi \in \Xi} |\Omega(\xi)| \leq K H S_\xi A.$$

Thus the following failure-probability allocation is sufficient for a union bound over all (k, h, ξ, ω).

For fixed (k, h, ξ, ω), apply Lemma C.12 with $B = 1$, $M = m_k \le W$ and

$$\delta' = \frac{\delta}{4KS_\xi AH}.$$

By $m_k \le W$,

$$\log(2m_k/\delta') \le \log(8WKS_\xi AH/\delta).$$

We obtain, for $N \ge 1$,

$$\left|\hat{r}_h^k(\xi,\omega) - \breve{r}_h^k(\xi,\omega)\right| \le 2\sqrt{\frac{\log(8WKS_\xi AH/\delta)}{N}}.$$

Taking a union bound over all (k, h, ξ, ω) and using $1 \le W \le K$, for an absolute constant $c > 0$, after choosing $C_{\log}$ sufficiently large,

$$\log(8WKS_\xi AH/\delta) \le c\,\log(C_{\log}S_\xi AHK/\delta).$$

At the same time, when $N \ge 1$ and $N \le W$,

$$(N+1) \wedge W \le 2N, \qquad \text{therefore} \qquad \frac{1}{N} \le \frac{2}{(N+1)\wedge W}.$$

Absorbing absolute constants and taking the supremum over all $N \ge 1$ gives the stated bound. This proves the claim.

□

This subsection establishes ℓ_1-concentration for $\hat{P}_h^k(\cdot \mid \xi, \omega) - \breve{P}_h^k(\cdot \mid \xi, \omega)$. Whenever it is necessary to control $(\hat{P}_h^k - \breve{P}_h^k)^\top f$, we use

$$|(\hat{P}_h^k - \breve{P}_h^k)^\top f| \le \|f\|_\infty \|\hat{P}_h^k - \breve{P}_h^k\|_1.$$

We do not need a separate concentration result for the random test function $\widetilde{V}$.

Lemma C.14 (Transition Concentration around the Conditional Mean)**.** *There exists an absolute constant $c_P > 0$ such that, with probability at least $1 - \delta/4$, uniformly over all*

$$k \in [K], \quad h \in [H], \quad \xi \in \Xi, \quad \omega \in \Omega(\xi)$$

quadruples satisfying $N_h^k(\xi, \omega) \ge 1$, we have

$$\left\|\hat{P}_h^k(\cdot \mid \xi, \omega) - \breve{P}_h^k(\cdot \mid \xi, \omega)\right\|_1 \le c_P\sqrt{\frac{S_\xi \log(C_{\log}S_\xi AHK/\delta)}{(N_h^k(\xi,\omega)+1)\wedge W}}.$$

The concentration event is stated for positive empirical counts. The zero-count regime is covered by the same clipped cold-start bonus mechanism used in the subsequent optimism and certificate bounds.

Proof. Fix k, h, ξ, ω, and follow the same candidate episode order $u_1 < \cdots < u_{m_k}$ and filtration construction as in Lemma C.13. If $m_k = 0$, then there are no quadruples with $N_h^k(\xi, \omega) \ge 1$, so the assertion is empty. We therefore assume $m_k \ge 1$.

For each $v \in \{-1, 1\}^{S_\xi}$, define

$$I_s := \mathbf{1}\{(\xi_h^{u_s}, \omega_h^{u_s}) = (\xi, \omega)\},$$

$$X_s^v := I_s\, v^\top \left(e_{\xi_{h+1}^{u_s}} - P_h^{(u_s)}(\cdot \mid \xi, \omega)\right).$$

Then $I_s \in \mathcal{G}_s$, $X_s^v \in \mathcal{H}_s$, and

$$|X_s^v| \le |v^\top e_{\xi_{h+1}^{u_s}}| + |v^\top P_h^{(u_s)}(\cdot \mid \xi, \omega)| \le 2.$$

On the event $I_s = 1$, the sampling model gives

$$\mathbb{E}[e_{\xi_{h+1}^{u_s}} - P_h^{(u_s)}(\cdot \mid \xi, \omega) \mid \mathcal{G}_s] = 0,$$

whereas on the event $I_s = 0$, $X_s^v = 0$. Therefore

$$\mathbb{E}[X_s^v \mid \mathcal{G}_s] = 0.$$

Let $N := N_h^k(\xi, \omega)$. If $N \ge 1$, then

$$v^\top \Big(\hat{P}_h^k(\cdot \mid \xi, \omega) - \breve{P}_h^k(\cdot \mid \xi, \omega) \Big) = \frac{1}{N} \sum_{s=1}^{m_k} X_s^v.$$

For fixed $v \in \{-1, 1\}^{S_\xi}$, the number of possible quadruples (k, h, ξ, ω) is at most

$$K \cdot H \cdot \sum_{\xi \in \Xi} |\Omega(\xi)| \le KHS_\xi A.$$

Thus the following failure-probability allocation is sufficient for a union bound over all quadruples and all sign vectors $v \in \{-1, 1\}^{S_\xi}$.

For fixed (k, h, ξ, ω, v), apply Lemma C.12 with $B = 2$, $M = m_k \le W$ and

$$\delta' = \frac{\delta}{4KS_\xi AH\, 2^{S_\xi}}.$$

By $m_k \le W$,

$$\log(2m_k/\delta') \le \log(8WKS_\xi AH\, 2^{S_\xi}/\delta).$$

Therefore, for $N \ge 1$,

$$\left| v^\top \Big(\hat{P}_h^k(\cdot \mid \xi, \omega) - \breve{P}_h^k(\cdot \mid \xi, \omega) \Big) \right| \le 4 \sqrt{\frac{\log(8WKS_\xi AH\, 2^{S_\xi}/\delta)}{N}}.$$

Taking the supremum over all $v \in \{-1, 1\}^{S_\xi}$ and applying a union bound over all (k, h, ξ, ω), together with the finite-dimensional dual representation

$$\|x\|_1 = \max_{v \in \{-1,1\}^{S_\xi}} v^\top x, \qquad x \in \mathbb{R}^{S_\xi},$$

and using $1 \le W \le K$, for an absolute constant $c > 0$, after choosing $C_{\log}$ sufficiently large,

$$\log(8WKS_\xi AH\, 2^{S_\xi}/\delta) \le c\, S_\xi \log(C_{\log} S_\xi AHK/\delta),$$

gives the simultaneous bound for all $N_h^k(\xi, \omega) \ge 1$

$$\big\| \hat{P}_h^k(\cdot \mid \xi, \omega) - \breve{P}_h^k(\cdot \mid \xi, \omega) \big\|_1 \le c_P \sqrt{\frac{S_\xi \log(C_{\log} S_\xi AHK/\delta)}{N_h^k(\xi, \omega)}}.$$

Since $N := N_h^k(\xi, \omega) \ge 1$ and $N \le W$,

$$\frac{1}{N} \le \frac{2}{(N+1) \wedge W}.$$

Absorbing absolute constants gives the stated bound. This proves the claim.

□

Define

$$\mathcal{E}_r := \left\{ \begin{array}{l} \forall k, h, \xi, \omega \text{ with } N_h^k(\xi, \omega) \geq 1, \\ \qquad \left| \hat{r}_h^k(\xi, \omega) - \breve{r}_h^k(\xi, \omega) \right| \leq c_r \sqrt{\dfrac{\log(C_{\log} S_\xi AHK/\delta)}{(N_h^k(\xi, \omega) + 1) \wedge W}} \end{array} \right\},$$

and

$$\mathcal{E}_P := \left\{ \begin{array}{l} \forall k, h, \xi, \omega \text{ with } N_h^k(\xi, \omega) \geq 1, \\ \qquad \left\| \hat{P}_h^k(\cdot \mid \xi, \omega) - \breve{P}_h^k(\cdot \mid \xi, \omega) \right\|_1 \leq c_P \sqrt{\dfrac{S_\xi \log(C_{\log} S_\xi AHK/\delta)}{(N_h^k(\xi, \omega) + 1) \wedge W}} \end{array} \right\}.$$

Define

$$\mathcal{E}_{\text{conc}} := \mathcal{E}_r \cap \mathcal{E}_P.$$

Lemma C.15 (Total Concentration Event)**.** *If $C_{\log}$ is sufficiently large, then*

$$\mathbb{P}(\mathcal{E}_{\text{conc}}) \geq 1 - \delta/2.$$

Proof. By Lemmas C.13 and C.14,

$$\mathbb{P}(\mathcal{E}_r^c) \leq \delta/4, \qquad \mathbb{P}(\mathcal{E}_P^c) \leq \delta/4.$$

A union bound gives

$$\mathbb{P}(\mathcal{E}_{\text{conc}}^c) \leq \mathbb{P}(\mathcal{E}_r^c) + \mathbb{P}(\mathcal{E}_P^c) \leq \delta/2.$$

This proves the claim. □

C.5 Proof of the Fixed-Window Theorem

C.5.1 Current True Model

This section has three formal responsibilities:

1. unify the model objects and notation appearing in all subsequent propositions;
2. specify the unified high-probability event and its allocation of failure probabilities;
3. fix the same bonus interface jointly used in subsequent Proposition C.20, Lemmas C.21–C.22, Proposition C.23, and Theorem C.1.

We first unify the objects used in the stability analysis and in the dynamic-regret theorem. The proof separates two roles that are useful in non-stationary analysis: **nonstationarity enters through a window-average conditional model, while statistical concentration enters through the empirical model**. This separation keeps the optimism and certificate arguments aligned with the sliding-window estimator.

Recall from Subsection C.1.1 that the true finite-horizon KL-regularized MDP in episode $k \in [K]$ is

$$\mathcal{M}^{(k)} = \left(\Xi, \{\Omega(\xi)\}_{\xi \in \Xi}, \{r_h^{(k)}\}_{h=1}^H, \{P_h^{(k)}\}_{h=1}^H, H \right),$$

where, for any $h \in [H]$, $\xi \in \Xi$, and $\omega \in \Omega(\xi)$,

$$r_h^{(k)}(\xi, \omega) \in [0, 1], \qquad P_h^{(k)}(\cdot \mid \xi, \omega) \in \Delta(\Xi).$$

Recall that the corresponding soft Bellman operator $\mathcal{T}_h^{(k)}$ was defined after the KL Bellman recursion in Subsection C.3.1. With that notation, the optimal value functions satisfy

$$V_{H+1}^{\star,(k)} \equiv 0, \qquad V_h^{\star,(k)} = \mathcal{T}_h^{(k)} V_{h+1}^{\star,(k)}.$$

C.5.2 Primitive Drift and Window Cumulative Drift

Recall that the primitive adjacent drifts $\Delta_r^{(j)}$ and $\Delta_p^{(j)}$, $j = 1, \dots, K-1$, are defined in Subsection C.1.4 immediately after the total variation budget.

For episode k, define the sliding window

$$\mathcal{W}_k := \{\max\{1, k-W\}, \dots, k-1\}, \qquad m_k := |\mathcal{W}_k|.$$

The window cumulative drift quantity used uniformly in the main proof is defined as

$$D_k := \sum_{j=\max\{1,k-W\}}^{k-1} \left(\Delta_r^{(j)} + H\Delta_p^{(j)}\right). \tag{27}$$

Throughout the appendix, an empty sum is zero. In particular, if $k = 1$, then $\mathcal{W}_1 = \varnothing$ and $D_1 = 0$. The main analysis uniformly uses D_k, because it satisfies a clean summability relation

$$\sum_{k=1}^{K} D_k \le W(B_r + HB_p). \tag{28}$$

Discrete counting note. The above display follows directly from discrete counting. Indeed,

$$\sum_{k=1}^{K} D_k = \sum_{k=1}^{K} \sum_{j=\max\{1,k-W\}}^{k-1} \left(\Delta_r^{(j)} + H\Delta_p^{(j)}\right) = \sum_{j=1}^{K-1} c_j \left(\Delta_r^{(j)} + H\Delta_p^{(j)}\right),$$

where

$$c_j := \#\{k \in [K] : j \in [\max\{1, k-W\}, k-1]\}.$$

For fixed j, condition $j \in [\max\{1, k-W\}, k-1]$ is equivalent to

$$j + 1 \le k \le \min\{K, j+W\},$$

Thus

$$0 \le c_j \le W.$$

Hence

$$\sum_{k=1}^{K} D_k \le W \sum_{j=1}^{K-1} \left(\Delta_r^{(j)} + H\Delta_p^{(j)}\right) = W(B_r + HB_p).$$

C.5.3 Window Reference Model

Remark C.2 (Conditional-Mean Reference Model). The main proof below depends only on the conditional-mean model $\breve{r}_h^k, \breve{P}_h^k$, the concentration event $\mathcal{E}_{\text{conc}}$, and the window drift quantity D_k. No additional window-average model is used in the proof of Theorem C.1.

C.5.4 Conditional-Mean Model

Recall that the conditional-mean reward and transition model $\breve{r}_h^k, \breve{P}_h^k$ was defined before the concentration events in Subsection C.4.4. The role of this model is that the concentration of $\hat{r}_h^k, \hat{P}_h^k$ holds with respect to $\breve{r}_h^k, \breve{P}_h^k$, and the subsequent proof uses this conditional-mean reference model together with the drift quantity D_k.

C.5.5 Empirical Model and Optimistic Planning Objects

The empirical model is,

$$\hat{r}_h^k(\xi,\omega) := \begin{cases} \dfrac{1}{N_h^k(\xi,\omega)} \displaystyle\sum_{j\in\mathcal{W}_k} r_h^j \, \mathbf{1}\{(\xi_h^j,\omega_h^j)=(\xi,\omega)\}, & N_h^k(\xi,\omega)\geq 1, \\ 0, & N_h^k(\xi,\omega)=0, \end{cases}$$

$$\hat{P}_h^k(\xi' \mid \xi,\omega) := \begin{cases} \dfrac{1}{N_h^k(\xi,\omega)} \displaystyle\sum_{j\in\mathcal{W}_k} \mathbf{1}\{(\xi_h^j,\omega_h^j,\xi_{h+1}^j)=(\xi,\omega,\xi')\}, & N_h^k(\xi,\omega)\geq 1, \\ P_h^\circ(\xi' \mid \xi,\omega), & N_h^k(\xi,\omega)=0. \end{cases}$$

At the same time define

$$\widetilde{V}_{H+1}^k(\xi)=0, \qquad \forall \xi\in\Xi,$$

and recursively for $h=H,\dots,1$

$$\widetilde{Q}_h^k(\xi,\omega) := \min\Big\{H-h+1,\ \hat{r}_h^k(\xi,\omega)+b_h^k(\xi,\omega)+\hat{P}_h^k(\cdot\mid\xi,\omega)^\top \widetilde{V}_{h+1}^k\Big\},$$

$$\widetilde{V}_h^k(\xi) := \alpha\log\sum_{\omega\in\Omega(\xi)} \pi_{\mathrm{ref}}(\omega\mid\xi)\exp\left(\frac{\widetilde{Q}_h^k(\xi,\omega)}{\alpha}\right),$$

$$\hat{\pi}_h^k(\omega\mid\xi) := \frac{\pi_{\mathrm{ref}}(\omega\mid\xi)\exp(\widetilde{Q}_h^k(\xi,\omega)/\alpha)}{\sum_{\omega'}\pi_{\mathrm{ref}}(\omega'\mid\xi)\exp(\widetilde{Q}_h^k(\xi,\omega')/\alpha)}.$$

The bonus is

$$b_h^k(\xi,\omega) = c_b H\sqrt{\frac{S_\xi\log(C_{\log}S_\xi AHK/\delta)}{(N_h^k(\xi,\omega)+1)\wedge W}}$$

where $C_{\log}\geq e^2$ is an absolute constant, $c_b>0$ is an absolute constant. Throughout the main proof, assume

$$\delta\in(0,1), \qquad 1\leq W\leq K, \qquad S_\xi, A, H, K\geq 1.$$

Let c_r and c_P denote respectively the reward and transition concentration constants in Lemmas C.13 and C.14, and let

$$c_{\mathrm{conc}} := c_r + c_P.$$

Choose $C_{\log}$ and c_b such that

$$c_b\geq 2(c_r+c_P+1)\geq c_{\mathrm{conc}}, \qquad c_b\sqrt{S_\xi\log(C_{\log}S_\xi AHK/\delta)}\geq 2.$$

Hence, when $N_h^k(\xi,\omega)=0$,

$$b_h^k(\xi,\omega) = c_b H\sqrt{S_\xi\log(C_{\log}S_\xi AHK/\delta)}\geq 2H.$$

All subsequent cold-start concentration absorption only invoke this parameter convention, and do not change the bonus separately. Here $(+1)$ is used to remove cold-start singularity, while $\wedge W$ is kept consistent with the effective sample size of the sliding window.

Concentration absorption convention. For any $n\in\{1,\dots,W\}$, because $H\geq 1$ and $S_\xi\geq 1$,

$$\begin{aligned} &c_r\sqrt{\frac{\log(C_{\log}S_\xi AHK/\delta)}{n}} + c_P H\sqrt{\frac{S_\xi\log(C_{\log}S_\xi AHK/\delta)}{n}} \\ &\quad\leq (c_r+c_P)H\sqrt{\frac{S_\xi\log(C_{\log}S_\xi AHK/\delta)}{n}} \leq b_h^k(\xi,\omega) \end{aligned}$$

whenever $n = (N_h^k(\xi,\omega)+1) \wedge W$. Thus the statistical reward and transition deviations are explicitly absorbed by the single bonus term; zero-count cases use only the preceding cold-start condition.

Unified interface convention. From this section onward, after Proposition C.20, Lemmas C.21–C.22, Proposition C.23, and Theorem C.1 and their proofs

$$b_h^k(\xi,\omega)$$

all refer to the same window-truncated bonus definition above; subsequent sections do not switch back to any other bonus notation. The near-optimism result, certificate decomposition, summed-bonus bound, and aggregate dynamic-regret theorem all hold for the same algorithmic object.

Lemma C.16 (Boundedness of Clipped Optimistic Values)**.** *For every episode k, stage $h \in [H]$, state $\xi \in \Xi$, and action $\omega \in \Omega(\xi)$, the optimistic planning objects defined in this subsection satisfy*

$$0 \le \widetilde{Q}_h^k(\xi,\omega) \le H-h+1, \qquad 0 \le \widetilde{V}_h^k(\xi) \le H-h+1.$$

Proof. The terminal condition gives $\widetilde{V}_{H+1}^k \equiv 0$. Suppose $0 \le \widetilde{V}_{h+1}^k(\xi') \le H-h$ for all $\xi' \in \Xi$. Since $\hat{r}_h^k(\xi,\omega) \ge 0$, $b_h^k(\xi,\omega) \ge 0$, and $\hat{P}_h^k(\cdot \mid \xi,\omega) \in \Delta(\Xi)$, the unclipped backup is nonnegative. Therefore the clipping definition gives

$$0 \le \widetilde{Q}_h^k(\xi,\omega) \le H-h+1.$$

Let

$$m := \min_{\omega\in\Omega(\xi)} \widetilde{Q}_h^k(\xi,\omega), \qquad M := \max_{\omega\in\Omega(\xi)} \widetilde{Q}_h^k(\xi,\omega).$$

Since $\pi_{\mathrm{ref}}(\cdot \mid \xi)$ is a probability distribution and $\alpha > 0$,

$$e^{m/\alpha} \le \sum_{\omega\in\Omega(\xi)} \pi_{\mathrm{ref}}(\omega \mid \xi) e^{\widetilde{Q}_h^k(\xi,\omega)/\alpha} \le e^{M/\alpha}.$$

Taking $\alpha \log$ gives

$$\min_{\omega\in\Omega(\xi)} \widetilde{Q}_h^k(\xi,\omega) \le \widetilde{V}_h^k(\xi) \le \max_{\omega\in\Omega(\xi)} \widetilde{Q}_h^k(\xi,\omega).$$

Thus $0 \le \widetilde{V}_h^k(\xi) \le H-h+1$. Backward induction proves the claim. □

Lemma C.17 (Unified Bonus Absorption)**.** *Under the parameter convention in this subsection, on $\mathcal{E}_{\mathrm{conc}}$, uniformly over all (k,h,ξ,ω), if $N_h^k(\xi,\omega) \ge 1$, then*

$$|\hat{r}_h^k(\xi,\omega) - \check{r}_h^k(\xi,\omega)| + H\|\hat{P}_h^k(\cdot \mid \xi,\omega) - \check{P}_h^k(\cdot \mid \xi,\omega)\|_1 \le b_h^k(\xi,\omega).$$

If $N_h^k(\xi,\omega) = 0$, then

$$b_h^k(\xi,\omega) \ge 2H.$$

Proof. For $N_h^k(\xi,\omega) \ge 1$, the event $\mathcal{E}_r$ controls the reward deviation and $\mathcal{E}_P$ controls the transition deviation. The concentration absorption convention then absorbs their sum into the single bonus $b_h^k(\xi,\omega)$. For $N_h^k(\xi,\omega) = 0$, the cold-start convention gives

$$b_h^k(\xi,\omega) = c_b H\sqrt{S_\xi \log(C_{\log} S_\xi AHK/\delta)} \ge 2H.$$

This proves the claim.

□

Measurability note. For each episode k, the planning objects

$$\hat{r}_h^k,\ \hat{P}_h^k,\ b_h^k,\ \widetilde{Q}_h^k,\ \widetilde{V}_h^k,\ \hat{\pi}_h^k$$

are all constructed only from episode k window history before the beginning of episode k, using only the window history $\mathcal{W}_k \subseteq \{1,\dots,k-1\}$. Therefore, for any h, the random function

$$\widetilde{V}_{h+1}^k$$

is measurable with respect to the beginning-of-episode k σ-algebra. This measurability is used only in the martingale filtration definitions in the executed-policy bridge; the transition concentration argument no longer treats $\widetilde{V}_{h+1}^k$ as a test function.

C.5.6 Model Difference Bounds

The proof of Theorem C.1 uses the conditional-mean drift bound below as the only model-comparison input in this subsection.

Remark C.3 (Only the Conditional-Mean Drift Bound Is Used). The proof of Theorem C.1 uses only Lemma C.18. No separate drift bound for an auxiliary window-average model is invoked.

Lemma C.18 (Conditional-Mean Model Drift)**.** *If* $N_h^k(\xi,\omega) \geq 1$*, then*

$$|\breve{r}_h^k(\xi,\omega) - r_h^{(k)}(\xi,\omega)| \leq D_k,$$

and

$$\|\breve{P}_h^k(\cdot \mid \xi,\omega) - P_h^{(k)}(\cdot \mid \xi,\omega)\|_1 \leq \frac{D_k}{H}.$$

Proof. Let

$$N := N_h^k(\xi,\omega) \geq 1.$$

The episodes in the window hitting (h,ξ,ω) have indices

$$j_1 < j_2 < \cdots < j_N, \qquad j_m \in \mathcal{W}_k.$$

By definition,

$$\breve{r}_h^k(\xi,\omega) = \frac{1}{N}\sum_{m=1}^{N} r_h^{(j_m)}(\xi,\omega).$$

Therefore

$$\breve{r}_h^k(\xi,\omega) - r_h^{(k)}(\xi,\omega) = \frac{1}{N}\sum_{m=1}^{N}\big(r_h^{(j_m)}(\xi,\omega) - r_h^{(k)}(\xi,\omega)\big).$$

Taking absolute values and using the triangle inequality,

$$|\breve{r}_h^k(\xi,\omega) - r_h^{(k)}(\xi,\omega)| \leq \frac{1}{N}\sum_{m=1}^{N} |r_h^{(j_m)}(\xi,\omega) - r_h^{(k)}(\xi,\omega)|.$$

For each m, because $j_m \in \mathcal{W}_k$, by telescoping the primitive drift definitions,

$$|r_h^{(j_m)}(\xi,\omega) - r_h^{(k)}(\xi,\omega)| \leq \sum_{\ell=j_m}^{k-1} \Delta_r^{(\ell)} \leq \sum_{\ell=\max\{1,k-W\}}^{k-1} \Delta_r^{(\ell)}.$$

Thus

$$|\breve{r}_h^k(\xi,\omega) - r_h^{(k)}(\xi,\omega)| \leq \sum_{\ell=\max\{1,k-W\}}^{k-1} \Delta_r^{(\ell)} \leq D_k.$$

Similarly,

$$\breve{P}_h^k(\cdot \mid \xi,\omega) = \frac{1}{N}\sum_{m=1}^{N} P_h^{(j_m)}(\cdot \mid \xi,\omega),$$

Hence

$$\|\breve{P}_h^k(\cdot \mid \xi,\omega) - P_h^{(k)}(\cdot \mid \xi,\omega)\|_1 \leq \frac{1}{N}\sum_{m=1}^{N} \|P_h^{(j_m)}(\cdot \mid \xi,\omega) - P_h^{(k)}(\cdot \mid \xi,\omega)\|_1.$$

Again by telescoping,

$$\|P_h^{(j_m)}(\cdot \mid \xi,\omega) - P_h^{(k)}(\cdot \mid \xi,\omega)\|_1 \leq \sum_{\ell=j_m}^{k-1} \Delta_p^{(\ell)} \leq \sum_{\ell=\max\{1,k-W\}}^{k-1} \Delta_p^{(\ell)}.$$

Thus

$$\|\breve{P}_h^k(\cdot \mid \xi,\omega) - P_h^{(k)}(\cdot \mid \xi,\omega)\|_1 \le \sum_{\ell=\max\{1,k-W\}}^{k-1} \Delta_p^{(\ell)} \le \frac{D_k}{H},$$

because

$$D_k = \sum_{\ell=\max\{1,k-W\}}^{k-1} \big(\Delta_r^{(\ell)} + H\Delta_p^{(\ell)}\big) \ge H \sum_{\ell=\max\{1,k-W\}}^{k-1} \Delta_p^{(\ell)}.$$

Boundary remark for $N_h^k(\xi,\omega)=0$. When $N_h^k(\xi,\omega)=0$, $\breve{r}_h^k(\xi,\omega)$ and $\breve{P}_h^k(\cdot \mid \xi,\omega)$ follow the fixed convention values; the subsequent near-optimism and certificate decomposition arguments cover this regime through optimistic clipping and maximal bonus, while Lemma C.18 is used for the positive-count regime.

This proves the claim. □

C.5.7 Near-Optimism under Sliding-Window Estimation

Lemma C.19 (Single-Step Optimistic Domination, Including Zero-Count Cases)**.** *On the event $\mathcal{E}_{\mathrm{conc}}$, under the parameter convention of Subsection C.5.5, there exists an absolute constant $C_{\mathrm{loc}} > 0$ such that, for any*

$$k \in [K], \quad h \in [H], \quad \beta_{h+1} \ge 0,$$

if

$$\widetilde{V}_{h+1}^k(\xi') \ge V_{h+1}^{\star,(k)}(\xi') - \beta_{h+1}D_k, \qquad \forall \xi' \in \Xi,$$

then, for every $\xi \in \Xi$ and every $\omega \in \Omega(\xi)$, we have

$$\widetilde{Q}_h^k(\xi,\omega) \ge Q_h^{\star,(k)}(\xi,\omega) - \big(\beta_{h+1} + C_{\mathrm{loc}}\big)D_k.$$

Proof. Fix k,h,ξ,ω, and write

$$N := N_h^k(\xi,\omega), \qquad U_h^k(\xi,\omega) := \hat{r}_h^k(\xi,\omega) + b_h^k(\xi,\omega) + \hat{P}_h^k(\cdot \mid \xi,\omega)^\top \widetilde{V}_{h+1}^k.$$

By the induction hypothesis and because $\hat{P}_h^k(\cdot \mid \xi,\omega) \in \Delta(\Xi)$,

$$\hat{P}_h^k(\cdot \mid \xi,\omega)^\top \widetilde{V}_{h+1}^k \ge \hat{P}_h^k(\cdot \mid \xi,\omega)^\top V_{h+1}^{\star,(k)} - \beta_{h+1}D_k.$$

Hence

$$U_h^k(\xi,\omega) \ge \hat{r}_h^k(\xi,\omega) + b_h^k(\xi,\omega) + \hat{P}_h^k(\cdot \mid \xi,\omega)^\top V_{h+1}^{\star,(k)} - \beta_{h+1}D_k.$$

First suppose $N \ge 1$. We compare the last display with

$$Q_h^{\star,(k)}(\xi,\omega) = r_h^{(k)}(\xi,\omega) + P_h^{(k)}(\cdot \mid \xi,\omega)^\top V_{h+1}^{\star,(k)}.$$

Decompose the reward and transition errors as

$$\hat{r}_h^k - r_h^{(k)} = (\hat{r}_h^k - \breve{r}_h^k) + (\breve{r}_h^k - r_h^{(k)})$$

and

$$\hat{P}_h^{k\top} V_{h+1}^{\star,(k)} - P_h^{(k)\top} V_{h+1}^{\star,(k)} = (\hat{P}_h^k - \breve{P}_h^k)^\top V_{h+1}^{\star,(k)} + (\breve{P}_h^k - P_h^{(k)})^\top V_{h+1}^{\star,(k)}.$$

On $\mathcal{E}_{\mathrm{conc}}$, Lemma C.17 gives

$$\big|\hat{r}_h^k - \breve{r}_h^k\big| + H\|\hat{P}_h^k - \breve{P}_h^k\|_1 \le b_h^k(\xi,\omega).$$

Moreover, by Lemma C.18,

$$\big|\breve{r}_h^k - r_h^{(k)}\big| \le D_k, \qquad \|\breve{P}_h^k - P_h^{(k)}\|_1 \le \frac{D_k}{H}.$$

By Lemma C.7, $\|V_{h+1}^{\star,(k)}\|_\infty \le H - h \le H$. Therefore, we obtain

$$\begin{aligned}
&\hat{r}_h^k(\xi,\omega) - r_h^{(k)}(\xi,\omega) + \big(\hat{P}_h^k(\cdot \mid \xi,\omega) - P_h^{(k)}(\cdot \mid \xi,\omega)\big)^\top V_{h+1}^{\star,(k)} \\
&\quad \ge -\big|\hat{r}_h^k - \check{r}_h^k\big| - H\|\hat{P}_h^k - \check{P}_h^k\|_1 - \big|\check{r}_h^k - r_h^{(k)}\big| - H\|\check{P}_h^k - P_h^{(k)}\|_1 \\
&\quad \ge -b_h^k(\xi,\omega) - 2D_k.
\end{aligned}$$

Substituting this bound into the lower bound for $U_h^k(\xi,\omega)$ yields

$$U_h^k(\xi,\omega) \ge Q_h^{\star,(k)}(\xi,\omega) - \big(\beta_{h+1} + 2\big)D_k.$$

Choose an absolute constant $C_{\mathrm{loc}} \ge 2$. Since Lemma C.7 also gives $Q_h^{\star,(k)}(\xi,\omega) \le H - h + 1$, and

$$\widetilde{Q}_h^k(\xi,\omega) = \min\{H - h + 1, U_h^k(\xi,\omega)\},$$

we get

$$\widetilde{Q}_h^k(\xi,\omega) \ge Q_h^{\star,(k)}(\xi,\omega) - \big(\beta_{h+1} + C_{\mathrm{loc}}\big)D_k.$$

It remains to consider $N = 0$. In this case, neither concentration nor Lemma C.18 is used. By the zero-count default convention,

$$\hat{r}_h^k(\xi,\omega) = 0, \qquad \hat{P}_h^k(\cdot \mid \xi,\omega) = P_h^\circ(\cdot \mid \xi,\omega).$$

By Lemma C.16, $\widetilde{V}_{h+1}^k \ge 0$, and hence

$$U_h^k(\xi,\omega) = b_h^k(\xi,\omega) + P_h^\circ(\cdot \mid \xi,\omega)^\top \widetilde{V}_{h+1}^k \ge b_h^k(\xi,\omega).$$

By Lemma C.17, in the zero-count case

$$b_h^k(\xi,\omega) \ge 2H \ge H - h + 1.$$

Therefore

$$U_h^k(\xi,\omega) \ge H - h + 1,$$

and so

$$\widetilde{Q}_h^k(\xi,\omega) = \min\{H - h + 1, U_h^k(\xi,\omega)\} = H - h + 1.$$

By Lemma C.7, $Q_h^{\star,(k)}(\xi,\omega) \le H - h + 1$. Therefore

$$\widetilde{Q}_h^k(\xi,\omega) \ge Q_h^{\star,(k)}(\xi,\omega) \ge Q_h^{\star,(k)}(\xi,\omega) - \big(\beta_{h+1} + C_{\mathrm{loc}}\big)D_k.$$

Combining the cases $N \ge 1$ and $N = 0$ proves the claim.

□

Proposition C.20 (Near-Optimism under Window Drift)**.** *On the event $\mathcal{E}_{\mathrm{conc}}$, there exists an absolute constant $C_{\mathrm{opt}} > 0$ such that, uniformly over all*

$$k \in [K], \quad h \in [H], \quad \xi \in \Xi,$$

we have

$$\widetilde{V}_h^k(\xi) \ge V_h^{\star,(k)}(\xi) - C_{\mathrm{opt}}(H - h + 1)D_k. \tag{29}$$

Proof. We prove the claim by backward induction on h. The boundary case $h = H + 1$ satisfies $\widetilde{V}_{H+1}^k(\xi) = V_{H+1}^{\star,(k)}(\xi) = 0$.

Fix

$$C_{\mathrm{opt}} := C_{\mathrm{loc}}.$$

Assume that the claim has already been established for $h+1 \le H+1$, namely

$$\widetilde{V}_{h+1}^k(\xi') \ge V_{h+1}^{\star,(k)}(\xi') - C_{\mathrm{opt}}(H-h)D_k, \qquad \forall \xi' \in \Xi.$$

By Lemma C.19, taking $\beta_{h+1} = C_{\mathrm{opt}}(H-h)$, for any (ξ, ω)

$$\widetilde{Q}_h^k(\xi,\omega) \ge Q_h^{\star,(k)}(\xi,\omega) - \big(C_{\mathrm{opt}}(H-h) + C_{\mathrm{loc}}\big)D_k.$$

Since $C_{\mathrm{opt}} = C_{\mathrm{loc}}$, the right-hand side is

$$Q_h^{\star,(k)}(\xi,\omega) - C_{\mathrm{opt}}(H-h+1)D_k.$$

By Lemma C.5, applied with

$$q_\omega = \widetilde{Q}_h^k(\xi,\omega), \qquad q'_\omega = Q_h^{\star,(k)}(\xi,\omega), \qquad \varepsilon = C_{\mathrm{opt}}(H-h+1)D_k,$$

we obtain

$$\widetilde{V}_h^k(\xi) \ge V_h^{\star,(k)}(\xi) - C_{\mathrm{opt}}(H-h+1)D_k.$$

This closes the induction and proves the claim. □

C.5.8 Certificate Decomposition

This subsection first establishes a pathwise bridge identity with a unified remainder and then derives the certificate upper bound on the event $\mathcal{E}_{\mathrm{conc}}$. The martingale-control argument uses the same conditional-mean identity, and Theorem C.1 invokes the resulting certificate bound on $\mathcal{E}_{\mathrm{tot}}$.

Lemma C.21 (Pathwise bridge identity and unified remainder)**.** *Fix an arbitrary episode $k \in [K]$. Define*

$$\Gamma_h^k(\xi) := \widetilde{V}_h^k(\xi) - V_h^{\hat{\pi}^k,(k)}(\xi), \qquad \Gamma_{H+1}^k \equiv 0.$$

Along the true trajectory $(\xi_h^k, \omega_h^k, \xi_{h+1}^k)$, define

$$G_h^k(\xi,\omega) := \widetilde{Q}_h^k(\xi,\omega) - r_h^{(k)}(\xi,\omega) - P_h^{(k)}(\cdot \mid \xi,\omega)^\top V_{h+1}^{\hat{\pi}^k,(k)},$$

$$a_h^k := \Gamma_h^k(\xi_h^k) - G_h^k(\xi_h^k,\omega_h^k),$$

$$t_h^k := P_h^{(k)}(\cdot \mid \xi_h^k,\omega_h^k)^\top \Gamma_{h+1}^k - \Gamma_{h+1}^k(\xi_{h+1}^k),$$

and

$$\varepsilon_h^k := a_h^k + t_h^k, \qquad M_k := \sum_{h=1}^H \varepsilon_h^k.$$

Then, for every $h \in [H]$ and every $\xi \in \Xi$,

$$\Gamma_h^k(\xi) = \sum_{\omega \in \Omega(\xi)} \hat{\pi}_h^k(\omega \mid \xi) G_h^k(\xi,\omega).$$

Consequently, along the realized trajectory,

$$a_h^k = \sum_{\omega \in \Omega(\xi_h^k)} \hat{\pi}_h^k(\omega \mid \xi_h^k) G_h^k(\xi_h^k,\omega) - G_h^k(\xi_h^k,\omega_h^k).$$

Moreover,

$$\Gamma_h^k(\xi_h^k) = G_h^k(\xi_h^k,\omega_h^k) + a_h^k,$$

and

$$\Gamma_h^k(\xi_h^k) = \Big[\widetilde{Q}_h^k(\xi_h^k,\omega_h^k) - r_h^{(k)}(\xi_h^k,\omega_h^k) - P_h^{(k)}(\cdot \mid \xi_h^k,\omega_h^k)^\top \widetilde{V}_{h+1}^k\Big] + \Gamma_{h+1}^k(\xi_{h+1}^k) + \varepsilon_h^k.$$

Proof. For any state ξ, apply Proposition C.3 to the vector $q_\omega = \widetilde{Q}_h^k(\xi,\omega)$, $\omega \in \Omega(\xi)$. By the definition of $\hat{\pi}_h^k$, this policy is the corresponding softmax maximizer, and the Fenchel equality in Proposition C.3 gives

$$\widetilde{V}_h^k(\xi) = \sum_{\omega\in\Omega(\xi)} \hat{\pi}_h^k(\omega \mid \xi) \left[\widetilde{Q}_h^k(\xi,\omega) - \alpha \log \frac{\hat{\pi}_h^k(\omega \mid \xi)}{\pi_{\mathrm{ref}}(\omega \mid \xi)}\right].$$

The policy value satisfies

$$V_h^{\hat{\pi}^k,(k)}(\xi) = \sum_{\omega\in\Omega(\xi)} \hat{\pi}_h^k(\omega \mid \xi)\Big(r_h^{(k)}(\xi,\omega) + P_h^{(k)}(\cdot \mid \xi,\omega)^\top V_{h+1}^{\hat{\pi}^k,(k)} - \alpha \log \frac{\hat{\pi}_h^k(\omega \mid \xi)}{\pi_{\mathrm{ref}}(\omega \mid \xi)}\Big).$$

Subtracting the two displays gives

$$\Gamma_h^k(\xi) = \sum_\omega \hat{\pi}_h^k(\omega \mid \xi)\, G_h^k(\xi,\omega).$$

For the action term, the definition of a_h^k gives

$$\Gamma_h^k(\xi_h^k) = G_h^k(\xi_h^k,\omega_h^k) + a_h^k.$$

On the other hand,

$$G_h^k(\xi_h^k,\omega_h^k) = \widetilde{Q}_h^k(\xi_h^k,\omega_h^k) - r_h^{(k)}(\xi_h^k,\omega_h^k) - P_h^{(k)}(\cdot \mid \xi_h^k,\omega_h^k)^\top \widetilde{V}_{h+1}^k + P_h^{(k)}(\cdot \mid \xi_h^k,\omega_h^k)^\top \Gamma_{h+1}^k.$$

By the definition of t_h^k,

$$P_h^{(k)}(\cdot \mid \xi_h^k,\omega_h^k)^\top \Gamma_{h+1}^k = \Gamma_{h+1}^k(\xi_{h+1}^k) + t_h^k,$$

we obtain the identity. This proves the claim.

□

Lemma C.22 (On-Event Pathwise Certificate Bound)**.** *On the event $\mathcal{E}_{\mathrm{conc}}$, there exists an absolute constant $C_{\mathrm{cert}} > 0$ such that, uniformly over all $k \in [K]$ and $h \in [H]$, we have*

$$\Gamma_h^k(\xi_h^k) \le 2\, b_h^k(\xi_h^k,\omega_h^k) + \Gamma_{h+1}^k(\xi_{h+1}^k) + \varepsilon_h^k + C_{\mathrm{cert}} D_k. \tag{30}$$

Therefore,

$$\widetilde{V}_1^k(\xi_1^k) - V_1^{\hat{\pi}^k,(k)}(\xi_1^k) \le 2\sum_{h=1}^H b_h^k(\xi_h^k,\omega_h^k) + M_k + C_{\mathrm{cert}} H D_k. \tag{31}$$

Proof. By the optimistic backup and Lemma C.21,

$$\Gamma_h^k(\xi_h^k) \le \hat{r}_h^k - r_h^{(k)} + b_h^k + (\hat{P}_h^k - P_h^{(k)})^\top \widetilde{V}_{h+1}^k + \Gamma_{h+1}^k(\xi_{h+1}^k) + \varepsilon_h^k,$$

where all unindexed terms are evaluated at (h,ξ_h^k,ω_h^k).

Let

$$N := N_h^k(\xi_h^k,\omega_h^k).$$

If $N \ge 1$, decompose

$$\hat{r}_h^k - r_h^{(k)} = (\hat{r}_h^k - \check{r}_h^k) + (\check{r}_h^k - r_h^{(k)}),$$

$$(\hat{P}_h^k - P_h^{(k)})^\top \widetilde{V}_{h+1}^k = (\hat{P}_h^k - \check{P}_h^k)^\top \widetilde{V}_{h+1}^k + (\check{P}_h^k - P_h^{(k)})^\top \widetilde{V}_{h+1}^k.$$

On the event $\mathcal{E}_{\mathrm{conc}}$, by Lemmas C.17 and C.18, and by Lemma C.16, $\|\widetilde{V}_{h+1}^k\|_\infty \le H$, for $N \ge 1$,

$$\hat{r}_h^k - r_h^{(k)} + (\hat{P}_h^k - P_h^{(k)})^\top \widetilde{V}_{h+1}^k \le b_h^k(\xi_h^k,\omega_h^k) + 2D_k.$$

If $N = 0$, Lemma C.18 is not invoked. Instead, by the default convention $\hat{r}_h^k(\xi_h^k, \omega_h^k) = 0$ and $r_h^{(k)} \in [0, 1]$, we get $\hat{r}_h^k - r_h^{(k)} = -r_h^{(k)} \le 0$. At the same time, $\hat{P}_h^k$ and $P_h^{(k)}$ are probability distributions, and by Lemma C.16, $0 \le \widetilde{V}_{h+1}^k \le H$, therefore

$$(\hat{P}_h^k - P_h^{(k)})^\top \widetilde{V}_{h+1}^k \le (\hat{P}_h^k)^\top \widetilde{V}_{h+1}^k \le H.$$

Hence

$$\hat{r}_h^k - r_h^{(k)} + (\hat{P}_h^k - P_h^{(k)})^\top \widetilde{V}_{h+1}^k \le 0 + H = H,$$

while Lemma C.17 gives $b_h^k \ge 2H \ge H$. Therefore the cold-start contribution is directly absorbed by the same bonus term.

Therefore, in both cases, and using $D_k \ge 0$ for the zero-count case, the unified concentration interface gives, for example with $C_0 = 2$,

$$\hat{r}_h^k - r_h^{(k)} + (\hat{P}_h^k - P_h^{(k)})^\top \widetilde{V}_{h+1}^k \le b_h^k(\xi_h^k, \omega_h^k) + C_0 D_k.$$

Substituting back into the initial inequality and taking $C_{\text{cert}} \ge C_0$ gives the certificate bound. Summing over $h = 1, \dots, H$, telescoping, and using $\Gamma_{H+1}^k \equiv 0$ gives the episode-wise bound. This proves the claim. □

C.5.9 Summed Bonus Bound

Proposition C.23 (Summed Bonus Bound under Overlapping Sliding-Window Counts)**.** *For any $1 \le W \le K$, consider the sliding-window bonus defined in Subsection C.5.5.*

$$b_h^k(\xi, \omega) = c_b H \sqrt{\frac{S_\xi \log(C_{\log} S_\xi AHK/\delta)}{(N_h^k(\xi, \omega) + 1) \wedge W}}.$$

Then there exists an absolute constant $C_{\text{bon}} > 0$ such that

$$\sum_{k=1}^{K} \sum_{h=1}^{H} b_h^k(\xi_h^k, \omega_h^k) \le C_{\text{bon}}\, H^2 K S_\xi \sqrt{\frac{A \log(C_{\log} S_\xi AHK/\delta)}{W}}. \tag{32}$$

Up to absolute constants,

$$\sum_{k=1}^{K} \sum_{h=1}^{H} b_h^k(\xi_h^k, \omega_h^k) = \widetilde{O}\left(H^2 K S_\xi \sqrt{\frac{A}{W}} \right).$$

Proof. We use an overlapping-window counting argument and keep the $\sqrt{S_\xi}$ factor from the bonus explicitly.

Define

$$\mathcal{Z} := \{(h, \xi, \omega) : h \in [H],\ \xi \in \Xi,\ \omega \in \Omega(\xi)\}.$$

Then

$$|\mathcal{Z}| \le H S_\xi A.$$

Partition the episode axis into non-overlapping blocks of length W

$$\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_B, \qquad B := \left\lceil \frac{K}{W} \right\rceil,$$

where

$$\mathcal{B}_b := \{(b-1)W + 1, \dots, \min\{bW, K\}\}.$$

For each fixed triple $z = (h, \xi, \omega) \in \mathcal{Z}$ and block $b \in [B]$, define the number of occurrences of z in the block

$$M_{b,z} := \sum_{k \in \mathcal{B}_b} \mathbf{1}\{(\xi_h^k, \omega_h^k) = (\xi, \omega)\}.$$

If $M_{b,z} \geq 1$, write its occurrence times as

$$k_{b,z,1} < k_{b,z,2} < \cdots < k_{b,z,M_{b,z}}, \qquad k_{b,z,j} \in \mathcal{B}_b.$$

For $i < j$, the two occurrence times $k_{b,z,i}$ and $k_{b,z,j}$ belong to the same block $\mathcal{B}_b$, whose length is at most W. Hence

$$1 \leq k_{b,z,j} - k_{b,z,i} \leq W - 1.$$

Therefore

$$k_{b,z,i} \in \{\max\{1, k_{b,z,j} - W\}, \ldots, k_{b,z,j} - 1\} = \mathcal{W}_{k_{b,z,j}}.$$

Thus, when the j-th occurrence of z in block b happens, the previous $j-1$ occurrences of the same z in that block are all counted in the sliding-window count. Consequently,

$$N_h^{k_{b,z,j}}(\xi, \omega) \geq j - 1, \qquad (N_h^{k_{b,z,j}}(\xi, \omega) + 1) \wedge W \geq j.$$

Therefore

$$b_h^{k_{b,z,j}}(\xi, \omega) \leq c_b H \sqrt{\frac{S_\xi \log(C_{\log} S_\xi AHK/\delta)}{j}}.$$

Summing over the occurrences of z inside block b and using $\sum_{j=1}^{m} j^{-1/2} \leq 2\sqrt{m}$, we obtain

$$\sum_{k \in \mathcal{B}_b} b_h^k(\xi_h^k, \omega_h^k) \mathbf{1}\{(\xi_h^k, \omega_h^k) = (\xi, \omega)\} \leq 2c_b H \sqrt{S_\xi \log(C_{\log} S_\xi AHK/\delta)} \sqrt{M_{b,z}}.$$

Summing this deterministic inequality over all blocks b and triples $z \in \mathcal{Z}$, and then applying Cauchy–Schwarz, gives

$$\sum_{k=1}^{K} \sum_{h=1}^{H} b_h^k(\xi_h^k, \omega_h^k) \leq 2c_b H \sqrt{S_\xi \log(C_{\log} S_\xi AHK/\delta)} \sum_{b=1}^{B} \sum_{z \in \mathcal{Z}} \sqrt{M_{b,z}}$$

and

$$\sum_{b=1}^{B} \sum_{z \in \mathcal{Z}} \sqrt{M_{b,z}} \leq \sqrt{B|\mathcal{Z}|} \sqrt{\sum_{b=1}^{B} \sum_{z \in \mathcal{Z}} M_{b,z}}.$$

Since

$$\sum_{b=1}^{B} \sum_{z \in \mathcal{Z}} M_{b,z} = KH, \qquad B \leq \left\lceil \frac{K}{W} \right\rceil \leq \frac{2K}{W}, \qquad |\mathcal{Z}| \leq HS_\xi A,$$

we have

$$\sum_{b=1}^{B} \sum_{z \in \mathcal{Z}} \sqrt{M_{b,z}} \leq \sqrt{\frac{2K}{W} \cdot HS_\xi A} \sqrt{KH} = \sqrt{2}\, HK \sqrt{\frac{S_\xi A}{W}}.$$

Substituting this bound yields

$$\sum_{k=1}^{K} \sum_{h=1}^{H} b_h^k(\xi_h^k, \omega_h^k) \leq 2\sqrt{2}\, c_b\, H^2 K S_\xi \sqrt{\frac{A \log(C_{\log} S_\xi AHK/\delta)}{W}}.$$

Absorbing the absolute factor $2\sqrt{2}\, c_b$ into C_{bon} proves the claim.

□

Remark C.4 (Use of the Summed-Bonus Bound). This proposition is a pathwise combinatorial bound controlling the realized bonus on the true trajectory. It is used pathwise in Theorem C.1.

C.5.10 Pathwise Martingale Control

Proposition C.24 (High-Probability Control of the Unified Remainder)**.** *Assume the sampling, global-filtration, environment-predictability, and algorithmic-measurability conventions from Subsection C.4.1 and Subsection C.5.5. In particular, at the beginning of episode k, the model $\mathcal{M}^{(k)}$, the empirical objects $\hat{r}^k, \hat{P}^k, b^k$, the optimistic planning objects $\widetilde{Q}^k, \widetilde{V}^k$, and the policy $\hat{\pi}^k$ are measurable with respect to the beginning-of-episode σ-algebra, and the action is sampled according to*

$$\omega_h^k \sim \hat{\pi}_h^k(\cdot \mid \xi_h^k)$$

before the reward and next state are revealed.

Using the remainder notation from Lemma C.21, define

$$\varepsilon_h^k = a_h^k + t_h^k, \qquad M_k = \sum_{h=1}^{H} \varepsilon_h^k.$$

Flatten the episode-stage indices

$$t = (k-1)H + h, \qquad t = 1, \dots, KH.$$

Let $(k(t), h(t))$ denote the inverse episode-stage map, and define

$$\eta_t := \varepsilon_{h(t)}^{k(t)}.$$

Define the flattened filtration as follows. For $t = (k-1)H + h$, let $\mathcal{F}_{t-1}$ contain all randomness completed strictly before the stage-(k,h) outcome is generated, together with the beginning-of-episode k planning objects and the current state ξ_h^k. Let

$$\mathcal{G}_t := \mathcal{F}_{t-1} \vee \sigma(\omega_h^k),$$

and let

$$\mathcal{F}_t := \mathcal{G}_t \vee \sigma(r_h^k, \xi_{h+1}^k).$$

If $h = H$ and $k < K$, we enlarge $\mathcal{F}_t$ further by including ξ_1^{k+1}. This enlargement preserves the increasing-filtration property. Since ξ_1^{k+1} is independent of the completed history and t_h^k depends only on the current transition outcome ξ_{h+1}^k, this enlargement does not affect the conditional zero-mean calculation for t_h^k.

Then there exists an absolute constant $C_{\mathrm{mg}} > 0$, such that the event

$$\mathcal{E}_{\mathrm{mg}} := \left\{ \left| \sum_{k=1}^{K} M_k \right| \le C_{\mathrm{mg}} H^2 \sqrt{KH \log(C_{\log}/\delta)} \right\} \tag{33}$$

satisfies

$$\mathbb{P}(\mathcal{E}_{\mathrm{mg}}) \ge 1 - \delta/2.$$

Proof. We prove the high-probability bound by showing that the flattened sequence η_t is a bounded martingale difference sequence, and then applying Lemma C.11.

Fix $t = (k-1)H + h$. By the measurability note in Subsection C.5.5, $\widetilde{Q}_h^k, \widetilde{V}_h^k, \hat{\pi}_h^k$ are fixed at the beginning of episode k; after the current state ξ_h^k is observed, ξ_h^k is $\mathcal{F}_{t-1}$-measurable. By the environment predictability convention in Subsection C.4.1, the current model objects

$$\{r_h^{(k)}(\xi, \omega), P_h^{(k)}(\cdot \mid \xi, \omega) : h \in [H], \xi \in \Xi, \omega \in \Omega(\xi)\}$$

are also fixed, or at least measurable with respect to the beginning-of-episode k σ-algebra. Since $\hat{\pi}^k$ is constructed at the beginning of episode k, the policy value functions $V_h^{\hat{\pi}^k,(k)}$ are deterministic functions of these beginning-of-episode objects. Hence, after ξ_h^k is included in $\mathcal{F}_{t-1}$, the map

$$\omega \mapsto G_h^k(\xi_h^k, \omega)$$

is $\mathcal{F}_{t-1}$-measurable. Lemma C.21 gives

$$\Gamma_h^k(\xi_h^k) = \sum_{\omega \in \Omega(\xi_h^k)} \hat{\pi}_h^k(\omega \mid \xi_h^k) G_h^k(\xi_h^k, \omega),$$

and therefore $\Gamma_h^k(\xi_h^k)$ is $\mathcal{F}_{t-1}$-measurable. Since

$$\omega_h^k \mid \mathcal{F}_{t-1} \sim \hat{\pi}_h^k(\cdot \mid \xi_h^k),$$

and

$$a_h^k = \Gamma_h^k(\xi_h^k) - G_h^k(\xi_h^k, \omega_h^k),$$

it follows that

$$\mathbb{E}[a_h^k \mid \mathcal{F}_{t-1}] = 0.$$

On the other hand,

$$t_h^k = P_h^{(k)}(\cdot \mid \xi_h^k, \omega_h^k)^\top \Gamma_{h+1}^k - \Gamma_{h+1}^k(\xi_{h+1}^k).$$

Conditioning on $\mathcal{G}_t$, (ξ_h^k, ω_h^k) has been fixed, while sampling model gives

$$\xi_{h+1}^k \mid \mathcal{G}_t \sim P_h^{(k)}(\cdot \mid \xi_h^k, \omega_h^k).$$

Therefore

$$\mathbb{E}[t_h^k \mid \mathcal{G}_t] = 0.$$

By tower property,

$$\mathbb{E}[\eta_t \mid \mathcal{F}_{t-1}] = \mathbb{E}[a_h^k + t_h^k \mid \mathcal{F}_{t-1}] = \mathbb{E}[a_h^k \mid \mathcal{F}_{t-1}] + \mathbb{E}[\mathbb{E}[t_h^k \mid \mathcal{G}_t] \mid \mathcal{F}_{t-1}] = 0.$$

Thus $(\eta_t, \mathcal{F}_t)_{t=1}^{KH}$ is MDS.

We next prove bounded increments. By clipping,

$$0 \le \widetilde{Q}_h^k(\xi, \omega) \le H - h + 1 \le H, \qquad 0 \le \widetilde{V}_h^k(\xi) \le H - h + 1 \le H.$$

By the definition of the optimistic softmax policy in (20) and the optimistic value in (19),

$$\alpha \log \frac{\hat{\pi}_h^k(\omega \mid \xi)}{\pi_{\mathrm{ref}}(\omega \mid \xi)} = \widetilde{Q}_h^k(\xi, \omega) - \widetilde{V}_h^k(\xi).$$

Therefore

$$\left| \alpha \log \frac{\hat{\pi}_h^k(\omega \mid \xi)}{\pi_{\mathrm{ref}}(\omega \mid \xi)} \right| \le H,$$

and regularized reward with respect to

$$\left| r_h^{(k)}(\xi, \omega) - \alpha \log \frac{\hat{\pi}_h^k(\omega \mid \xi)}{\pi_{\mathrm{ref}}(\omega \mid \xi)} \right| \le 1 + H \le 2H.$$

gives

$$|V_h^{\hat{\pi}^k,(k)}(\xi)| \le 2H(H - h + 1) \le 2H^2.$$

This H^2 envelope follows from the clipped Q, V-range and the softmax identity above. The bound is stated in a form sufficient for the variation-dependent sublinearity result used in the main text. Hence

$$|\Gamma_h^k(\xi)| \le |\widetilde{V}_h^k(\xi)| + |V_h^{\hat{\pi}^k,(k)}(\xi)| \le H + 2H^2 \le 3H^2.$$

At the same time,

$$|G_h^k(\xi, \omega)| \le |\widetilde{Q}_h^k(\xi, \omega)| + |r_h^{(k)}(\xi, \omega)| + \left| P_h^{(k)}(\cdot \mid \xi, \omega)^\top V_{h+1}^{\hat{\pi}^k,(k)} \right| \le H + 1 + 2H^2 \le 4H^2, \qquad H \ge 1.$$

because

$$a_h^k = \Gamma_h^k(\xi_h^k) - G_h^k(\xi_h^k, \omega_h^k) = \mathbb{E}_{\omega \sim \hat{\pi}_h^k(\cdot|\xi_h^k)} G_h^k(\xi_h^k, \omega) - G_h^k(\xi_h^k, \omega_h^k),$$

we get

$$|a_h^k| \le 2 \sup_{\omega \in \Omega(\xi_h^k)} |G_h^k(\xi_h^k, \omega)| \le 8H^2.$$

Similarly,

$$|t_h^k| \le \left| P_h^{(k)}(\cdot \mid \xi_h^k, \omega_h^k)^\top \Gamma_{h+1}^k \right| + |\Gamma_{h+1}^k(\xi_{h+1}^k)| \le 2\|\Gamma_{h+1}^k\|_\infty \le 6H^2.$$

Therefore

$$|\eta_t| = |\varepsilon_h^k| \le |a_h^k| + |t_h^k| \le 14H^2.$$

We subsequently use an absolute-constant H^2 boundedness. Thus we write

$$|\eta_t| \le C_\eta H^2$$

with C_η later absorbed into C_{mg}.

Applying Lemma C.11 to the martingale difference sequence gives

$$\mathbb{P}\left(\left| \sum_{t=1}^{KH} \eta_t \right| \ge u \right) \le 2 \exp\left(-\frac{u^2}{2C_\eta^2 H^4 KH} \right).$$

Take

$$u = C_{\mathrm{mg}} H^2 \sqrt{KH \log(C_{\log}/\delta)}$$

and C_{mg} large enough, gives

$$\mathbb{P}(\mathcal{E}_{\mathrm{mg}}) \ge 1 - \delta/2.$$

Finally,

$$\sum_{t=1}^{KH} \eta_t = \sum_{k=1}^{K} \sum_{h=1}^{H} \varepsilon_h^k = \sum_{k=1}^{K} M_k.$$

This proves the claim. □

C.5.11 Unified High-Probability Event

The analysis uses the following high-probability events. Recall that

$$\mathcal{E}_{\mathrm{conc}} = \mathcal{E}_r \cap \mathcal{E}_P,$$

where $\mathcal{E}_r$ and $\mathcal{E}_P$ are the reward and transition concentration events defined in Subsection C.4.4. By Lemma C.15,

$$\mathbb{P}(\mathcal{E}_{\mathrm{conc}}) \ge 1 - \delta/2.$$

The martingale event is

$$\mathcal{E}_{\mathrm{mg}} := \left\{ \left| \sum_{k=1}^{K} M_k \right| \le C_{\mathrm{mg}} H^2 \sqrt{KH \log(C_{\log}/\delta)} \right\},$$

where M_k is the unified remainder from Lemma C.21. By Proposition C.24,

$$\mathbb{P}(\mathcal{E}_{\mathrm{mg}}) \ge 1 - \delta/2.$$

Finally define

$$\mathcal{E}_{\mathrm{tot}} := \mathcal{E}_{\mathrm{conc}} \cap \mathcal{E}_{\mathrm{mg}}.$$

A union bound gives

$$\mathbb{P}(\mathcal{E}_{\mathrm{tot}}) \ge 1 - \delta.$$

All pathwise regret bounds in Theorem C.1 are proved on $\mathcal{E}_{\mathrm{tot}}$.

C.5.12 Proof of Theorem C.1

Proof of Theorem C.1. By Subsection C.5.11, $\mathbb{P}(\mathcal{E}_{\mathrm{tot}}) \geq 1-\delta$. We fix an outcome in $\mathcal{E}_{\mathrm{tot}}$ and proceed pathwise.

For each episode k, at the realized initial state ξ_1^k decompose:

$$V_1^{\star,(k)}(\xi_1^k) - V_1^{\hat{\pi}^k,(k)}(\xi_1^k) = \big(V_1^{\star,(k)}(\xi_1^k) - \widetilde{V}_1^k(\xi_1^k)\big) + \big(\widetilde{V}_1^k(\xi_1^k) - V_1^{\hat{\pi}^k,(k)}(\xi_1^k)\big).$$

By Proposition C.20,

$$V_1^{\star,(k)}(\xi_1^k) - \widetilde{V}_1^k(\xi_1^k) \leq C_{\mathrm{opt}} H D_k.$$

By Lemma C.22,

$$\widetilde{V}_1^k(\xi_1^k) - V_1^{\hat{\pi}^k,(k)}(\xi_1^k) \leq 2\sum_{h=1}^{H} b_h^k(\xi_h^k, \omega_h^k) + M_k + C_{\mathrm{cert}} H D_k.$$

Here $M_k = \sum_{h=1}^{H} \varepsilon_h^k$ is the unified remainder from Lemma C.21 controlled by Proposition C.24. Summing over episodes and using the pathwise definition of $\mathrm{DynReg}(K)$ gives

$$\mathrm{DynReg}(K) \leq 2\sum_{k=1}^{K}\sum_{h=1}^{H} b_h^k(\xi_h^k, \omega_h^k) + \sum_{k=1}^{K} M_k + C_\star H \sum_{k=1}^{K} D_k,$$

where $C_\star := C_{\mathrm{opt}} + C_{\mathrm{cert}}$.

The three terms are controlled as follows:

1. Proposition C.23 gives the pathwise summed-bonus bound:

$$2\sum_{k=1}^{K}\sum_{h=1}^{H} b_h^k(\xi_h^k, \omega_h^k) \leq 2C_{\mathrm{bon}} H^2 K S_\xi \sqrt{\frac{A \log(C_{\log} S_\xi A H K/\delta)}{W}}.$$

2. Proposition C.24 on $\mathcal{E}_{\mathrm{mg}}$ gives

$$\sum_{k=1}^{K} M_k \leq C_{\mathrm{mg}} H^2 \sqrt{KH \log(C_{\log}/\delta)}.$$

   This has the same logarithmic factor as the third term in (21), so the martingale contribution is absorbed after increasing the absolute constant.

3. Equation (28) gives

$$\sum_{k=1}^{K} D_k \leq W(B_r + HB_p).$$

   Therefore

$$C_\star H \sum_{k=1}^{K} D_k \leq C_\star W H (B_r + HB_p).$$

Renaming absolute constants gives the theorem statement. This proves the claim. □

C.6 From Fixed-Window Regret to the Main-Text Informal Statement

This subsection proves Corollary C.2. The result is a known-variation tuning statement: the window length is selected before the run as a deterministic function of the variation budget, and the fixed-window theorem is then applied to that algorithm instance.

C.6.1 Proof of Corollary C.2

Proof of Corollary C.2. Fix an arbitrary window chosen before the algorithm is run, $W_{\mathrm{orc}} \in \{1, \dots, K\}$. Because the algorithm, empirical windows, bonuses, and policy sequence are then fixed by this window choice, Theorem C.1 applies directly to that algorithm instance and gives

$$\mathrm{DynReg}(K) \le \widetilde{O}\left(H^2 K S_\xi \sqrt{\frac{A}{W_{\mathrm{orc}}}} + W_{\mathrm{orc}} H V_K + H^2 \sqrt{KH} \right)$$

with probability at least $1 - \delta$. Since the oracle may choose any deterministic window before the run, choosing a window that minimizes the deterministic upper-bound function gives the oracle upper envelope.

For a uniform display over all variation regimes, we use the smoothed quantity $V_K + 1$. This affects only constant-level edge cases and yields the same variation-dependent scaling in the non-stationary regimes of interest. In the untruncated case, ignoring constants gives

$$H^2 K S_\xi \sqrt{\frac{A}{W}} \asymp W H (V_K + 1),$$

and hence

$$H K S_\xi \sqrt{A} \asymp W^{3/2} (V_K + 1).$$

Therefore

$$W_\star = \left(\frac{H^2 S_\xi^2 A K^2}{(V_K + 1)^2} \right)^{1/3}.$$

When $1 \le W_\star \le K$, $W_{\mathrm{orc}} \asymp W_\star$, substitution gives

$$H^2 K S_\xi \sqrt{\frac{A}{W_\star}} \asymp H W_\star (V_K + 1) \asymp H^{5/3} S_\xi^{2/3} A^{1/3} K^{2/3} (V_K + 1)^{1/3}.$$

The martingale remainder $H^2 \sqrt{KH}$ gives the tuned-window bound.

For the growing-dimension statement, the first term in (23) is $o(K)$ whenever

$$H^{5/3} S_\xi^{2/3} A^{1/3} K^{2/3} (V_K + 1)^{1/3} = o(K).$$

Cubing both sides and canceling K^2 gives

$$H^5 S_\xi^2 A (V_K + 1) = o(K).$$

Together with $H^2 \sqrt{KH} = o(K)$, this implies $\mathrm{DynReg}(K) = o(K)$. When H, S_ξ, A are fixed, the condition reduces to $V_K = o(K)$, and the martingale term is lower order.

This is a known-variation tuning statement. No simultaneous high-probability guarantee over all window lengths is needed for the stated corollary, because the algorithm is run with a single deterministic window chosen before observing the trajectory. The minimization above is over the deterministic upper-bound expression, not over realized pathwise regrets from multiple window choices. A separate simultaneous guarantee over all window lengths could be obtained by a standard union bound over $W \in [K]$, at the cost of an additional logarithmic factor. This proves the claim. □

D Deployable Executor Instantiation

This appendix specifies the deployable instantiation of MPCR used in the fixed-library transfer experiments. MPCR is implemented as an executor-side wrapper around a base web agent and a frozen learned skill library.

SWOP (analysis)	MPCR (executor)
Reference prior π_{ref}	Fixed skill-library and base-executor preference
Sliding-window recent evidence	Current-page feasibility verification
Optimism for unresolved uncertainty	Soft scoring that downweights rather than filters
KL regularization toward the prior	Reranking constrained to the proposed shortlist

Table 6: Componentwise correspondence between the SWOP analysis model and the MPCR executor.

D.1 Correspondence between SWOP and MPCR

SWOP and MPCR are two views of the same adaptation principle: stay anchored to what the learned library prefers, and let recent execution evidence decide which of those preferences is actionable right now. Table 6 maps each element of the analysis onto the executor stage that realizes it.

This mapping is what fixes the executor's structure, and each row settles a concrete design question. The reference prior is the frozen skill library and base executor, so adaptation may not touch either; every reported gain is therefore attributable to test-time selection rather than to further skill acquisition. Recent evidence enters as current-page feasibility verification, which places adaptation inside a single episode, the timescale on which executability actually changes under environment evolution. Optimism is realized as soft scoring rather than hard filtering: a candidate whose target cannot be confirmed is downweighted but retained, so a partially observed page state cannot discard the action that would have recovered the task. The regularization term is then confined to reranking the proposed shortlist, which limits how far selection may move and keeps the executor from drifting away from the learned preference when the feasibility signal is noisy. The analysis therefore supplies the design: what to hold fixed, what to condition on, how to act when that evidence is inconclusive, and how far to move. Theorem 4.1 characterizes the principle in the idealized model, and the evidence that it carries into browser execution is empirical.

Three results support the mapping, one per row. Appendix D.8.1 shows that its two active halves contribute separately and are not interchangeable: constrained reranking over the shortlist adds 4.31 points of success over the unmodified proposal prior, and feasibility evidence adds a further 2.14 points on top of it. Appendix D.8.2 confirms the regularization row directly, in that enlarging the candidate set dilutes the effective weight of the skill-library preference and monotonically suppresses skill invocation, which is precisely the behavior the regularization term exists to prevent. Appendix E.4 shows that the same pressure-dependent gain pattern reappears when the mapping is instantiated over a structurally different learned library, indicating that it tracks a mechanism rather than a property of one skill format.

D.2 Executor Interface and Execution Loop

At each decision step, MPCR receives the task goal, the current browser state, a summary of progress, the primitive browser actions, the base executor, and the fixed learned skill library. It returns one executable next action. The base executor's top action is not executed directly; it enters the stage sequence of Table 6 as one candidate among the proposed set. The selected action is executed in the browser, and the observed page change, error, blocker, or timeout is incorporated into the next decision step.

Algorithm 2 summarizes the full MPCR execution loop. The loop exposes the four operational stages used throughout this appendix: proposal, verification, scoring, and reranking. It also shows how browser feedback is folded into the next decision step and where fallback behavior is invoked.

The remaining subsections unpack each component of Algorithm 2. Section D.3 specifies the constrained proposal prompt and the structured candidate interface. Section D.4 describes the browser-state evidence used to annotate local feasibility. Section D.5 explains how these annotations are converted into conservative shortlisting scores. Section D.6 specifies the constrained prior-guided reranking prompt. Section D.7 describes the fallback rules that make the executor total under malformed generations, uncertain probes, ambiguous rankings, and weak shortlists.

Algorithm 2 MPCR: Masked Prior-Guided Candidate Reranking

Require: Task goal g, browser state B_0, fixed skill library $\mathcal{L}$, primitive action set $\mathcal{A}_{\text{prim}}$, base executor E, candidate budget N_{cand}, shortlist size M, maximum horizon T

Ensure: Executed trajectory τ

$\tau \leftarrow \emptyset$, $h_0 \leftarrow \emptyset$
for $t = 1, \ldots, T$ **do**
  $o_t \leftarrow \textsc{Observe}(B_{t-1})$
  if $\textsc{TaskComplete}(g, o_t, h_{t-1})$ **then**
    return τ
  end if
  // Phase 1: Propose
  $C_t \leftarrow \textsc{Propose}_E(g, h_{t-1}, o_t, \mathcal{L}, \mathcal{A}_{\text{prim}}, N_{\text{cand}})$
  if C_t is invalid or empty **then**
    $a_t \leftarrow \textsc{SafeRecovery}(o_t, h_{t-1})$
  else
  // Phase 2: Verify
    for all $c \in C_t$ **do**
      $v_t(c) \leftarrow \textsc{AnnotateFeasibility}(c, o_t, B_{t-1})$
    end for
  // Phase 3: Score
    for all $c \in C_t$ **do**
      $s_t(c) \leftarrow \textsc{FeasibilityScore}(c, v_t(c), g, h_{t-1})$
    end for
    $S_t \leftarrow \textsc{TopM}(C_t, s_t, M)$
  // Phase 4: Rank
    $a_t \leftarrow \textsc{PairwiseRank}(S_t, g, h_{t-1}, \{v_t(c), s_t(c)\}_{c \in S_t})$
    if a_t is invalid, ambiguous, or outside S_t **then**
      $a_t \leftarrow \arg\max_{c \in S_t} s_t(c)$
    end if
  end if
  $(B_t, f_t) \leftarrow \textsc{Execute}(B_{t-1}, a_t)$
  $\tau \leftarrow \tau \cup \{(o_t, C_t, a_t, f_t)\}$
  $h_t \leftarrow \textsc{UpdateProgress}(h_{t-1}, a_t, f_t)$
end for
return τ

D.3 Candidate Interface and Proposal

The proposal stage defines the candidate interface used by all downstream MPCR components. It instantiates the $\textsc{Propose}_E$ call in Algorithm 2 with a strict structured-output contract, so that verification, scoring, and ranking can be applied deterministically. At each decision step, the proposal module must return exactly N_{cand} candidates. Each candidate contains a complete executable action string, including both the action name and its arguments, together with a concise rationale grounded in the current observation.

Constrained Candidate-Proposal Prompt

Instruction. Review the current page state, previous execution feedback, and all provided task information to propose the best next actions for accomplishing the goal. The output will be interpreted and executed by a browser-control program, so it must follow the required format exactly.

Inputs. The prompt is instantiated with the following runtime fields:

- task goal;
- current progress summary;
- currently open tabs and active URL;
- current page accessibility tree;
- current page screenshot, when available;
- available primitive actions and fixed-library skill actions;
- previous actions and execution feedback, when available.

Candidate-generation rule. Produce exactly N_{cand} next-action candidates, ranked from best to worst. The candidates may use the same action type with different arguments. The ordering is treated as a proposal prior, but the final action is selected only after verification, scoring, and reranking.

Action constraints. Each candidate must be an executable action string from the provided action space. Do not output an action outside the available primitive or skill actions. Do not invent hidden capabilities, unavailable tools, or unprovided skill calls.

Reason constraint. Each candidate must include a concise reason grounded in the current observation, task goal, and available action space.

Output. Return valid JSON only. Do not output markdown or extra text.

```
{
  "candidates": [
    {
      "action": "click | type | navigate | scroll | skill_call | ...",
      "reason": "brief justification"
    }
  ]
}
```

This contract serves three purposes. First, it bounds the action set considered at each decision step. Second, it makes the candidate set parseable by deterministic verification and scoring modules. Third, it exposes primitive actions and fixed-library skills through the same interface, while preventing the proposal module from bypassing MPCR with an unstructured direct action.

The proposal stage is therefore not a broad search procedure. It is a bounded prior-guided expansion of the base executor's next-action preferences. MPCR preserves several plausible alternatives before committing to one action, but all later decisions remain constrained to the generated candidate set.

D.4 Local Feasibility Annotation

MPCR collects local feasibility evidence from the browser state at the moment of decision. The design goal is to add a small amount of grounded execution evidence without turning the executor into a broad search procedure or a new learning algorithm. The evidence layer is intentionally lightweight: it checks properties that are directly relevant to immediate action feasibility, while leaving final selection to the scoring and ranking stages.

D.4.1 Element-Level Readiness Evidence

For candidates that refer to an interface element, MPCR checks whether the target appears present and actionable in the current page state. The annotation considers whether the element can be found through available identifiers or selectors, whether it is visible, whether it appears enabled, whether it has a meaningful bounding region, whether it is inside the current viewport, and whether it may be blocked by another element.

The annotation is not used as a universal hard filter. Browser state can be incomplete or transient, and some useful actions may require scrolling, focusing, waiting, or navigating before the final target becomes available. MPCR therefore treats element-level evidence as a soft feasibility signal. This keeps the executor conservative without making it brittle.

D.4.2 Viewport and Reachability Evidence

MPCR also records page-level reachability evidence, especially whether the current viewport may omit task-relevant content that is reachable through scrolling. This distinction matters because an element that is not currently visible is not necessarily absent. In long pages, administrative dashboards, product tables, issue lists, and content-management interfaces, relevant controls often appear below the fold or inside scrollable regions.

The reachability annotation helps the executor avoid conflating "not currently visible" with "not available." If the page appears scrollable, an otherwise weak candidate may be interpreted as requiring an intermediate exposure action rather than being discarded. Conversely, if the page offers little additional reachable content, the executor can treat missing or hidden targets as stronger evidence against immediate execution.

This design supports MPCR's conservative adaptation principle: use current browser evidence to reduce obvious execution mistakes, but preserve plausible recovery paths when the evidence indicates that the page state may be incomplete.

D.5 Conservative Feasibility Scoring

The scoring stage is a bounded and reproducible intermediate selection layer. Its role is not to define an optimal policy, but to provide a transparent conservative prior over immediate executability before the final pairwise comparison. This makes the ranker compare candidates that have already been grounded in the current page state, rather than asking it to resolve semantic relevance, feasibility, and risk from scratch.

For each candidate $c \in C_t$, MPCR computes a heuristic feasibility-aware score:

$$s_t(c) = \lambda_{\text{rel}} R_t(c) + \lambda_{\text{prog}} P_t(c) + \lambda_{\text{ready}} E_t(c) + \lambda_{\text{expose}} X_t(c) + \lambda_{\text{stable}} U_t(c) - \lambda_{\text{risk}} Z_t(c),$$

where $R_t(c)$ measures task relevance, $P_t(c)$ estimates immediate progress, $E_t(c)$ captures current-page readiness, $X_t(c)$ captures whether the action may expose required information or controls, $U_t(c)$ captures stability of the action structure or grounding evidence, and $Z_t(c)$ captures execution risk. The coefficients are fixed across evaluations.

The evidence families have the following interpretations:

- **Task relevance** measures whether the candidate is semantically aligned with the user instruction and the current progress state.
- **Progress** estimates whether the candidate is likely to move the task forward immediately.
- **Readiness** measures whether the target appears executable from the current page state.
- **Exposure** rewards actions that reveal information, controls, or page regions needed for later progress.
- **Stability** captures whether the candidate is supported by robust identifiers, reliable action structure, or unambiguous grounding evidence.
- **Risk** penalizes actions that are destructive, irreversible, speculative, or likely to move the agent away from the task.

The scoring rule also accounts for page-region and readiness cues in a conservative way. Actions in task-relevant regions, such as main content, navigation, forms, or management panels, receive more favorable treatment than actions in low-value regions. Candidates that appear ready now are preferred over candidates that appear blocked or require unresolved preconditions. However, these cues are deliberately soft. A candidate marked as `blocked` or `needs_scroll` is downweighted, but it may still survive if it remains highly task-relevant and no stronger alternative exists.

This soft treatment is central to MPCR's design. Under environment evolution, feasibility evidence is informative but imperfect. A hard filter can remove the correct recovery action when the page state is partially observed; an unconstrained prior can overcommit to an action that is locally infeasible. Conservative scoring occupies the middle ground: it reduces the priority of fragile candidates while preserving enough action-domain coverage for the ranker to make a context-sensitive comparison.

The scoring rule is fixed across the reported evaluations. This is important for attribution. MPCR's gains are not explained by per-condition tuning or by a hidden adaptive learner; they arise from applying the same feasibility-aware selection interface across the fixed learned library.

D.6 Constrained Prior-Guided Reranking

The final ranking step uses the LLM as a constrained local comparator over the shortlist S_t, following the broader use of LLMs for reranking and pairwise comparison (Sun et al., 2023; Qin et al., 2024; Wu et al., 2025). The ranker is conditioned on the task state, candidate rationales, heuristic scores, and feasibility annotations. It must choose among the provided candidates only.

This constrained design preserves the prior-guided nature of MPCR. The proposal stage reflects the prior induced by the base executor and fixed skill library, while the verification and scoring stages expose current-page feasibility evidence. The ranker then compares a small shortlist using both sources of information. It is not allowed to invent a new action, call an unseen skill, or change the candidate action strings.

In the reported experiments, the shortlist size is set to $M = 2$, so the pairwise comparator is invoked once per decision step. For larger shortlists, the same comparator can be applied in a deterministic tournament order sorted by the heuristic score. The current winner is compared against the next candidate, and ties are broken by the heuristic score. This keeps reranking bounded while ensuring that the final action is selected only from the verified shortlist.

Constrained Pairwise Reranking Prompt

Role. You are a strict pairwise ranker for web-agent actions.

Objective. Choose the better next single action right now. Prefer the action that is more executable, more directly useful for the task, more likely to unblock progress, and lower risk.

Decision criteria.

1. Prefer the candidate that better advances the current task goal.
2. Prefer the candidate that appears executable in the current page state.
3. Prefer actions that reveal required information or remove blockers.
4. Avoid destructive, irreversible, or speculative actions unless required.

Executability evidence. Treat a candidate as weaker if its target is missing, invisible, disabled, occluded, or outside the reachable page state. For skill calls or multi-step candidates, compare them using their signatures, descriptions, current context, and available feasibility annotations.

Candidate constraint. Choose only among the provided candidates. Do not invent a new action, call an unseen skill, assume hidden skill effects, or change the candidate action strings.

Tie rule. Return `tie` only when neither candidate clearly dominates. If one action is moderately better in immediate usefulness or executability, choose it.

Output. Return strict JSON only:

```
{"winner":"a"|"b"|"tie","why":"<=80 words"}
```

The pairwise prompt is intentionally local. It asks only which candidate is better as the next action under the current browser state. This prevents the ranker from replacing the executor with an open-ended planner and keeps MPCR aligned with its role as a test-time reranking mechanism.

D.7 Totality, Fallbacks, and Recovery

MPCR includes explicit recovery behavior so that the executor remains well-defined under parser uncertainty, incomplete probes, or ranker ambiguity. These fallbacks are safety guards, not additional adaptation mechanisms. They ensure that the executor always returns an action, while preserving the fixed-library and candidate-constrained design.

- **Generation failure.** If candidate generation returns invalid JSON or an unusable candidate set, the executor follows a safe default recovery action.
- **Probe uncertainty.** If a candidate cannot be confidently verified, it is retained but downweighted through readiness-related penalties.
- **Ranking ambiguity.** If the pairwise ranker returns invalid or ambiguous output, the executor selects the highest-scored candidate from the heuristic stage.
- **Weak shortlist.** If all shortlisted candidates appear blocked or insufficiently ready, the executor still returns the highest-scored candidate, ensuring that the action-selection procedure remains total.

The guiding principle is conservative continuity. MPCR uses additional deliberation when feasibility signals are informative, but degrades to deterministic behavior when structured judgments are noisy. This makes the deployable executor robust to malformed model outputs, incomplete browser probes, transient page states, and ambiguous comparisons without introducing skill relearning or unconstrained exploration.

D.8 Design Validation

The preceding subsections specify what MPCR does. This subsection provides the empirical basis for two design decisions embedded in that specification: that each of the four stages is necessary, and that the default candidate budget is $N_{\text{cand}} = 2$.

Both studies are run on Magento with GPT-5 mini over the same fixed ASI library and the same six evaluation conditions used in Section 5. Because the main tables pool GitLab, Magento, and WordPress, absolute values here are not comparable to Table 2; full MPCR averages 39.18% on this single-site subset against 41.26% pooled across three sites. All comparisons below are made within this subset.

D.8.1 Component Ablation

We ablate the verification, scoring, and reranking stages of Algorithm 2, keeping the proposal stage in every variant since it defines the candidate interface. Table 7 reports success rate and average skill calls, averaged over the six conditions.

Variant	Ver.	Sco.	Rer.	SR (%)	ASC
Propose only	✗	✗	✗	32.73	0.60
Verify + Score	✓	✓	✗	27.93	0.15
Rerank only (blind)	✗	✗	✓	37.04	0.68
Verify + Score + Random	✓	✓	rand.	8.09	0.31
MPCR (full)	✓	✓	✓	**39.18**	0.24

Table 7: Component ablation of MPCR on Magento, averaged over the six evaluation conditions. "Ver.", "Sco.", and "Rer." denote the verification, scoring, and reranking stages. "Rerank only" applies the pairwise comparator to the raw proposal order without feasibility evidence, and therefore issues the same number of reranking calls as full MPCR. "Verify + Score + Random" keeps the verified shortlist but replaces the comparator with a fixed-seed random choice, and therefore issues no reranking call at all.

Additional deliberation is not what produces the gain. The randomized control is decisive on this point. It shares the pipeline, the feasibility annotations, and the shortlist with full MPCR, and differs only in that the final comparison is a fixed-seed coin flip that emits no reranking call. Success collapses from 39.18% to 8.09%, far below the 32.73% obtained by taking the base executor's first proposal and doing nothing else. Since the scaffolding is held constant and the compute is strictly lower, the gain cannot be attributed to the pipeline structure or to spending an additional model call per step. What matters is the informational content of the comparison. The same control also shows that a bounded shortlist is not self-improving: selecting arbitrarily within it is much worse than not shortlisting at all.

The gain decomposes across stages. Adding blind reranking to the proposal prior raises success from 32.73% to 37.04%, and adding feasibility evidence on top of it raises success further to 39.18%. The second increment, 2.14 points, is smaller than the first but consistent, holding in five of the six conditions. Neither stage subsumes the other: full MPCR exceeds both "Rerank only," which has the comparator but no feasibility evidence, and "Verify + Score," which has feasibility evidence but no comparator.

Feasibility evidence mainly buys reliable skill use. Reading success alone understates what the verification and scoring stages contribute. Full MPCR attains its higher success while issuing 0.24 skill calls per task against 0.68 for blind reranking, a reduction of roughly threefold. The effect concentrates exactly where it should: under Execution Shift, full MPCR reduces skill invocation to 0.04 calls per task, that is, it correctly stops calling a skill that the runtime blocker has made unexecutable, whereas blind reranking continues to invoke and fail it. The feasibility mask is therefore better described as a mechanism for avoiding infeasible skill execution than as a primary driver of success.

Why feasibility evidence alone underperforms. "Verify + Score" reaches only 27.93%, below the 32.73% of the unmodified proposal prior. We read this as a property of the design rather than as a defect in the evidence. Feasibility annotations are constructed to be consumed by the comparator, which weighs them against task context; taking a hard top-1 by heuristic score discards that contextual judgment and commits to whichever candidate the fixed coefficients happen to favor. The conservative scoring rule of Section D.5 is deliberately soft for this reason. That the evidence helps when it informs the comparator and hurts when it replaces it is direct evidence that the two stages are complementary rather than separable.

D.8.2 Candidate-Budget Selection

MPCR is evaluated with $N_{\text{cand}} = 2$, where the candidate count equals the shortlist size, $M = N_{\text{cand}}$. We sweep $N_{\text{cand}} \in \{2, 3, 4\}$ to establish that this default is an empirical choice rather than an under-resourced configuration. Table 8 reports success rate and average skill calls under each of the six conditions, and Table 9 pools the same results by shift type.

Aggregate success is non-monotone and peaks at the default. Overall success is 39.18%, 35.92%, and 37.73% for $N_{\text{cand}} = 2, 3, 4$. Enlarging the candidate set does not improve transfer on aggregate, which establishes the default as an empirically favorable lightweight operating point rather than a budget-

Condition	N_{cand}=2		N_{cand}=3		N_{cand}=4	
	SR	**ASC**	**SR**	**ASC**	**SR**	**ASC**
`Src`	51.75	0.35	44.23	0.08	48.08	0.01
`Src-Perc`	56.14	0.26	40.38	0.12	50.96	0.07
`Src-Exec`	30.70	0.39	42.31	0.05	41.35	0.03
`Tgt`	40.35	0.18	36.84	0.04	32.46	0.00
`Tgt-Perc`	37.72	0.23	31.58	0.01	30.70	0.00
`Tgt-Exec`	18.42	0.04	20.18	0.02	22.81	0.01
Mean	**39.18**	0.24	35.92	0.05	37.73	0.02

Table 8: Per-condition success rate (%) and average skill calls under candidate budgets $N_{\text{cand}} \in \{2, 3, 4\}$ on Magento.

Pooled view	N_{cand}=2	N_{cand}=3	N_{cand}=4
Non-execution	**46.49**	38.26	40.55
Execution	24.56	31.25	**32.08**
Overall	**39.18**	35.92	37.73

Table 9: Success rate (%) pooled by shift type. "Non-execution" pools `Src`, `Src-Perc`, `Tgt`, and `Tgt-Perc`; "Execution" pools `Src-Exec` and `Tgt-Exec`.

constrained compromise. Given a single run per budget, we do not draw conclusions from the exact ordering of $N_{\text{cand}} = 3$ against $N_{\text{cand}} = 4$.

Budget interacts with shift type. The aggregate conceals the more informative pattern, shown in Table 9. Under Execution Shift, larger budgets help: pooled success rises from 24.56% to 31.25% to 32.08%, and `Tgt-Exec` increases monotonically from 18.42% to 22.81%. Under clean and perceptual conditions, larger budgets hurt: pooled success falls from 46.49% to 40.55%. The interaction runs in the same direction on the source and target sides independently, which makes it the most defensible conclusion of this sweep. It is also the reading our framing predicts. When local executability is fragile, additional candidates give the feasibility-aware ranker more room to recover; when the learned prior is already reliable, additional candidates mostly introduce distractors.

Enlarging the candidate set dilutes the skill prior. Skill invocation collapses monotonically with the budget, from 0.24 to 0.05 to 0.02 calls per task. Since the library and the proposal interface are unchanged, this indicates that a wider candidate set reduces the effective weight of the skill-library preference, so the agent progressively stops invoking skills it has already learned. This is the empirical counterpart of the KL regularization toward π_{ref} in SWOP and of the correspondence stated in Table 6. It also explains the interaction above: on clean and perceptual conditions the diluted prior was worth keeping, so dilution costs success, whereas under Execution Shift the prior is unreliable in the first place and the trade is favorable. Since the favorable direction of this trade-off flips with shift type, a fixed budget cannot be optimal for both regimes; a shift-adaptive candidate budget, widened under detected execution pressure and kept narrow otherwise, is a promising extension we leave to future work.

E Experimental Details and Supplementary Analyses

This appendix provides supplementary details for the experiments in Section 5. We first describe the compared learned-skill methods and their implementation protocols, then report descriptive skill-library statistics after learning, and provide backbone-level diagnostic results showing that the observed transfer patterns are not tied to a single LLM backbone. The remaining subsections report supplementary analyses around the main comparison: transfer of MPCR to a second skill-learning method, stability across random seeds, robustness to more aggressive Execution Shift schedules, and a comparison against relearning the skill library in the target environment. These supplementary studies are run on Magento, whereas the main tables pool GitLab, Magento, and WordPress; absolute values are therefore not comparable across the two, and each subsection states its scope.

E.1 Compared-Method Cards

The compared published skill-learning baselines are AWM, ASI, SkillWeaver, and WALT (Wang et al., 2025b,a; Zheng et al., 2025; Prabhu et al., 2026). These four systems provide representative coverage of design choices in skill representation, integration mechanism, learning signal source, and verification

rigor. Concretely, they cover textual, executable, API-function, and tool-based skill forms; prompt injection, action-space extension, and tool-call integration; and episode-driven, exploration-driven, and function-discovery learning signals.

The four published baselines also differ in their underlying agent frameworks and action-selection strategies (Wang et al., 2025b,a; Zheng et al., 2025; Prabhu et al., 2026). They observe page state through DOM, accessibility-tree, or snapshot representations, but differ in execution stack and decision mechanism. Such heterogeneity is an inherent consequence of comparing published systems: each method is tightly coupled to its native execution stack, and re-implementing all methods within a single framework would risk distorting their original design intent. Our aim is therefore to evaluate representative end-to-end skill-learning systems under shared tasks and perturbation conditions. The disturbance layer is applied consistently across backends, although this does not remove all execution-stack confounds. We provide detailed method cards below, focusing on each method's skill representation, acquisition signal, verification mechanism, execution interface, and relevance to the fixed-library transfer comparison.

AWM (Agent Workflow Memory) represents learned procedures as *textual workflows* stored in agent memory (Wang et al., 2025b). Each workflow consists of a high-level natural-language description and a sequence of steps, where each step records the environment state, the agent's reasoning, and the executed action. Workflows are induced by prompting the LLM to extract common sub-routines from successful episodes, abstracting concrete values into variables (e.g., replacing a specific product name with `{product_name}`). At inference time, relevant workflows are retrieved and injected into the LLM prompt as contextual guidance. Because workflows are consumed as text rather than executed as code, AWM does not extend the agent's action space. It therefore serves as a prompt-level workflow-reuse baseline rather than an executable-skill baseline.

ASI (Inducing Programmatic Skills for Agentic Tasks) represents skills as *executable Python programs*. Given a successful episode, a skill induction module generates one or more Python functions (e.g., `search_product(name)`) that encapsulate reusable sub-procedures (Wang et al., 2025a). Each induced skill undergoes a three-stage verification: (1) *correctness*—the task is still completed when the original trajectory is rewritten to invoke the new skill; (2) *skill usage*—the rewritten trajectory actually calls the skill; and (3) *skill validity*—every skill invocation causes an observable environment change (Wang et al., 2025a). Only skills that pass all three checks are added to the skill library. During evaluation, induced skills are available as callable high-level actions alongside the primitive action set.

SkillWeaver acquires skills through a *self-driven curriculum*: the agent autonomously proposes candidate skills by observing the website, practices them through active exploration, distills successful execution traces into reusable Playwright API functions, and iteratively debugs these functions through automated test cases (Zheng et al., 2025). Unlike ASI and AWM, which learn only from task-solving episodes, SkillWeaver generates learning opportunities through environment exploration independent of the benchmark task set. The resulting skills are Python `async` functions that operate on Playwright `Page` objects and are composed hierarchically, with higher-level skills calling lower-level ones (Zheng et al., 2025). SkillWeaver therefore represents an exploration-driven API-function approach to skill acquisition.

WALT (Web Agents that Learn Tools) takes a fundamentally different approach by *reverse-engineering website-native functionality* into structured, callable tools. WALT first discovers candidate website functions through systematic exploration, then applies a *demonstrate–generate–validate* cycle: a browser agent demonstrates the function end-to-end, a tool-generation agent converts the execution trace into a structured tool definition with typed input parameters and step sequences, and a test agent validates the tool against pre-selected inputs (Prabhu et al., 2026). When possible, WALT further optimizes tools by replacing UI interaction sequences with direct URL-level or API-level operations (Prabhu et al., 2026). The resulting tools are deterministic action scripts with formal schemas (Prabhu et al., 2026).

E.2 Skill-Library Supplements

Table 10 reports the number of skills (or workflows/tools) retained in each method's library after the learning phase, broken down by LLM backbone and website. This table serves as a descriptive diagnostic of the learning regimes rather than a larger-is-better comparison. The counted units are not semantically identical across methods: an AWM item is a textual workflow, an ASI item is an executable Python skill,

Method	DeepSeek-V3.2			GPT-5 mini			Claude Haiku 4.5		
	GitLab	Magento	WordPress	GitLab	Magento	WordPress	GitLab	Magento	WordPress
AWM	35	23	5	27	15	6	10	6	3
ASI	14	19	12	9	13	4	11	13	10
SkillWeaver	28	18	23	9	26	15	12	17	14
WALT	8	8	6	6	7	5	5	6	3

Table 10: Skill library size after learning. Counts are descriptive rather than directly comparable across methods because the retained units have different semantics.

a SkillWeaver item is a Playwright-style API function, and a WALT item is a structured tool (Wang et al., 2025b,a; Zheng et al., 2025; Prabhu et al., 2026). Therefore, the table is not used to normalize methods by raw library size. Instead, it shows how different skill-acquisition mechanisms expose different amounts and types of reusable structure before transfer evaluation.

Several patterns are worth noting. First, library size varies substantially by method. Averaged over the nine website–backbone combinations, SkillWeaver produces the largest libraries (18.0 items), followed by AWM (14.4), ASI (11.7), and WALT (6.0). This ordering reflects the methods' acquisition mechanisms. SkillWeaver actively explores the environment and can therefore accumulate many candidate API functions. AWM extracts reusable textual routines from successful trajectories, which also permits relatively large libraries when successful episodes contain repeated sub-procedures. In contrast, ASI and WALT apply stronger executable or tool-level validation, yielding smaller but more constrained libraries. WALT is especially compact, ranging only from 3 to 8 tools across all settings, because its demonstrate–generate–validate pipeline filters candidate tools before evaluation.

Second, the table shows that library scale is strongly affected by both the backbone and the website. DeepSeek-V3.2 generally produces more retained items than GPT-5 mini or Claude Haiku 4.5, especially for AWM and SkillWeaver. Across all methods and websites, the total number of retained items is 199 for DeepSeek-V3.2, 142 for GPT-5 mini, and 110 for Claude Haiku 4.5. This suggests that acquisition scale and transfer robustness should be distinguished. Library size measures how many reusable items are retained after learning, whereas transfer performance also depends on executor-side selection, grounding, and execution under changed environment conditions.

Third, website structure also affects how much reusable behavior can be extracted. Aggregating over methods and backbones, GitLab and Magento yield substantially larger libraries than WordPress, with 174, 171, and 106 retained items respectively. This is consistent with the nature of the environments: developer-platform and e-commerce-administration tasks expose more repeated navigation, filtering, editing, and object-management patterns, whereas the WordPress task set induces a smaller set of reusable operations. This site-level variation is useful for SkillShift-Bench because it prevents the benchmark from being reduced to a single library-density regime. The same transfer protocol is evaluated under websites that differ in how much reusable structure they naturally expose.

Most importantly, Table 10 helps rule out a simple "more skills imply better robustness" explanation. SkillWeaver often produces the largest libraries, while WALT uses the smallest libraries because its tools are more heavily validated. ASI occupies an intermediate regime: it learns fewer items than AWM or SkillWeaver in many settings, but retains executable skills that are directly usable during interaction. These patterns show that library size mainly reflects acquisition scale and method design, rather than transfer robustness itself. A larger library may provide more reusable candidates, but successful transfer still depends on whether the agent can select, ground, and execute the retained items under changed environment conditions.

E.3 Backbone Analyses

Table 11 provides a backbone-level decomposition of the transfer results and shows that the benchmark stress is not an artifact of a single LLM choice. GPT-5 mini obtains the highest overall success (SR = 35.23%), the lowest skill failure rate (SFR = 13.52%), and the lowest trajectory cost (AS = 8.77).

Metric	LLM	Method				Environment						O/A
		AWM	ASI	Skill-Weaver	WALT	Src	Src-Perc	Src-Exec	Tgt	Tgt-Perc	Tgt-Exec	
SR (%)	DS	35.30	**38.32**	17.12	36.44	**47.89**	37.32	23.98	35.81	29.76	16.01	31.80
	GPT	**37.56**	37.90	**24.99**	**40.46**	46.30	**45.41**	**27.14**	**38.72**	**33.58**	**20.23**	**35.23**
	Cld	33.00	32.64	18.32	27.50	45.07	40.60	14.87	32.03	24.73	9.89	27.86
AS	DS	7.35	6.94	17.53	15.90	8.65	10.45	14.35	11.25	12.03	14.84	11.93
	GPT	**6.20**	**6.21**	**12.80**	9.87	**7.25**	**7.50**	11.31	**7.51**	**8.56**	10.49	**8.77**
	Cld	7.47	7.51	15.05	**6.62**	8.12	8.81	**10.67**	8.15	8.96	**10.25**	9.16
ASC	DS	–	0.32	1.89	0.03	0.55	0.69	0.76	0.80	0.73	0.94	0.74
	GPT	–	0.06	2.16	0.18	0.74	0.72	0.80	0.77	0.85	0.92	0.80
	Cld	–	0.11	0.00	0.05	0.07	0.06	0.07	0.05	0.05	0.03	0.05
SFR (%)	DS	–	39.51	0.21	**19.25**	5.09	19.71	33.38	**7.10**	18.89	33.78	19.66
	GPT	–	**13.86**	5.71	20.98	**4.15**	**7.46**	**21.26**	11.77	16.68	**19.77**	**13.52**
	Cld	–	30.71	**0.00**	32.15	10.89	21.39	32.05	19.96	**16.08**	25.36	20.95

Table 11: Expanded transfer-oriented diagnostic results across methods and environment settings. O/A denotes the overall average across the corresponding settings. LLM abbreviations are as follows: DS = DeepSeek-V3.2, GPT = GPT-5 mini, and Cld = Claude Haiku 4.5 (DeepSeek-AI, 2025; OpenAI, 2025; Anthropic, 2025). Bold values mark the best SR, AS, and SFR entries. ASC is reported as a non-directional diagnostic. "–" denotes non-applicable metrics: AWM uses textual workflows, so executable-skill metrics are undefined.

DeepSeek-V3.2 remains competitive in overall success (SR = 31.80%), but requires substantially longer trajectories (AS = 11.93), suggesting stronger exploration or recovery at a higher interaction cost. Claude Haiku 4.5 has lower overall success (SR = 27.86%) and more frequent skill failures (SFR = 20.95%), indicating less stable transfer under shifted execution conditions.

The environment columns show a consistent degradation pattern across all three backbones. Clean source-context performance is highest or near-highest for each model, but success decreases under target-context transfer and drops most sharply under execution perturbations. For example, GPT-5 mini decreases from 46.30% on `Src` to 38.72% on `Tgt`, and further to 27.14% on `Src-Exec` and 20.23% on `Tgt-Exec`. DeepSeek-V3.2 and Claude Haiku 4.5 follow the same qualitative trend, with `Tgt-Exec` reaching only 16.01% and 9.89%, respectively. Thus, stronger backbones improve the absolute level of transfer, but do not remove the difficulty ordering induced by Contextual Shift, Perceptual Shift, and especially Execution Shift.

The trajectory-cost and skill-failure rows further separate two failure channels. Execution-shift settings increase the interaction burden for all backbones, with AS on `Tgt-Exec` reaching 14.84 for DeepSeek-V3.2, 10.49 for GPT-5 mini, and 10.25 for Claude Haiku 4.5. Skill failures also become more frequent in the same setting, with SFR reaching 33.78%, 19.77%, and 25.36%, respectively. This indicates that Execution Shift does not only make tasks longer. It also makes selected skills or tool actions less locally executable. GPT-5 mini is comparatively more robust because it combines higher success with lower SFR, rather than relying on longer trajectories.

The method columns reveal additional interactions between backbone capability and skill format. SkillWeaver remains the weakest method under all three backbones, although GPT-5 mini improves its success to 24.99%, compared with 17.12% for DeepSeek-V3.2 and 18.32% for Claude Haiku 4.5. WALT benefits strongly from GPT-5 mini and reaches 40.46% success, but its SFR remains nontrivial at 20.98%, showing that structured tools can still face local executability failures. ASI is competitive with both DeepSeek-V3.2 and GPT-5 mini, but its skill failures are much less frequent with GPT-5 mini (SFR = 13.86%) than with DeepSeek-V3.2 (39.51%) or Claude Haiku 4.5 (30.71%). This suggests that the same executable-skill framework can behave differently depending on the backbone's ability to select, ground, and recover from skill executions.

Finally, skill-call frequency should be interpreted together with success and SFR, rather than as a standalone performance metric. GPT-5 mini and DeepSeek-V3.2 invoke executable skills at similar overall frequencies (ASC = 0.80 and 0.74), but GPT-5 mini achieves higher success and lower SFR, indicating more reliable skill use. Claude Haiku 4.5 has much lower skill-call frequency (ASC = 0.05), but this does not imply stronger robustness. Together with its lower success, it suggests limited effective use of the learned executable-skill interface. Overall, Table 11 supports the main conclusion that stronger LLMs improve absolute transfer performance, while the benchmark's degradation pattern remains stable across backbones.

E.4 Generalization to a Second Skill-Learning Method

MPCR is evaluated in the main paper over ASI's executable skill library. To test whether its benefit depends on that particular skill format, we apply it to AWM, which reuses prompted textual workflows rather than executable skill invocations and for which the executable-skill diagnostics in Table 1 are undefined. The two methods therefore differ in what a "candidate" is: for ASI a skill candidate is a callable action in the extended action space, whereas for AWM the retrieved workflow enters the prompt as guidance and the executable candidates are the primitive actions it recommends. MPCR is attached at the same point in both cases, namely the proposal interface of Section D.3: the proposal stage enumerates the primitive actions suggested by the retrieved workflow together with alternatives from the base executor, and verification, scoring, and reranking then operate unchanged. The workflow library itself remains frozen, exactly as the skill library does for ASI.

Condition	AWM	AWM + MPCR	Rel.
`Src`	32.46	41.23	+27.0%
`Src-Perc`	31.00	40.35	+30.2%
`Src-Exec`	13.27	20.35	+53.4%
`Tgt`	49.11	57.02	+16.1%
`Tgt-Perc`	32.46	39.47	+21.6%
`Tgt-Exec`	10.00	18.42	+84.2%
Average	**28.05**	**36.14**	**+28.8%**

Table 12: Success rate (%) of AWM with and without MPCR on Magento. "Rel." is the relative gain over the AWM baseline.

Table 12 reports the result on Magento. Two observations follow. First, the gain holds in all six conditions, with no condition in which the reranker degrades the base method. Because AWM's integration mechanism is architecturally different from ASI's, this indicates that feasibility-aware test-time selection is not tied to the executable-skill format. Second, and more informatively, the pressure-dependent pattern reported in Section 5.3 reappears independently: relative gains are largest under Execution Shift, at $+53.4\%$ on `Src-Exec` and $+84.2\%$ on `Tgt-Exec`, well above the clean and perceptual conditions. The correspondence in Table 6 therefore predicts *where* the gain concentrates on a second method whose library it was not derived from, which is stronger evidence for the underlying mechanism than the aggregate improvement alone.

E.5 Stability across Random Seeds

To characterize how much of the MPCR gain is attributable to run-to-run variation, we evaluate MPCR and the fixed-library ASI baseline over three seeds on Magento across all six conditions. The three seeds comprise the run reported in the main paper plus two additional seeds; this is a variance characterization of the reported configuration rather than an independent replication. Since the main tables pool three sites, the absolute values in Table 13 are not comparable to Table 2, and we compare only within this subset.

Cross-seed variation is small for both methods, with per-condition standard deviations at or below 2.32 points, and the mean MPCR success exceeds the mean ASI success in every condition. Pooled over Magento, MPCR improves success by 18.9% relative to ASI. The gain is therefore stable rather than

Condition	MPCR	ASI	Rel.
Src	49.12 ± 2.32	43.27 ± 1.34	+13.5%
Src-Perc	53.80 ± 2.03	38.30 ± 1.34	+40.5%
Src-Exec	29.24 ± 1.34	17.54 ± 0.88	+66.7%
Tgt	41.52 ± 1.34	40.06 ± 0.51	+3.6%
Tgt-Perc	36.84 ± 0.88	36.55 ± 1.34	+0.8%
Tgt-Exec	19.01 ± 0.51	17.25 ± 1.01	+10.2%
Mean	**38.26**	**32.16**	**+18.9%**

Table 13: Success rate (%), mean $\pm$ standard deviation over three seeds on Magento. "Rel." is the relative gain of MPCR over the ASI baseline.

seed-driven, and the direction and magnitude are consistent with the three-site result reported in the main paper.

The distribution of the gain is as informative as its size. The three largest improvements are all on the source side, and Src-Exec shows the largest relative improvement at $+66.7\%$, matching the ordering predicted by our local-executability account. The two near-zero cells, Tgt and Tgt-Perc, are equally consistent with it: in those conditions the fixed library already transfers well, so there is little executability mismatch left for test-time selection to recover. Finally, MPCR is not worse than ASI in any condition, so feasibility-aware selection helps where the learned prior loses feasibility and is harmless where the prior remains aligned.

E.6 Robustness to Repeated and Randomized Interruption

The Execution Shift conditions in the main paper use the once injection regime of Appendix B.2.3, under which a dismissed blocker is not reinstated. This subsection asks whether the fixed-library reranking advantage survives more aggressive schedules. We evaluate MPCR and ASI under all three regimes on Magento, in both the source and the target context.

Regime	Context	MPCR	ASI	Rel.
once	Src	30.70	16.96	+81.0%
	Tgt	18.42	16.67	+10.5%
multiple	Src	20.35	17.95	+13.4%
	Tgt	8.77	7.86	+11.6%
random	Src	30.97	26.95	+14.9%
	Tgt	20.18	18.18	+11.0%

Table 14: Success rate (%) under the three Execution Shift injection regimes on Magento. "Rel." is the relative gain of MPCR over the ASI baseline.

The comparative conclusion is robust to the schedule. MPCR exceeds ASI in all six regime–context cells, so the advantage is not an artifact of the single-dismissal design: it persists when the blocker returns after every navigation and when its timing is randomized.

Absolute success, by contrast, is clearly sensitive to injection frequency, and in a direction that reinforces the benchmark's framing. The multiple regime is uniformly the hardest, reducing MPCR from 30.70% to 20.35% in the source context and from 18.42% to 8.77% in the target context, while random is the mildest because probabilistic injection sometimes produces no interruption at all. The once setting used in the main results is thus a conservative instantiation of Execution Shift, and more aggressive schedules strengthen rather than weaken the finding that Execution Shift exerts the strongest pressure among the three modes.

The margin does narrow under the most aggressive schedule, from $+81.0\%$ to $+13.4\%$ in the source context. We read this as a limit of the mechanism rather than as an anomaly. When the blocker is

reinstated after every clearance, the environment variation that SWOP measures as V_K is large, and the variation-dependent guarantee of Theorem 4.1 degrades accordingly: transfer-time action selection alone has limited headroom against an obstruction that reappears faster than it can be recovered from. Complementary recovery mechanisms, which act on the interruption rather than only on the choice of action, are a natural direction for future work.

E.7 Comparison with Target-Environment Re-Learning

A natural alternative to test-time adaptation is to relearn the skill library directly in the evolved environment. We evaluate this on Magento by running ASI's induction procedure in `Tgt`, so that the induced library is matched to the target interface, and then testing that target-native library under `Tgt-Perc` and `Tgt-Exec`, against the fixed source library used by ASI and MPCR.

Method	`Tgt-Perc`	`Tgt-Exec`
ASI (fixed `Src` library)	36.84	16.67
Re-learned in `Tgt`	35.09	14.91
MPCR (same fixed library)	**37.72**	**18.42**

Table 15: Success rate (%) of target-environment re-learning against fixed-library transfer under the two shifted target conditions on Magento.

Re-learning a static library does not address runtime executability mismatch. Under Perceptual Shift the target-native library is the weakest of the three methods (35.09%), and under Execution Shift it falls to 14.91%, below not only MPCR (18.42%) but also the fixed source library it was meant to replace (16.67%). MPCR exceeds it by 7.5% relative under Perceptual Shift and by 23.5% relative under Execution Shift, and the margin widens exactly where local executability is the binding constraint. A modal blocker injected at deployment is a per-episode runtime event, and no library induced before deployment, however well matched to the target interface, can encode a procedure for it.

The comparison is not confounded by the adaptation budget. Re-learning pays its cost once, during induction, and leaves inference unmodified thereafter, so the target-native library is not a compute-starved configuration. Its collapse under Execution Shift therefore reflects the limits of the static-library paradigm rather than an insufficient re-learning budget.

Re-learning in the target environment therefore does not remove the need for test-time adaptation. It requires target rollouts, and the library it yields is still fixed at deployment, so it inherits the same runtime feasibility gap that it was introduced to close. MPCR requires no target training data and recovers exactly the execution-shift loss that a re-learned library leaves open.

E.8 Compute Budget and Infrastructure

All experiments were conducted in self-hosted browser environments, with LLM inference accessed through hosted API services. We evaluate DeepSeek-V3.2, GPT-5 mini, and Claude Haiku 4.5. DeepSeek-V3.2 is an open-weight MoE model with 685B total parameters, while the exact parameter counts of GPT-5 mini and Claude Haiku 4.5 are not publicly disclosed by their providers. Since inference was performed through APIs, we did not use local GPU training or fine-tuning. The local infrastructure was used for browser orchestration, logging, and automatic grading. MPCR used a fixed lightweight reranking configuration with at most 2 candidate actions per step and a top-2 shortlist, without hyperparameter search beyond the candidate-budget sweep reported in Appendix D.8.2.